

模块9 中央处理器



本章主要内容

- CPU的组成和功能
- 数据通路
- 指令周期与时序
- 硬布线控制器设计
- MIPS单周期CPU设计及实验
- 微程序控制器设计。



6.1.1 CPU主要功能

■ 取指令并执行指令的部件-----CPU

取指令 ———— 操作控制、时序控制 ———— → 执行指令

◆ **数据加工：** 算术/逻辑运算

运算器

◆ **程序控制：** 程序中指令执行顺序的控制

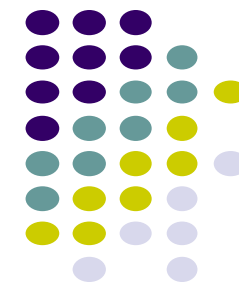
◆ **操作控制：** 将机器指令翻译成执行部件所需**操作控制信号**

◆ **时序控制：** 控制操作信号的产生时间、持续时间

◆ **异常控制：** 异常处理，外设交互。

控制器

6.1.2 CPU的基本组成 (重)



中央处理器**CPU** = 运算器 + **cache** + 控制器。

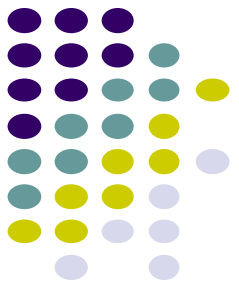
(1) 运算器

- **ALU**
- 通用寄存器组
- 数据缓冲寄存器 **DR**
- 状态字寄存器 **PSW**

(2) **cache**: 指令**cache**、数据**cache**.

(3) 控制器

控制器组成: 程序计数器**PC**、指令寄存器、数据缓冲器、地址寄存器、通用寄存器、状态寄存器、时序发生器、指令译码器、内部总线 (数据通路)。



CPU中主要寄存器

■ PC (Program Counter)-----程序计数器 X86: **EIP** MIPS:**PC**

- 用来存放将要执行的下一条指令的地址
- 顺序执行时, 每执行一条指令, PC的值应 加 1
- 要改变程序执行顺序的情况时, 一般由转移类指令将转移目标地址送往PC, 可实现程序的转移.

■ GR (General Register) -----通用寄存器组

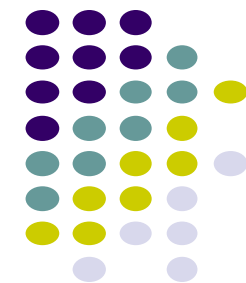
■ IR (Instruction Register)-----指令寄存器 [可选]

- 指令寄存器 用来存放从存储器中取出的待执行的指令
- 在执行当前指令的过程中, 指令寄存器的内容不允许发生变化, 以保证实现指令的全部功能。

CPU中主要寄存器

- AR (Address Register)-----地址寄存器 MAR [可选]
- DR (Data Register)-----数据缓冲寄存器 MDR [可选]
- AC (Accumulate Count)-----累加寄存器 [可选]
- PSW (Program Status Word)----- 状态字寄存器 (保存由算术逻辑运算指令、测试结果等建立的各种条件标志, 比如 溢出标志等) [可选]。

控制器 (操作控制器 和 时序产生器)



(1) **操作控制器**: 为数据通路的建立提供各种操作信号, 把有关数据打入到一个寄存器, 从而完成取指令和执行指令的控制。根据设计方法不同, 可分为:

- **硬布线控制器 (时序逻辑型) ---硬件实现**
- **微程序控制器 (存储程序型) ---软件实现**

(2) **时序产生器**: 提供定时和时序信号。

数据通路DataPath (重+难)

6.2, 6.3节

■ 数据通路-----执行部件间传送信息的路径。

◆ 通路的建立由控制信号控制，受时钟脉冲驱动

◆ 不同指令、同一指令在执行的不同阶段的数据通路不同

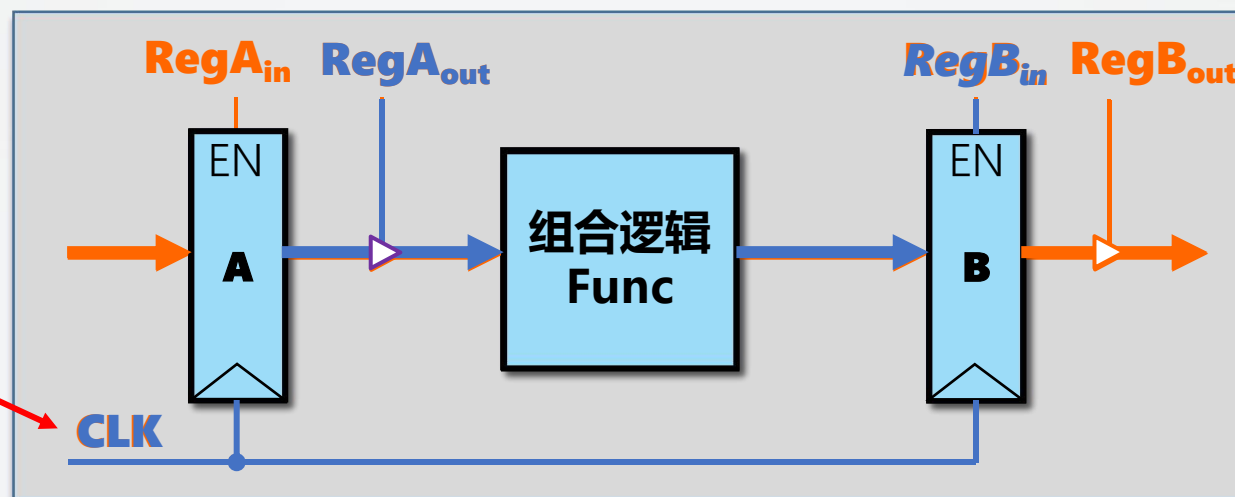
◆ 分类：共享通路 (总线)、专用通路

◆ 指令的执行流程、执行效率不同

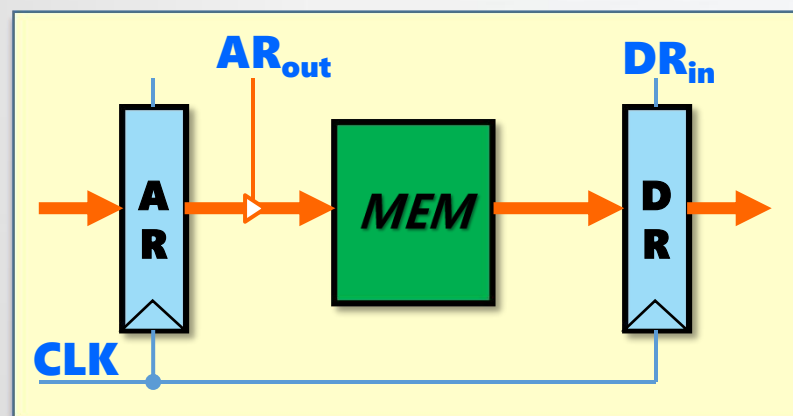
◆ 微操作控制信号的时序安排。

数据通路抽象模型 (例: 寄存器传输)

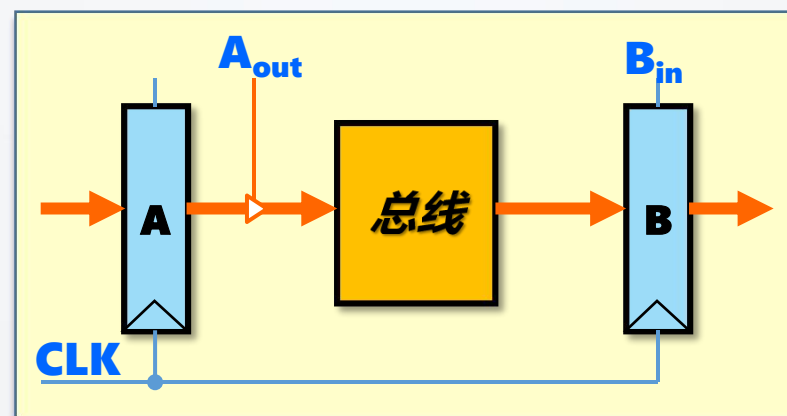
时钟频率



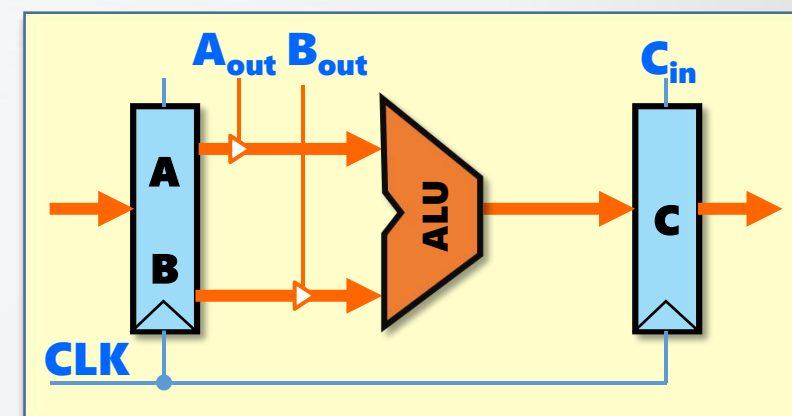
Func : (A) → B



访存通路



总线传输



运算通路

数据通路分类

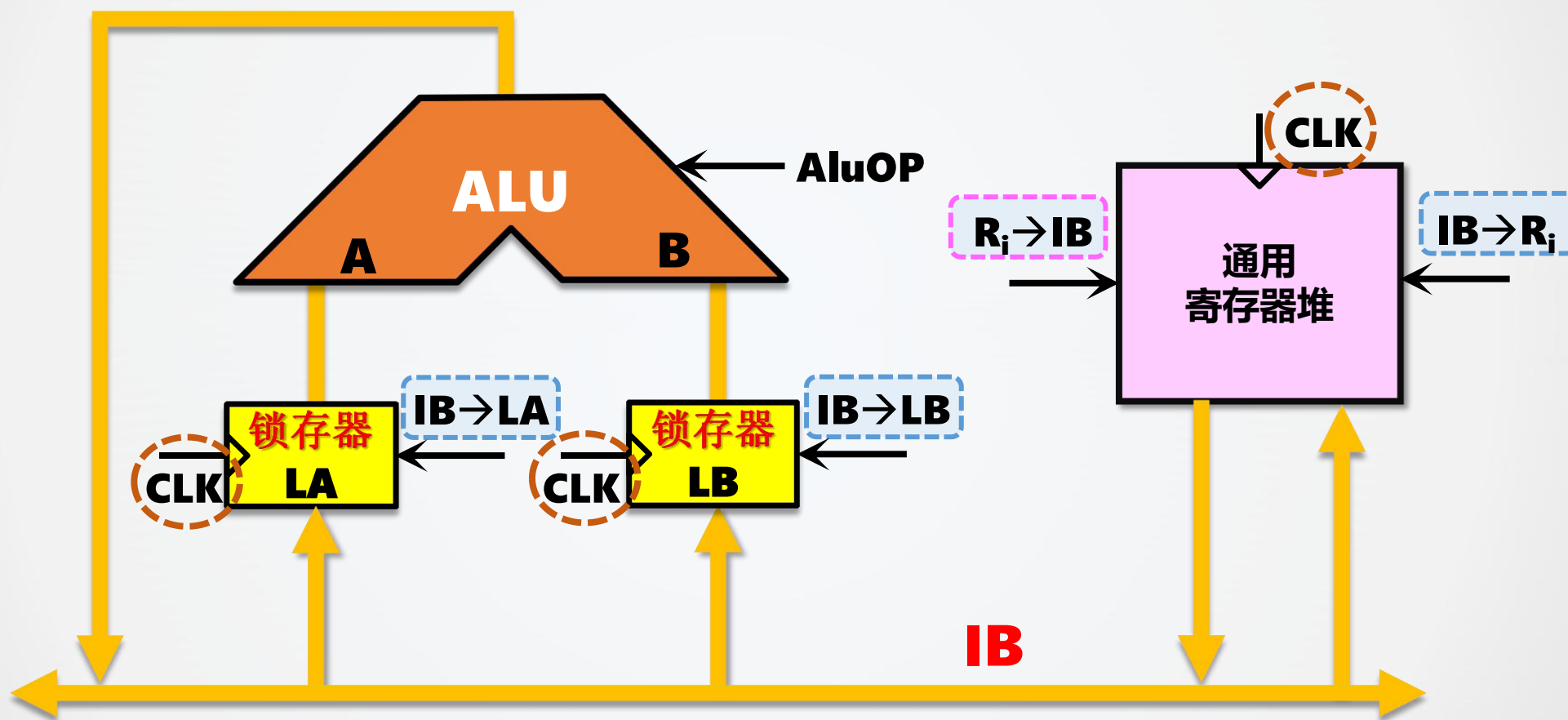
■ 共享通路（总线型）

- ◆ 主要部件都连接在公共总线上，各部件间通过总线进行数据传输
- ◆ 结构简单，实现容易，但并发性较差，需分时使用总线，效率低。

■ 专用通路

- ◆ 并发度高，性能佳；设计复杂，成本高
- ◆ 可以看作多总线结构。

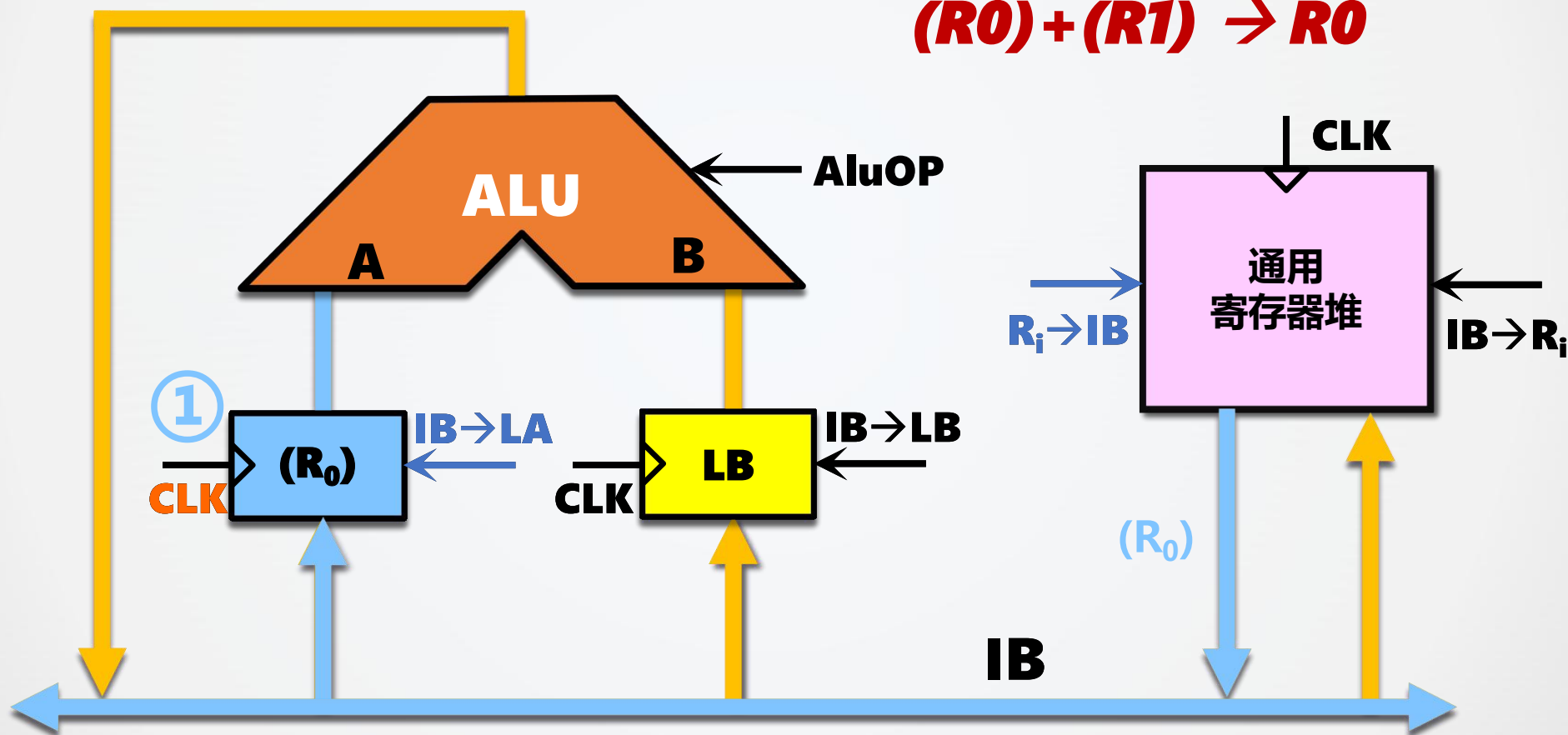
单总线结构中的数据通路



单总线结构中的数据通路

例: **ADD R0,R1**

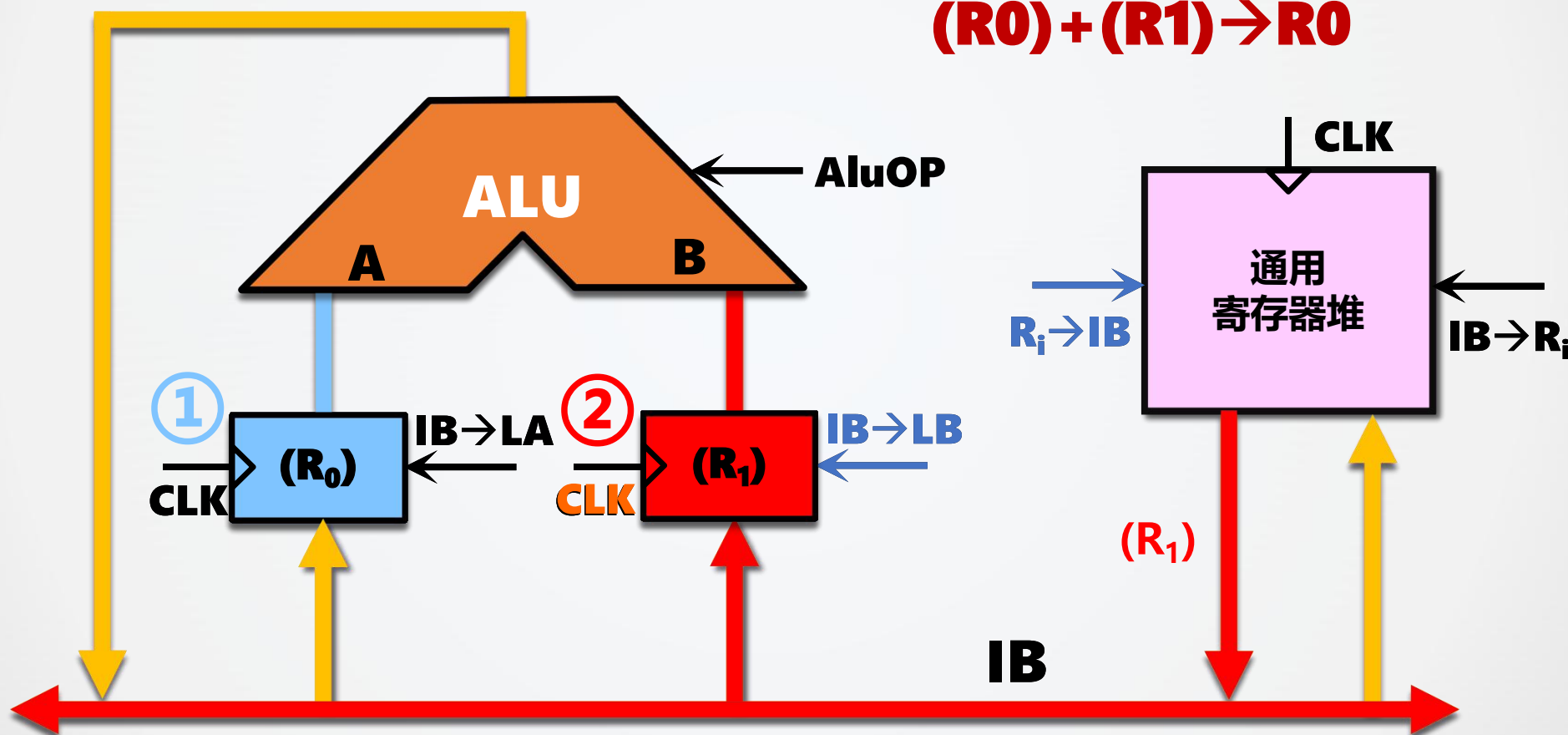
$(R0) + (R1) \rightarrow R0$



单总线结构中的数据通路

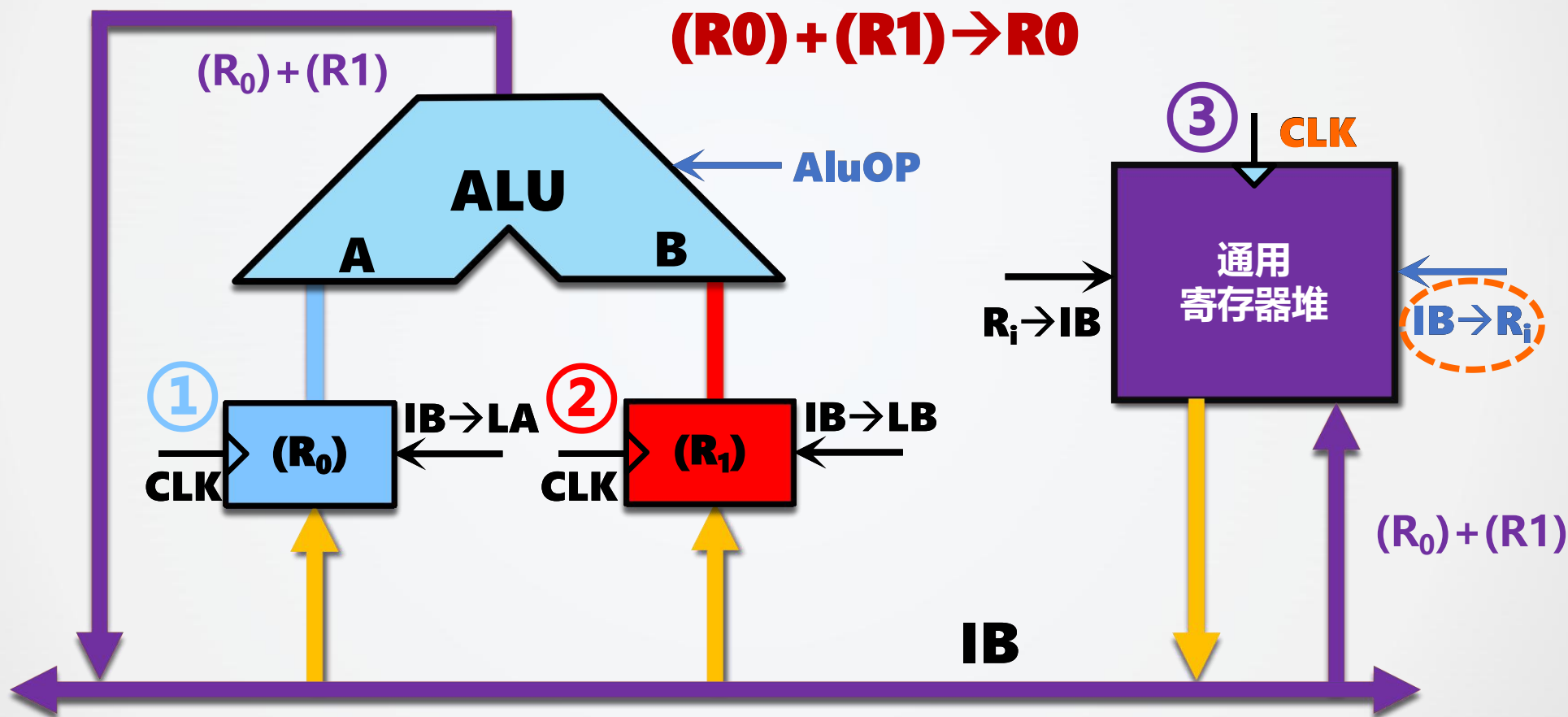
ADD R0,R1

$(R0) + (R1) \rightarrow R0$



单总线结构中的数据通路

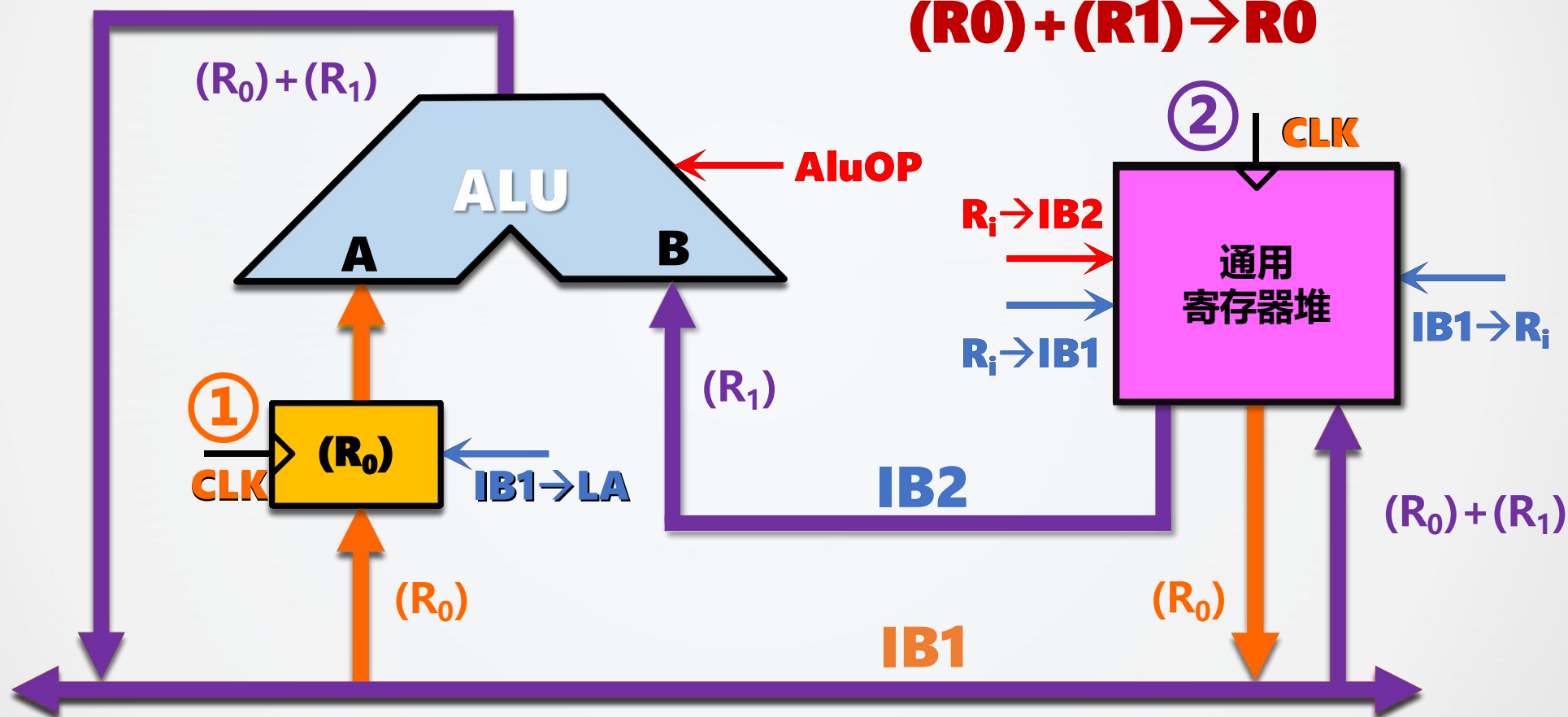
■ 单总线：两个锁存器，**3**个时钟周期



双总线结构与运算通路

ADD R0,R1

$(R_0) + (R_1) \rightarrow R_0$



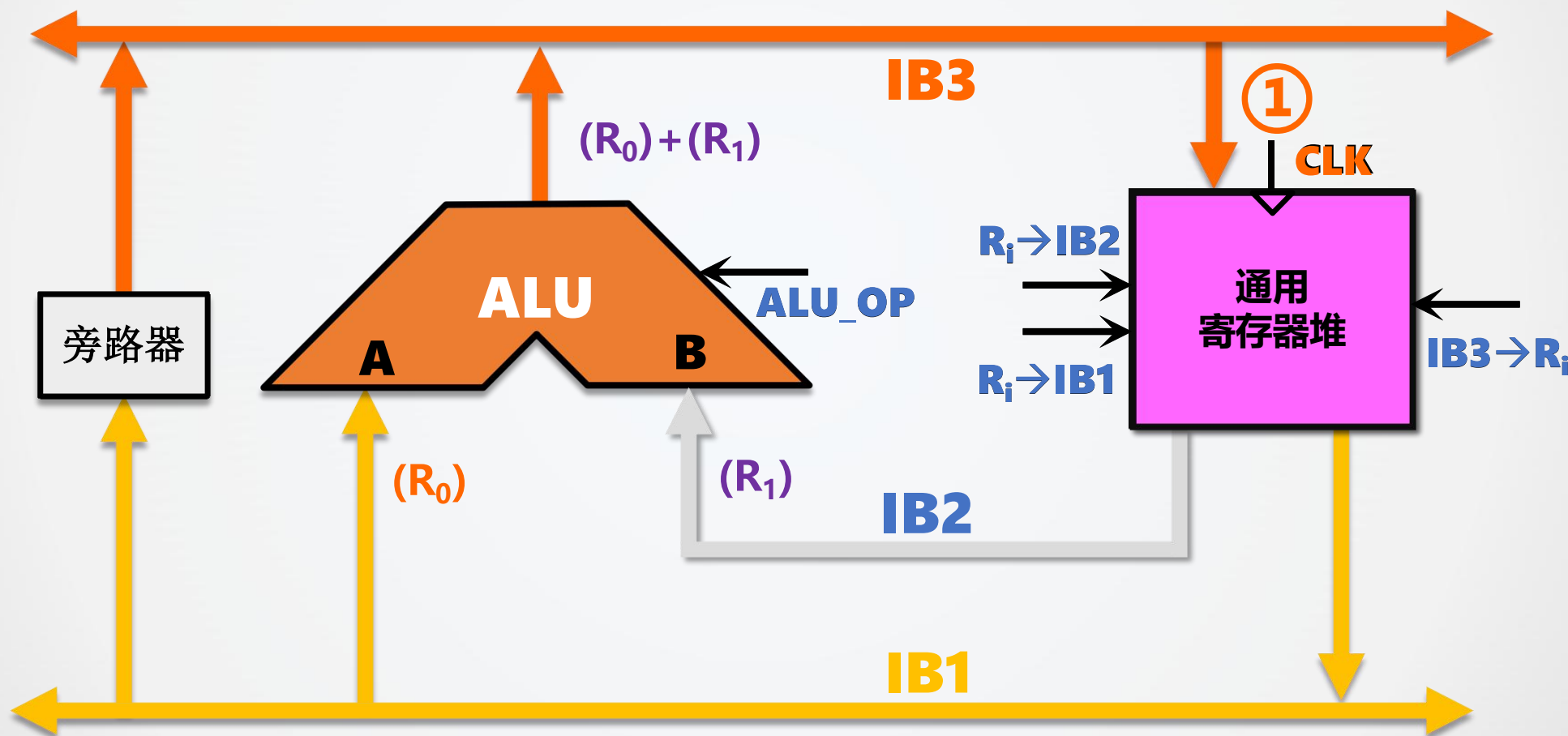
■ 双总线：1个锁存器，2个时钟周期。

三总线结构与运算通路

ADD R0,R1

$(R_0) + (R_1) \rightarrow R_0$

■ 三总线：0个锁存器，1个时钟周期。



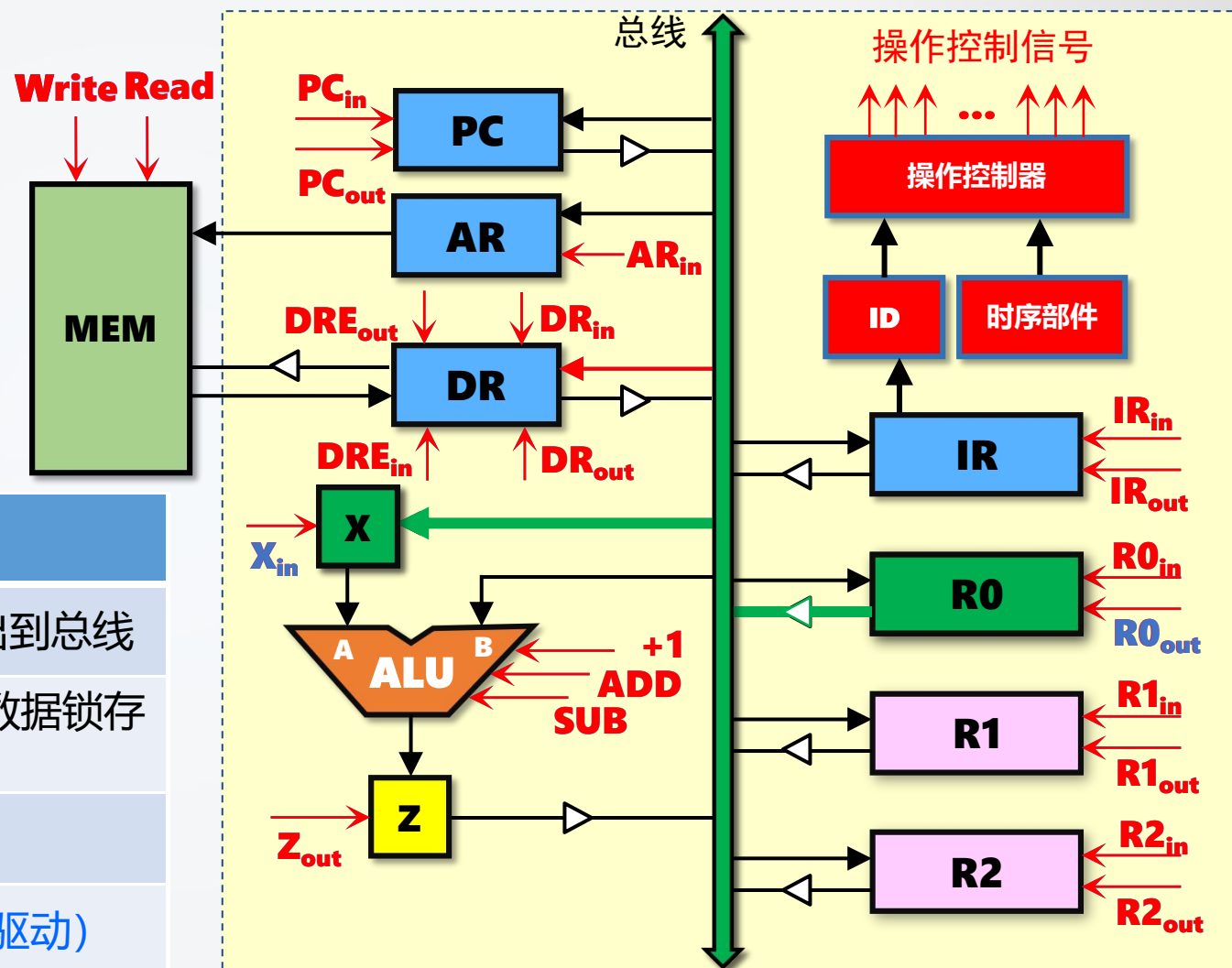
数据通路小结 例 ***ADD R0,R1***

- 单总线：2个锁存器，3个时钟周期
- 双总线：1个锁存器，2个时钟周期
- 三总线：0个锁存器，1个时钟周期
- 总线越多，性能越好。

单总线结构CPU实例

- 主要部件都连接在总线上
- 各部件间通过总线进行传输

控制信号	作用说明
IR_{out} 、 PC_{out} 、 $\dots R1_{out}$	控制三态门将寄存器值输出到总线
IR_{in} 、 PC_{in} 、 $\dots R1_{in}$	控制寄存器使能端, 将总线数据锁存 (时钟驱动)
$+1$ 、 ADD 、 SUB	运算器控制信号
$Write$ 、 $Read$	主存读写控制信号 (时钟驱动)

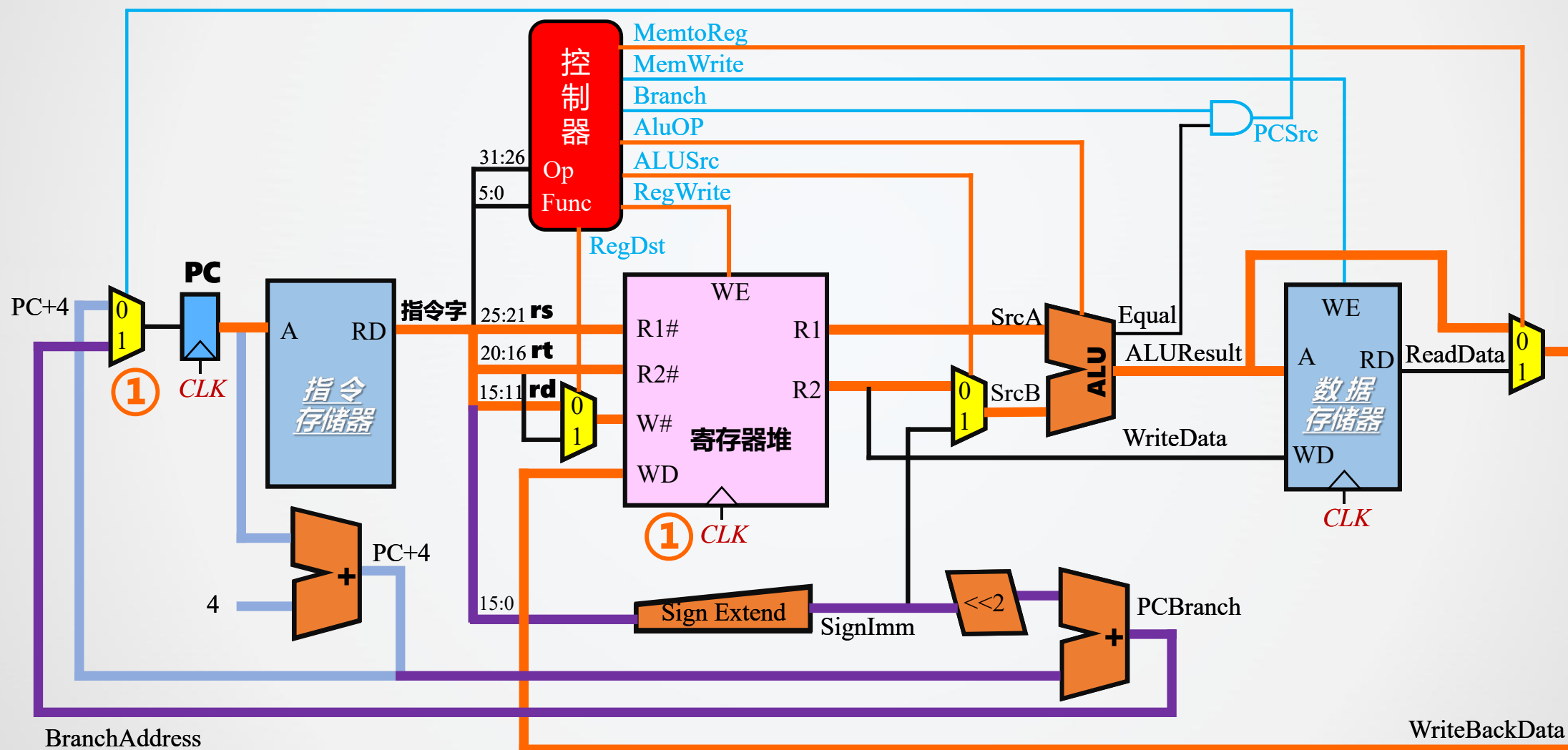


—→: 数据流 —→: 控制流

专用通路 单周期MIPS CPU

R 型指令

6bits	5bits	5bits	5bits	5bits	6bits
OP	R_s	R_t	R_d	shamt	funct



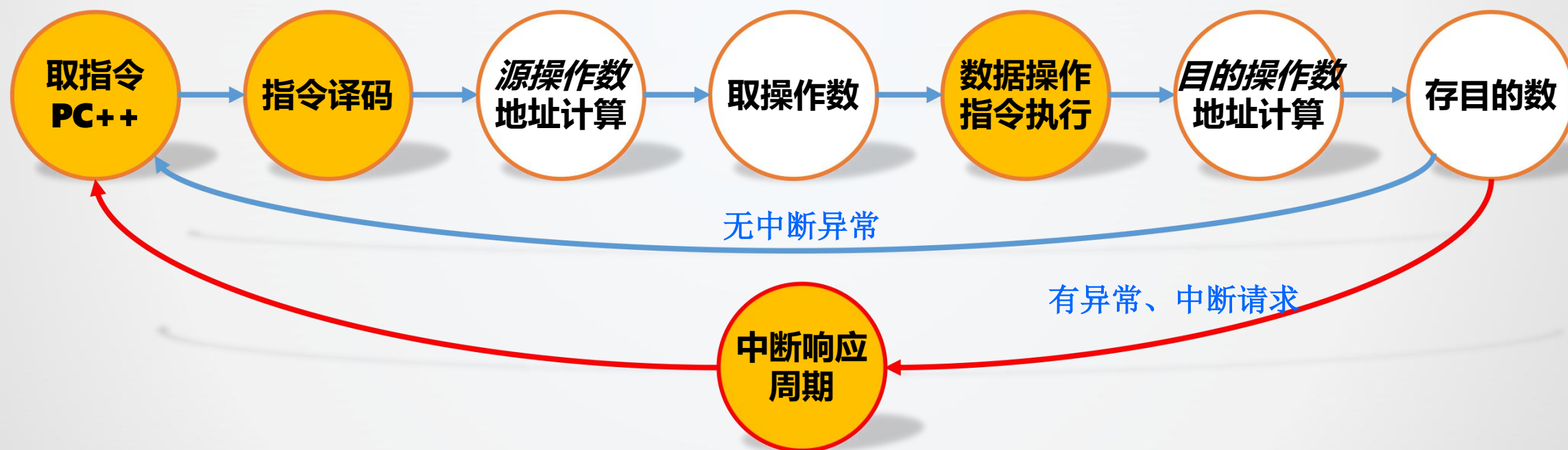
本章主要内容

- CPU的组成和功能
- 数据通路
- 指令周期与时序
- 硬布线控制器设计
- MIPS单周期CPU设计及实验
- 微程序控制器设计。



指令执行一般流程

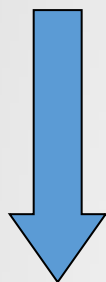
- 取指令、执行指令 反复循环。
- 不同指令功能不同，则数据通路不同，执行时间不同，**如何安排时序？(按指令周期)**
- 访存指令、寄存器运算指令、加法指令 等。



指令周期基本概念

短

长



■ 时钟周期 = 节拍脉冲 = 震荡周期

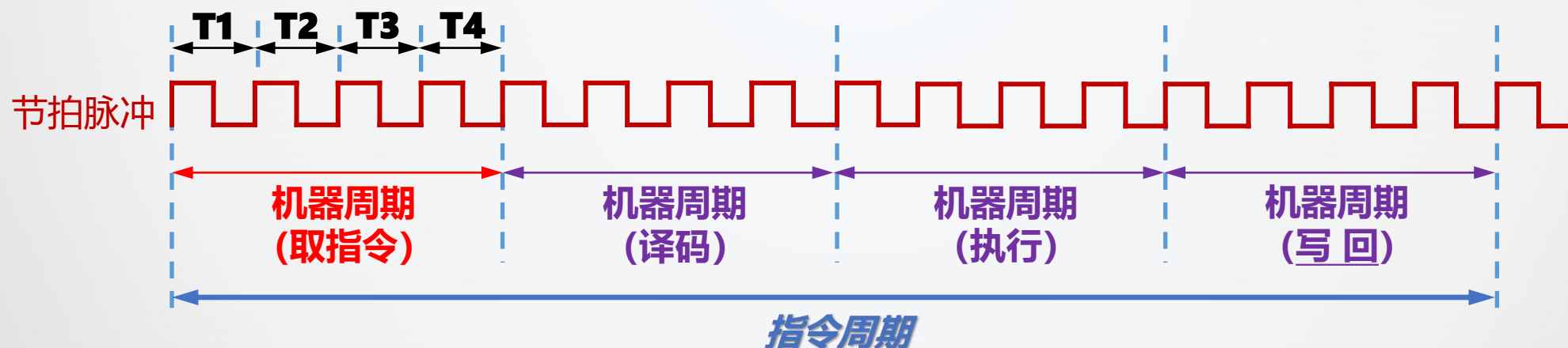
能完成一次微操作

■ 机器周期 = CPU周期。从主存读出一条指令的最短时间。

可完成复杂操作

■ 指令周期：从主存取出一条指令并执行指令的时间

◆ 指令周期 由若干 机器周期 组成; 机器周期 包含若干 时钟周期。

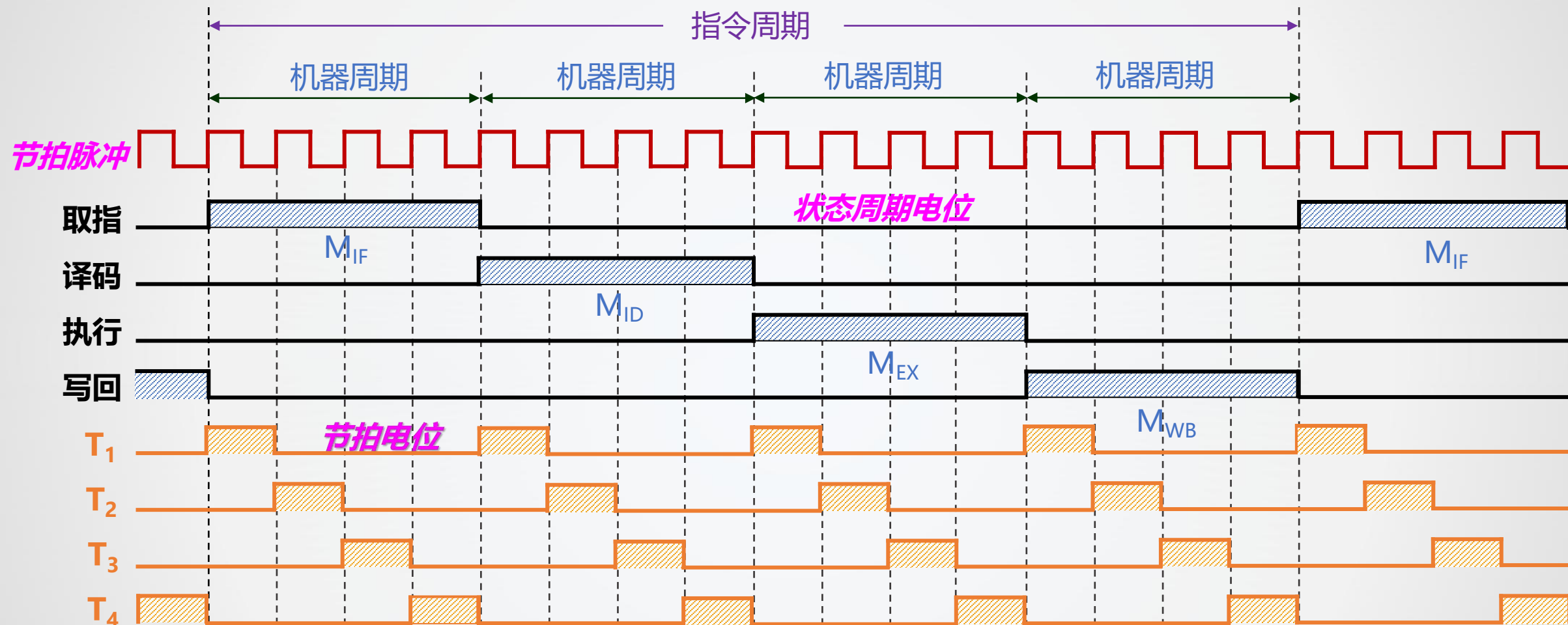


指令周期同步

- 指令周期含的机器周期数 以及每个机器周期所含时钟周期数 不一定是固定的，和具体实现有关，分为2种
 - ◆ 定长指令周期： 同步控制 （早期三级时序系统）
 - ◆ 机器周期数固定， 节拍数固定， 按机器周期同步， 例：MIPS单周期CPU
 - ◆ 变长指令周期： 异步控制 （现代时序系统）
 - ◆ 机器周期数可变， 节拍数可变， 按时钟周期同步， 例：MIPS多周期CPU。



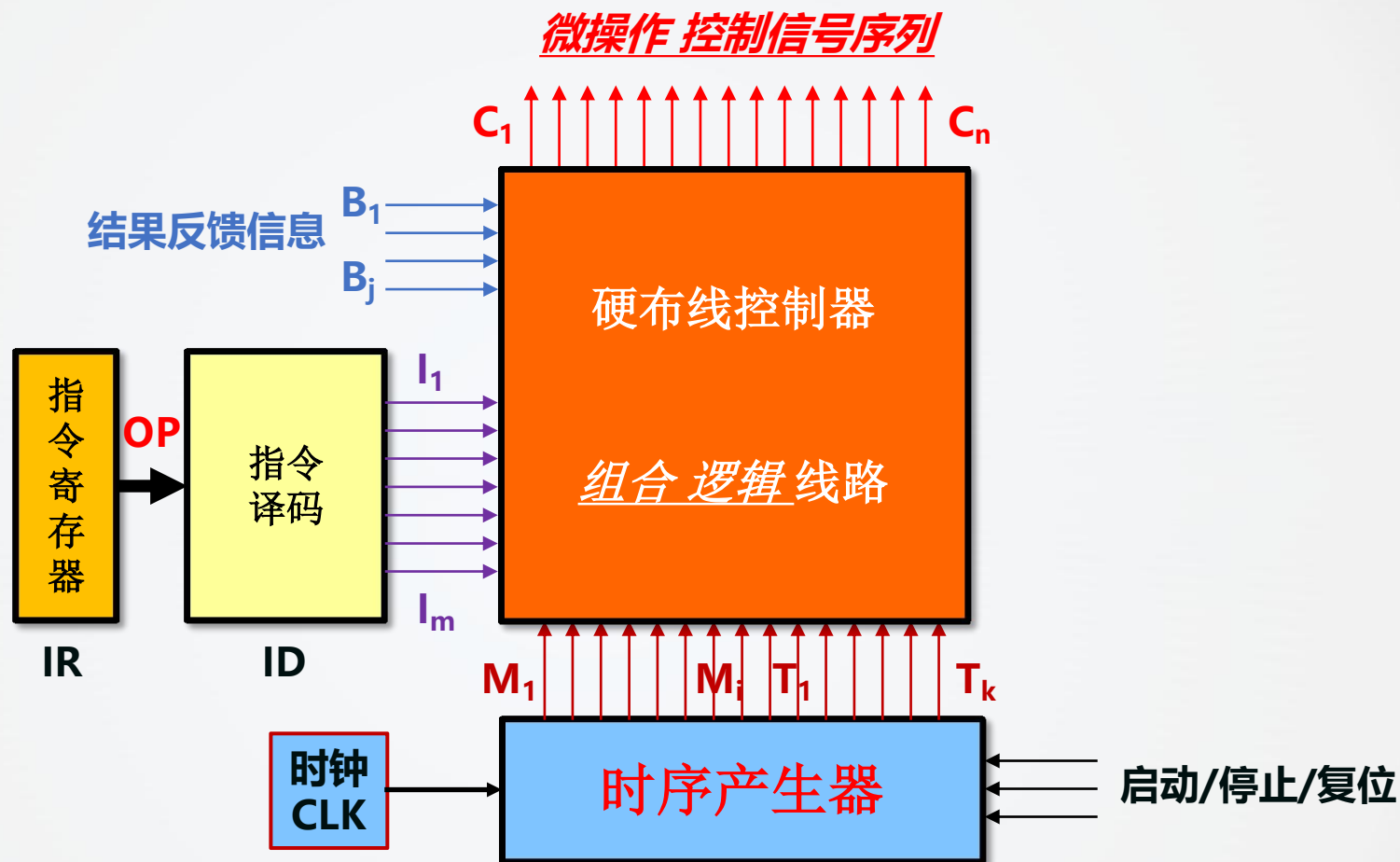
早期 三级时序系统



优势: 时钟脉冲 → 机器周期(状态周期)电位、节拍电位信号; 硬布线 控制器 → 组合逻辑

例 $\text{MemRead} = M_{IF} \cdot (T_2 + T_3) + \text{Load} \cdot M_{EX} \cdot (T_2 + T_3)$

时序产生器与控制器--同步控制



时序产生器 循环产生 状态周期电位、节拍电位, 供控制器对信号进行时间调制

例 $\text{MemRead} = M_{IF} \cdot (T_2 + T_3) + Load \cdot M_{EX} \cdot (T_2 + T_3)$

单总线结构CPU 指令周期 (重)

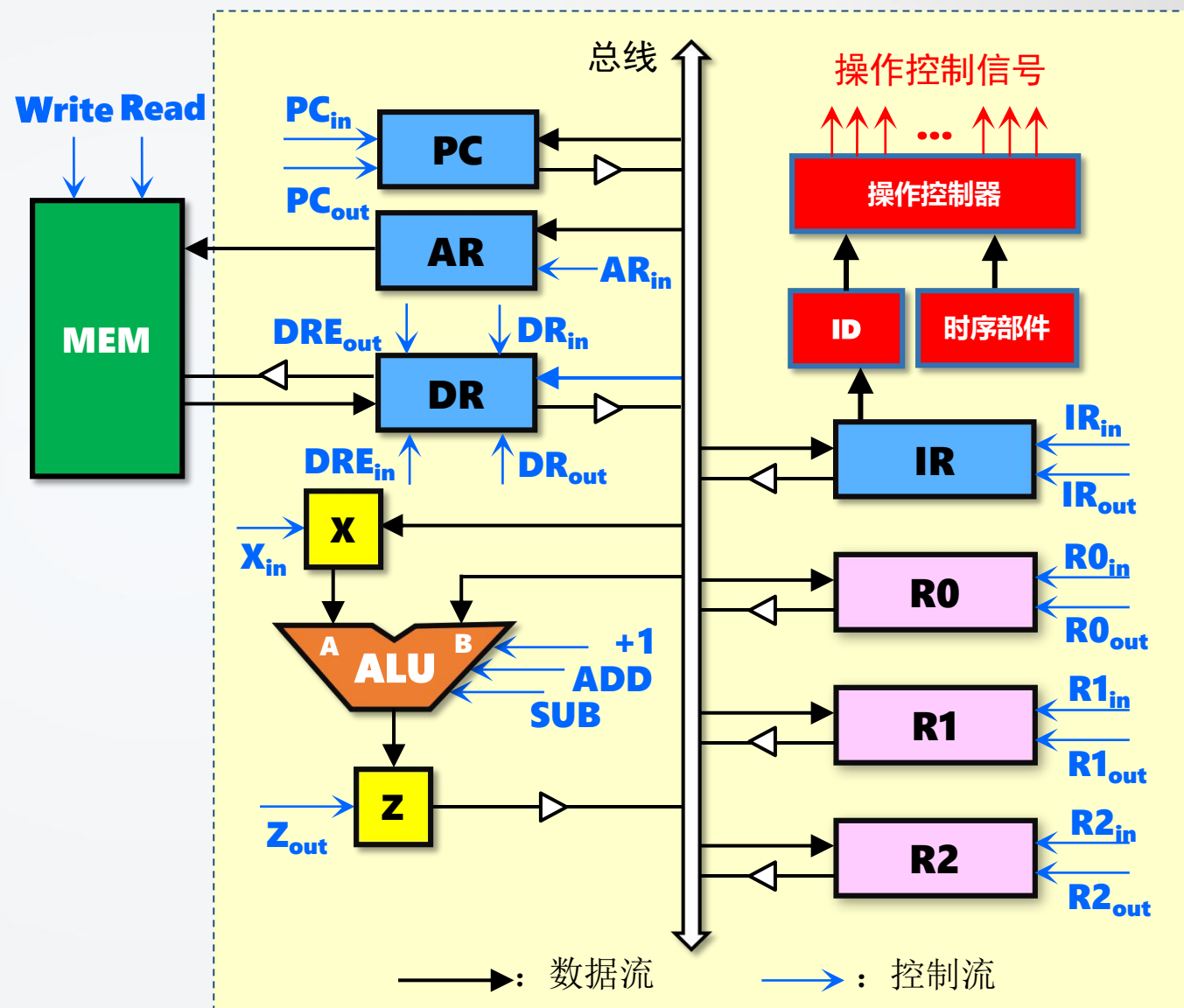
1. **LOAD R0,6#**

2. **MOVE R1,10**

3. **ADD R0,R1**

4. **STORE R0,(R2)**

5. **JMP 1000**



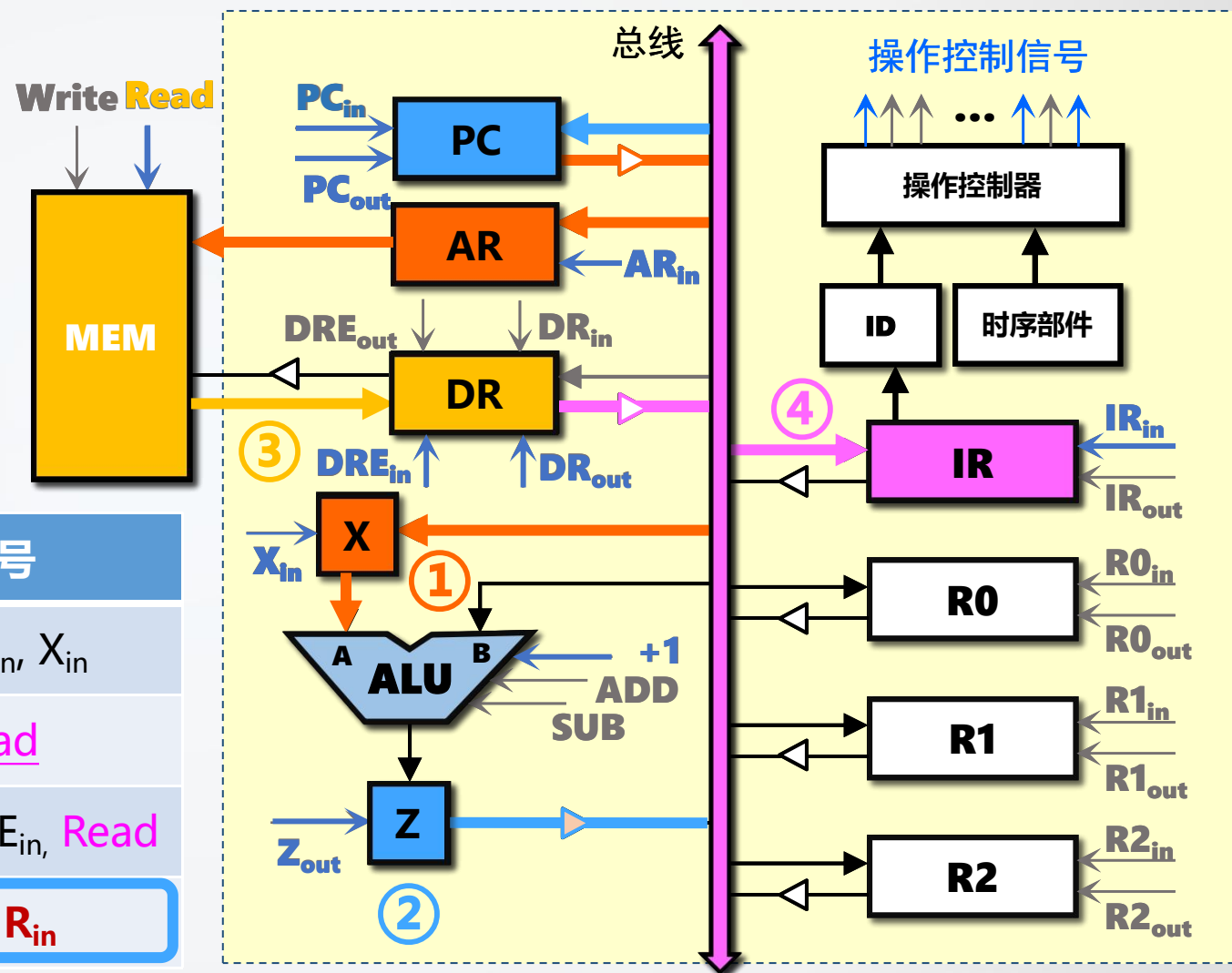
- 后述用的寄存器传送语言RTL，见教材P191。

#	指令	指令功能
1	LOAD R0,6#	(Mem[6])→ R0
2	MOVE R1,10	10 → R1
3	ADD R0,R1	(R0) + (R1) → R0
4	STORE R0,(R2)	(R0) → Mem[(R2)]
5	JMP 1000	1000 → PC

单总线结构CPU 取指周期的数据通路

Mem[PC++] → IR

节拍	数据通路	控制信号
T1	(PC)→AR, (PC)→X	PC _{out} , AR _{in} , X _{in}
T2	(X)+1 → Z	+1, <u>Read</u>
T3	(Z)→PC, Mem[AR]→DR	Z _{out} , PC _{in} , DRE _{in} , <u>Read</u>
T4	(DR)→IR	DR_{out}, IR_{in}

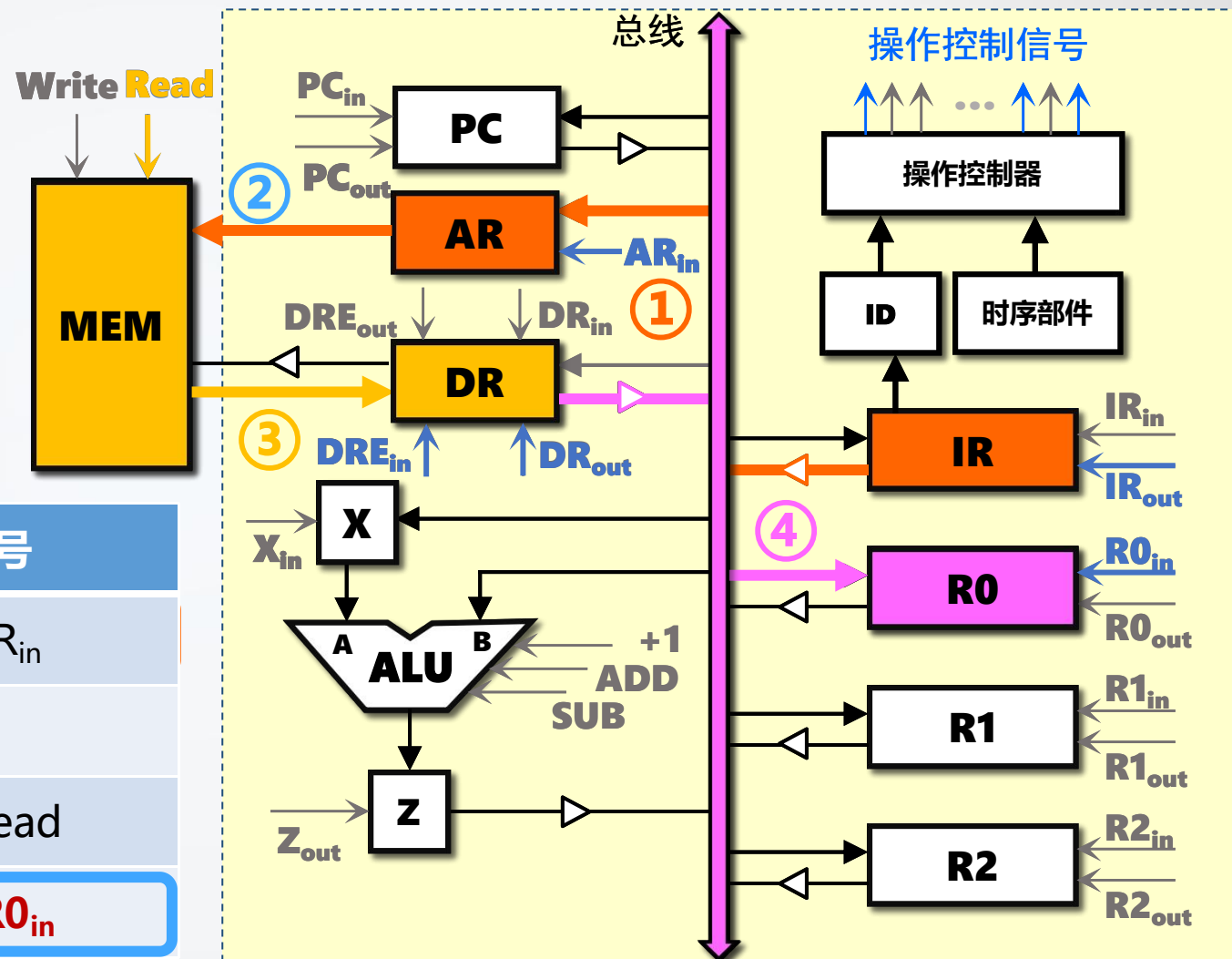


单总线结构CPU Load指令 执行周期的数据通路

LOAD R0,6#

即 $\text{Mem}[\text{IR}_A] \rightarrow \text{Reg}$

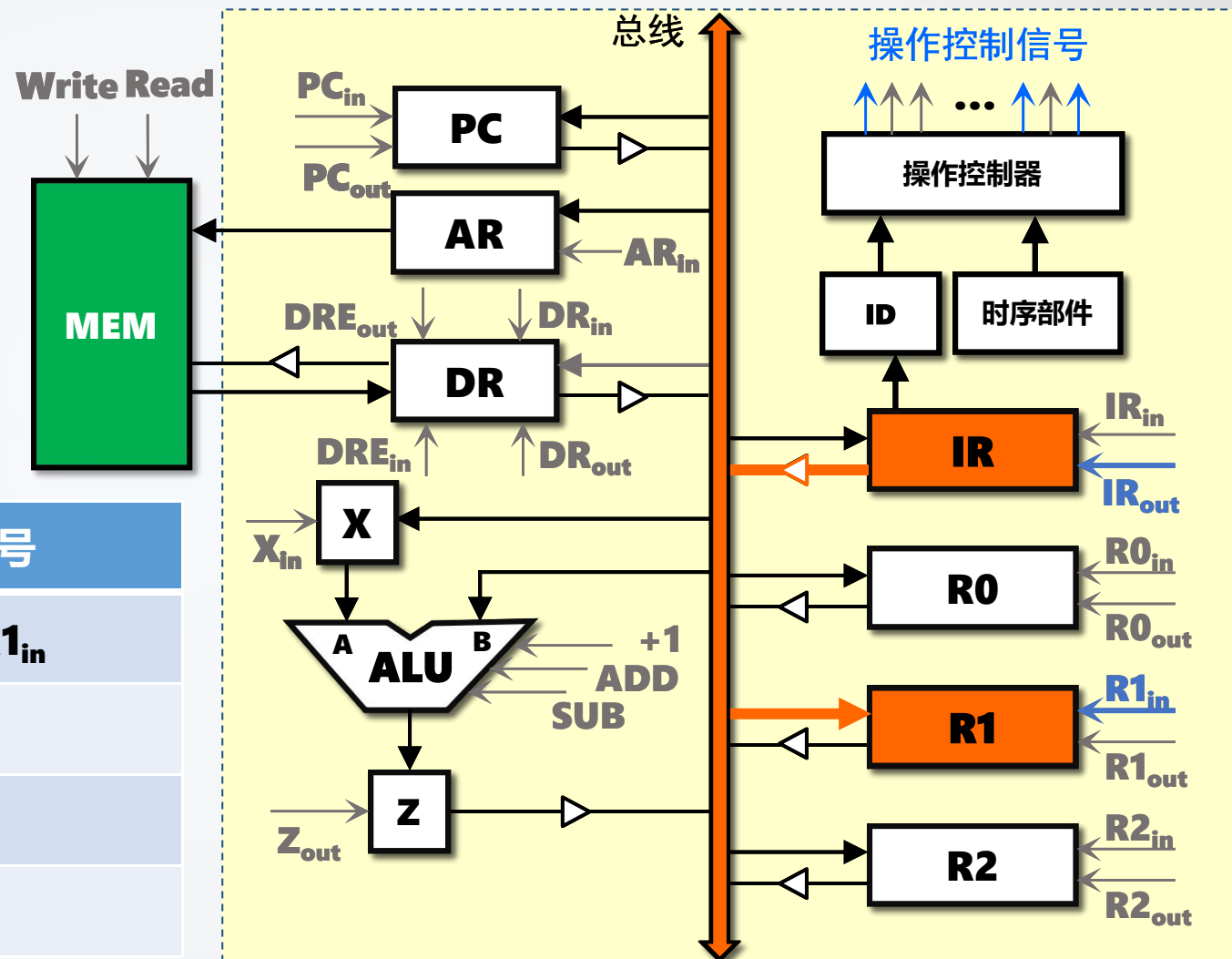
节拍	数据通路	控制信号
T1	$(\text{IR}_A) \rightarrow \text{AR}$	$\text{IR}_{\text{out}}, \text{AR}_{\text{in}}$
T2	Read Mem[AR]	Read
T3	$\text{Mem}[\text{AR}] \rightarrow \text{DR}$	$\text{DRE}_{\text{in}}, \text{Read}$
T4	$(\text{DR}) \rightarrow \text{R0}$	$\text{DR}_{\text{out}}, \text{R0}_{\text{in}}$



单总线结构CPU MOVE指令执行周期的数据通路

MOVE R1,10
即 $(IR_A) \rightarrow Reg$

节拍	数据通路	控制信号
T1	$(IR_A) \rightarrow R[1]$	$IR_{out}, R1_{in}$
T2		
T3		
T4		

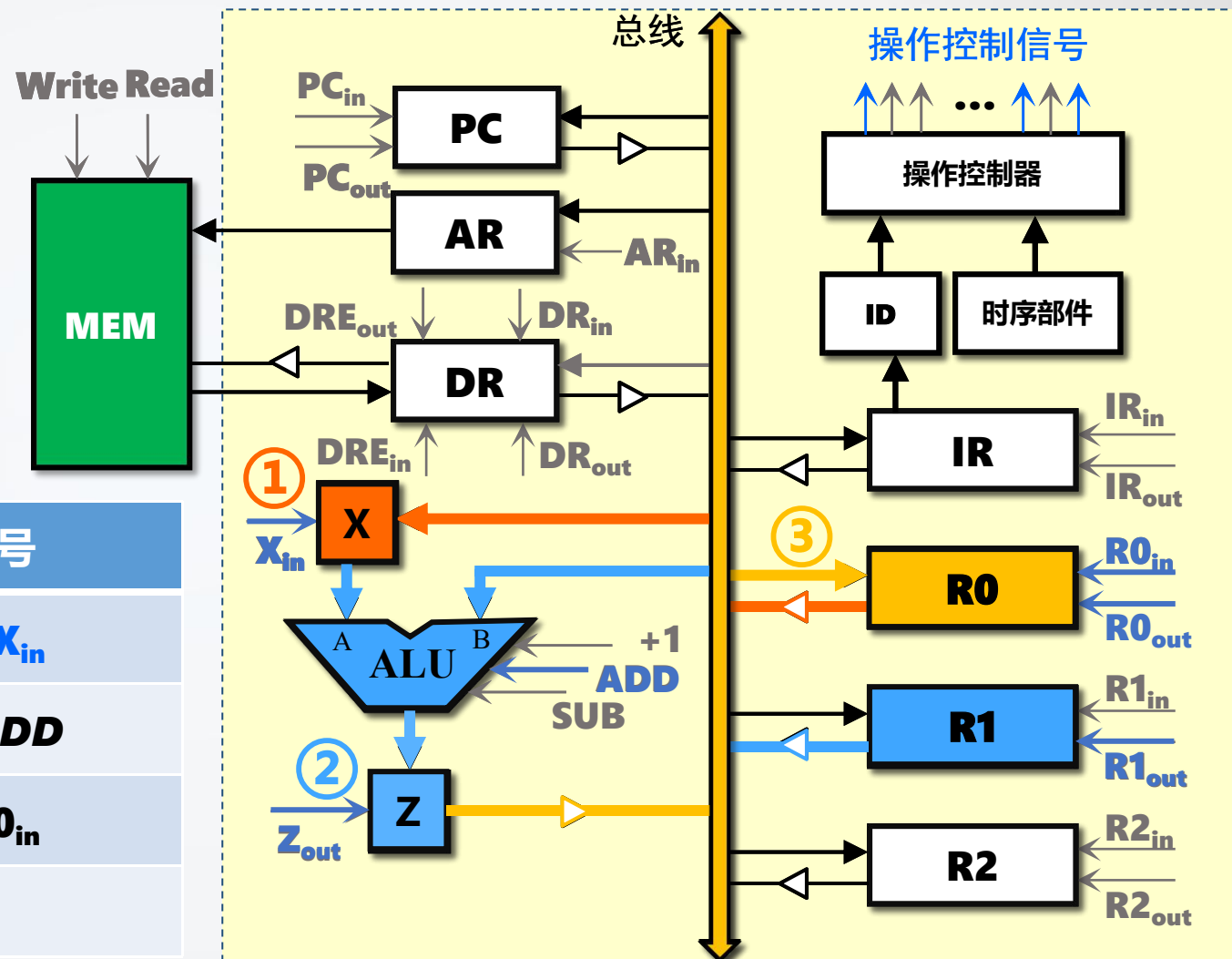


单总线结构CPU ADD指令 执行周期的数据通路

ADD R0,R1

即 $(R0) + (R1) \rightarrow R0$

节拍	数据通路	控制信号
T1	$(R0) \rightarrow X$	$R0_{out}, X_{in}$
T2	$(X) + (R1) \rightarrow Z$	$R1_{out}, ADD$
T3	$(Z) \rightarrow R0$	$Z_{out}, R0_{in}$
T4		

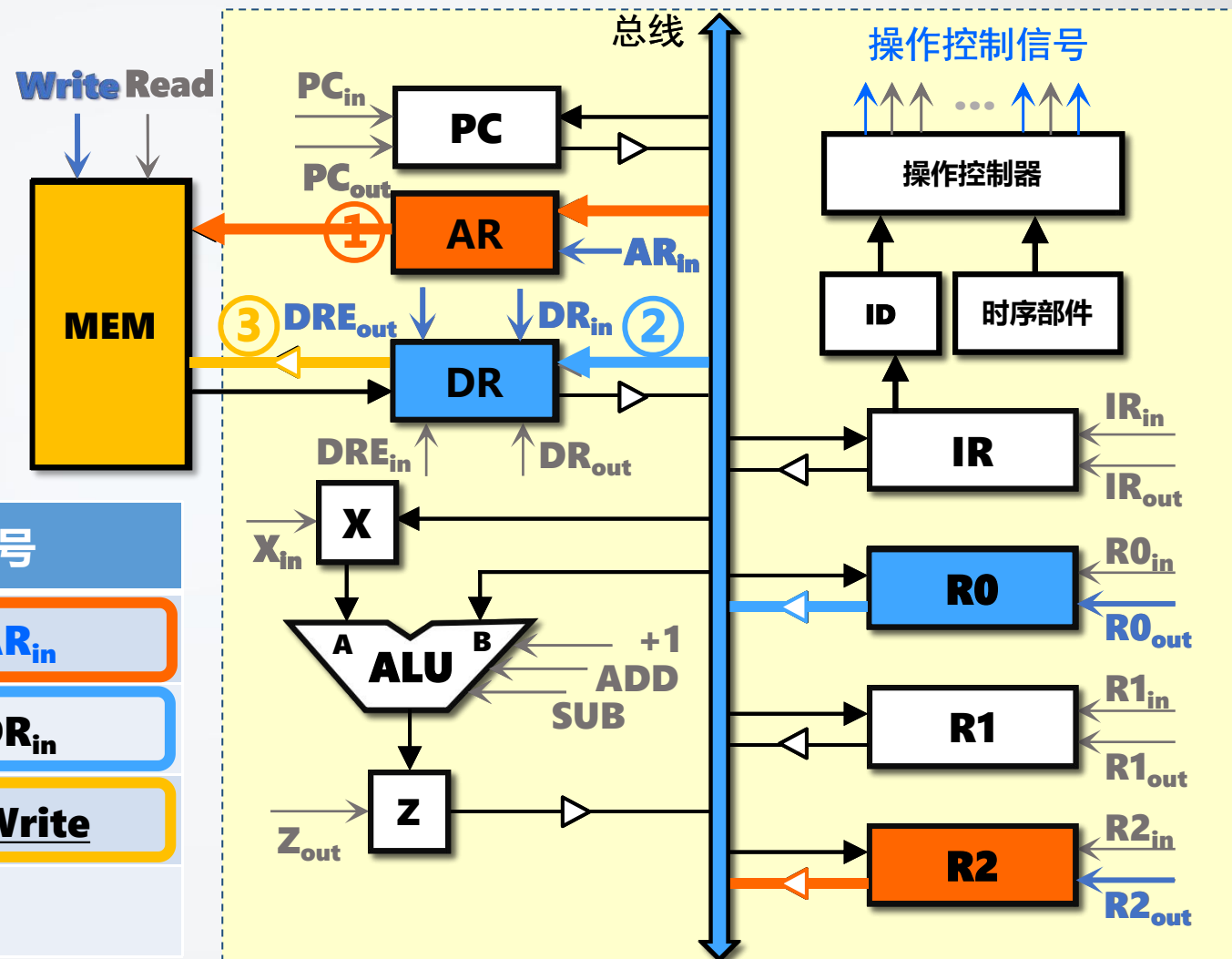


单总线结构CPU STORE指令 执行周期的数据通路

STORE R0,(R2)

即 $(R0) \rightarrow \text{Mem}[(R2)]$

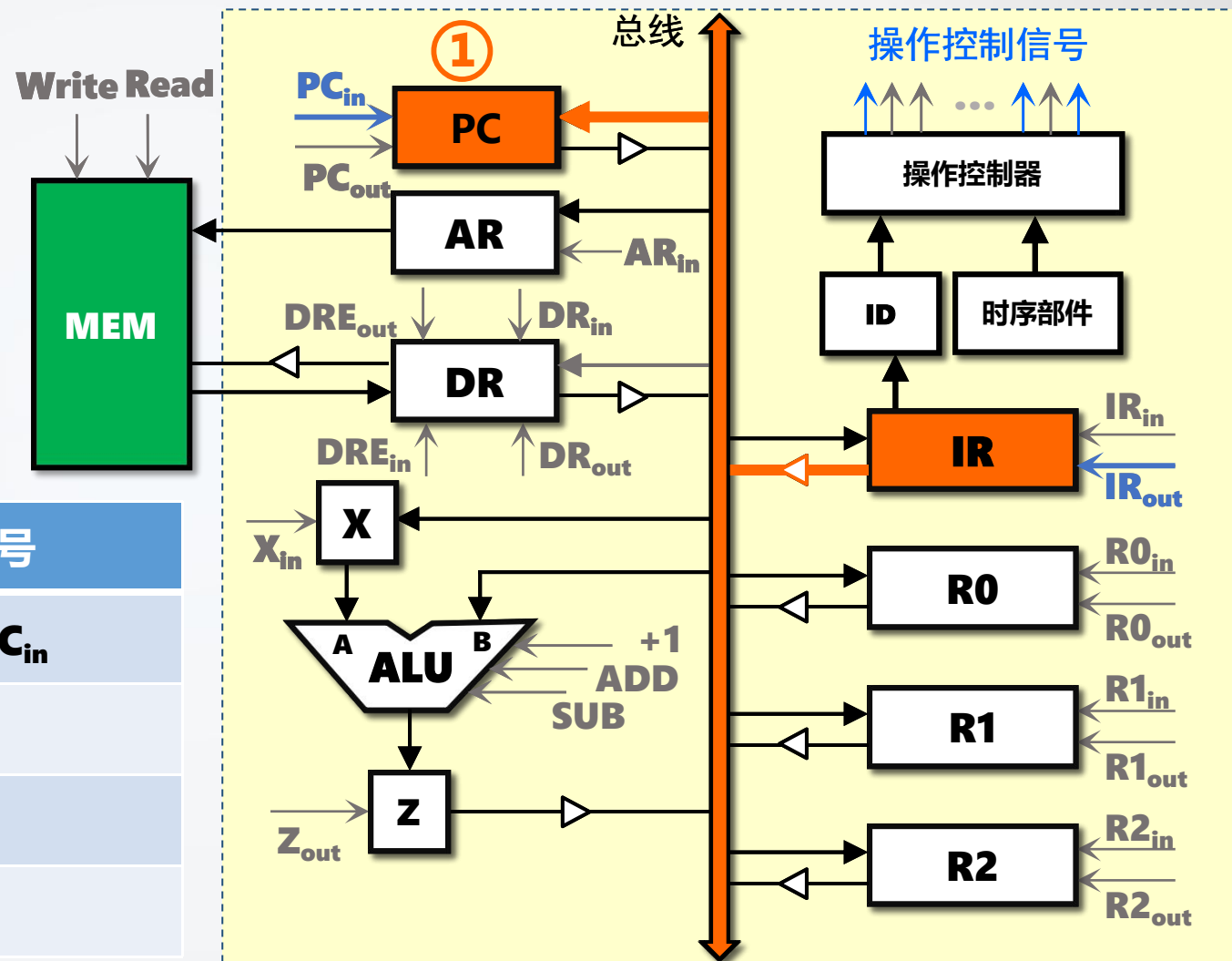
节拍	数据通路	控制信号
T1	$(R2) \rightarrow AR$	$R2_{out}, AR_{in}$
T2	$(R0) \rightarrow DR$	$R0_{out}, DR_{in}$
T3	$(DR) \rightarrow \text{Mem}[AR]$	DRE_{out}, Write
T4		



单总线结构CPU JMP指令 执行周期的数据通路

JMP 1000
即 $(IR_A) \rightarrow PC$

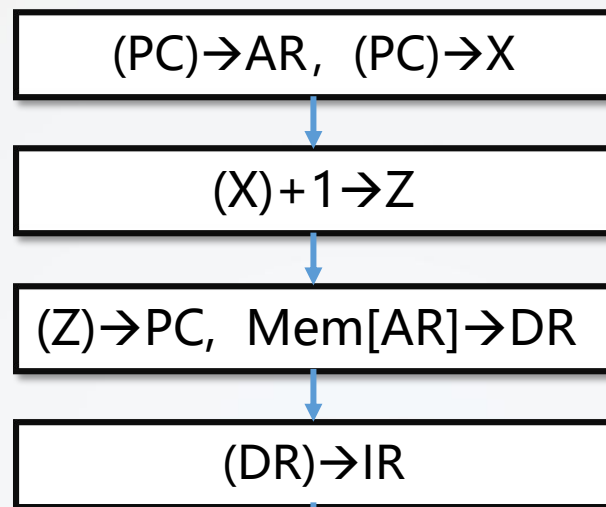
节拍	数据通路	控制信号
T1	$(IR_A) \rightarrow PC$	IR_{out}, PC_{in}
T2		
T3		
T4		





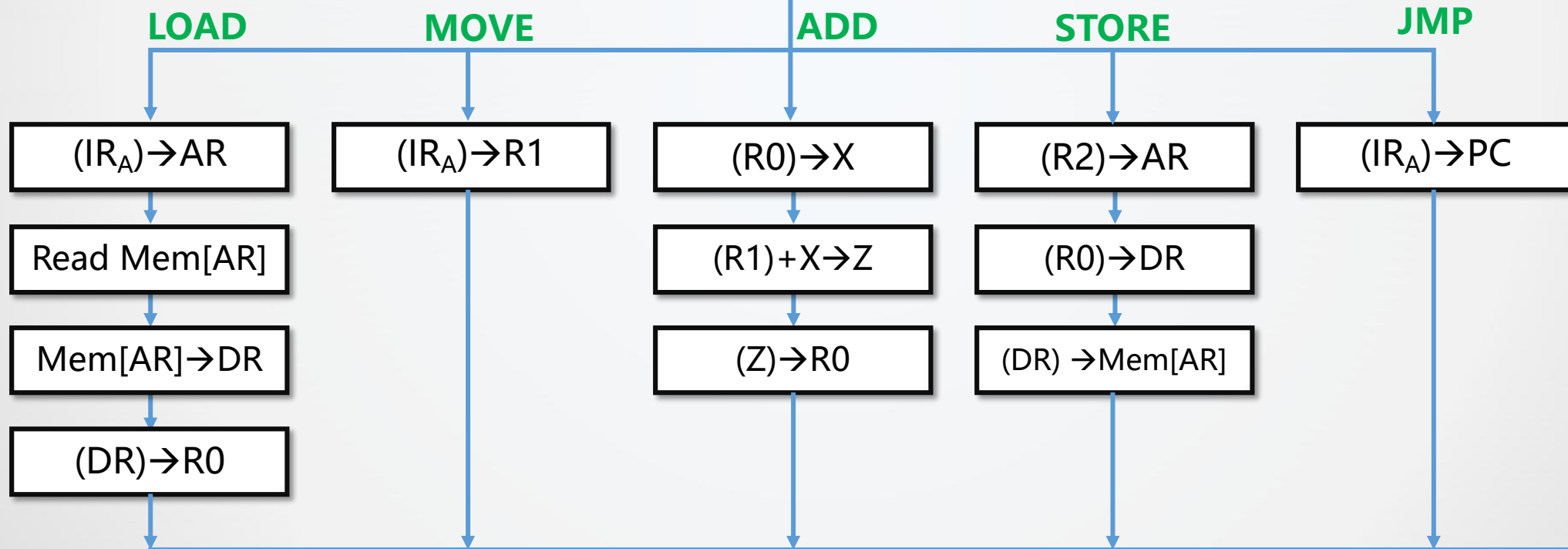
指令周期方框图

取指(令)周期



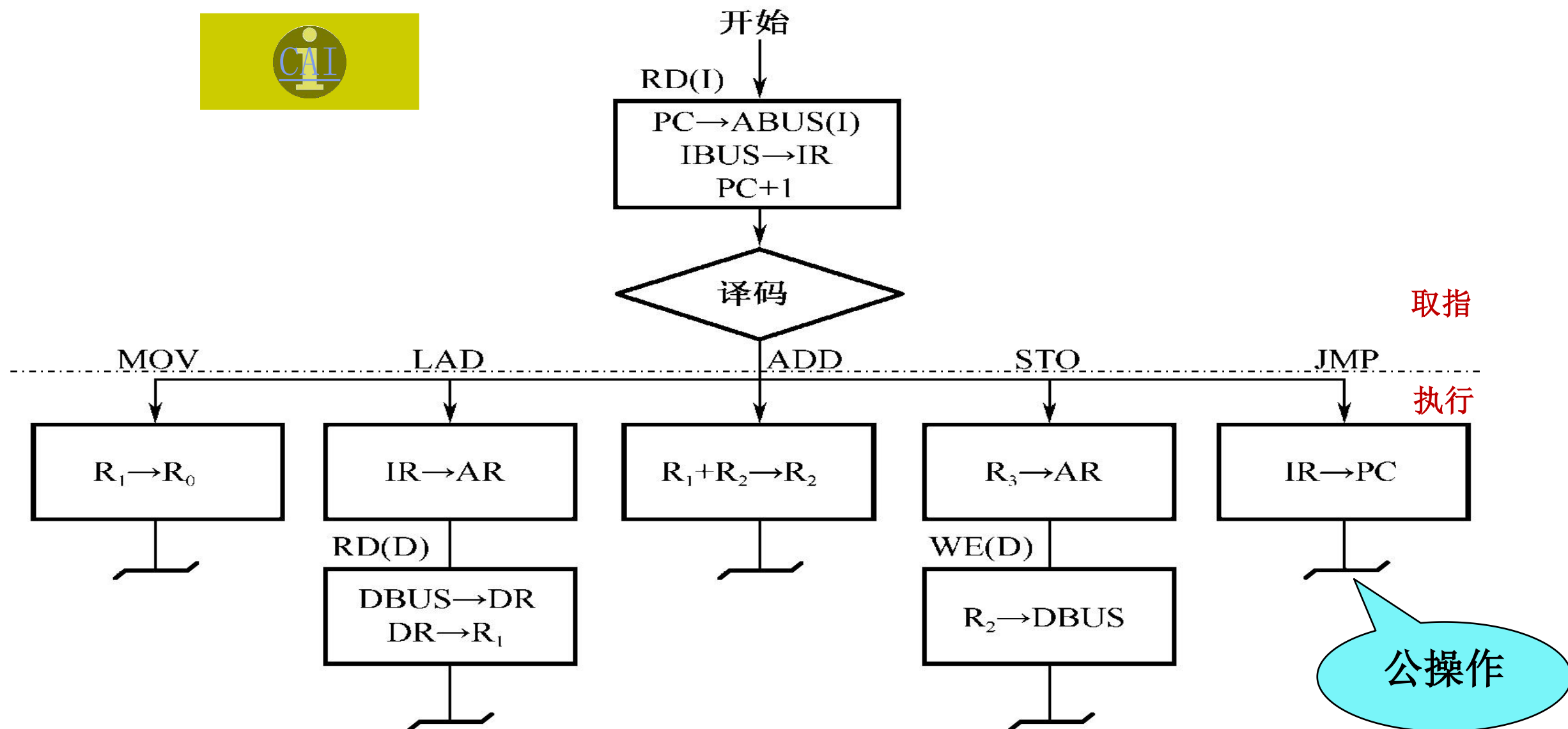
公操作

执行周期



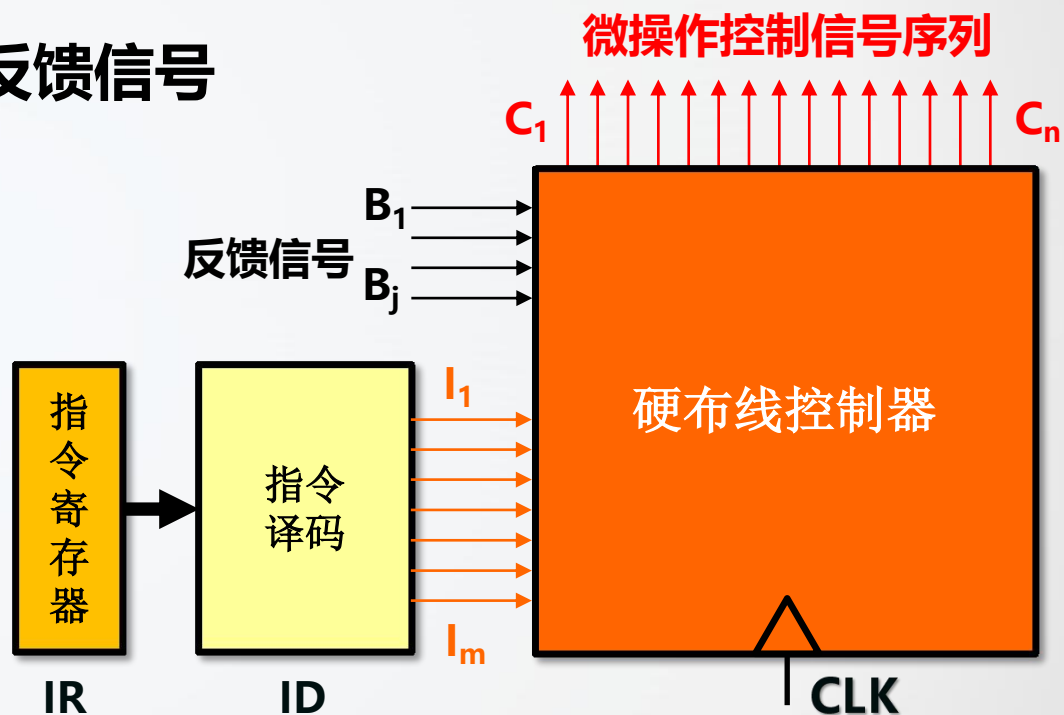
指令周期流程图:

《白》5.2.7节



控制器基本原理

- 将控制器看成产生固定时序控制信号的逻辑电路
- 输入信号：含 指令译码信号，时钟信号，反馈信号
- 输出信号：功能部件的控制信号序列
- 设计目标：最少元件，最快速度
- 理论基础：布尔代数
- 组成器件：门电路，触发器。



机器指令字 → 控制器信号序列

单总线结构CPU指令周期

取指周期

节拍	控制信号 (4 cycles)
T1	PC _{out} , AR _{in} , X _{in}
T2	+1, Read
T3	Z _{out} , PC _{in} , DRE _{in} , Read
T4	DR _{out} , IR _{in}

■ 定长指令周期：传统三级时序

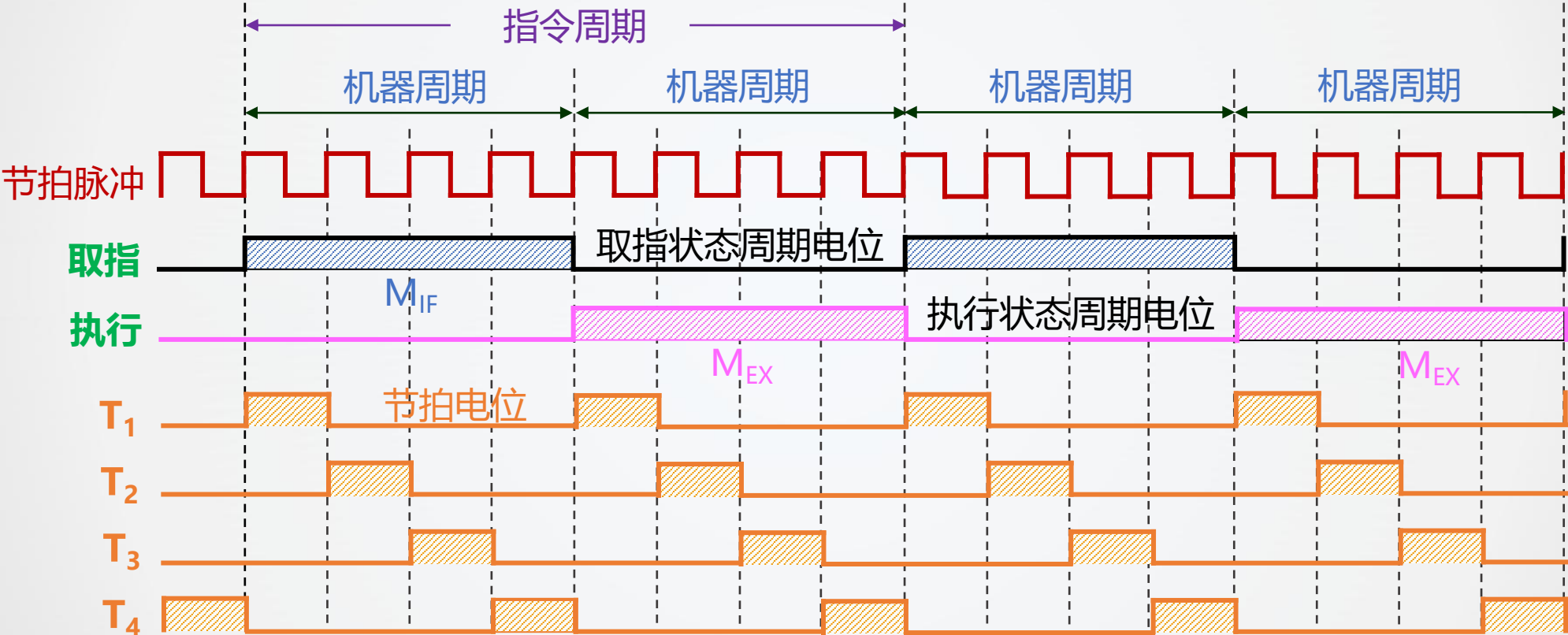
◆ 这里仅设定**2**个机器周期（取指+执行周期）、**8**个时钟周期。执行慢、但设计简单

■ 变长指令周期：现代时序。

执行周期

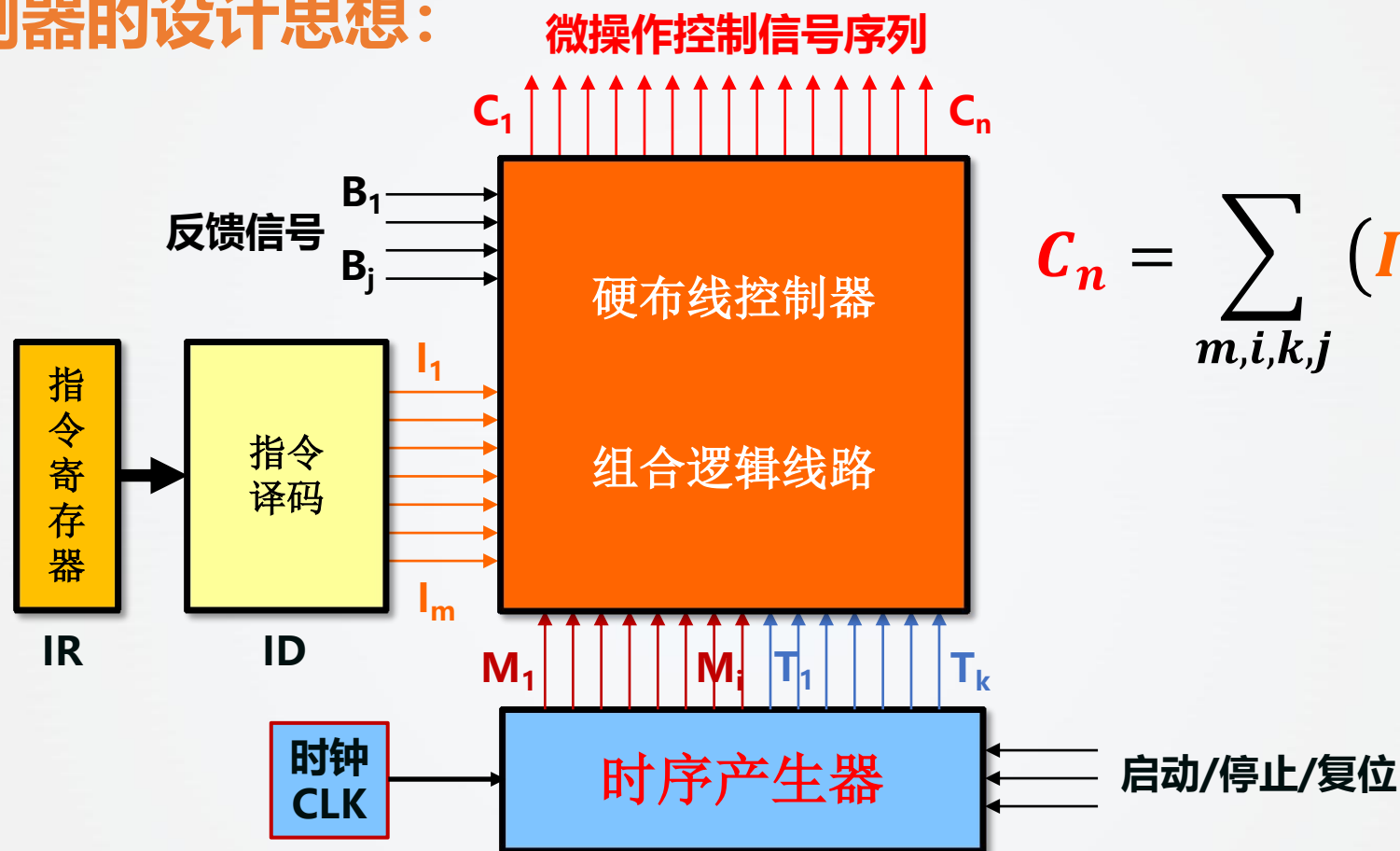
节拍	LOAD (4 cycles)	MOVE (1 cycles)	ADD (3 cycles)	STORE (3 cycles)	JMP (1 cycles)
T5	IR _{out} , AR _{in}	IR _{out} , R1 _{in}	R0 _{out} , X _{in}	R2 _{out} , AR _{in}	IR _{out} , PC _{in}
T6	Read		R1 _{out} , ADD	R0 _{out} , DR _{in}	
T7	DRE _{in} , Read		Z _{out} , R0 _{in}	DRE _{out} , Write	
T8	DR _{out} , R0 _{in}				

固定 2个机器周期， 8个时钟节拍



硬布线控制器基本架构---同步控制

➤ 硬布线控制器的设计思想:



$$C_n = \sum_{m,i,k,j} (I_m \cdot M_i \cdot T_k \cdot B_j)$$

时序产生器循环产生状态周期电位、节拍电位, 供控制器对信号进行时间调制。

单总线CPU 控制信号生成举例

取指周期
 M_{IF}

节拍	控制信号
T1	PC_{out} , AR_{in} , X_{in}
T2	+1, Read
T3	Z_{out} , PC_{in} , DRE_{in} , Read
T4	DR_{out} , IR_{in}

执行周期
 M_{EX}

节拍	LOAD	MOVE	ADD	STORE	JMP
T1	IR_{out} , AR_{in}	IR_{out} , $R1_{in}$	$R0_{out}$, X_{in}	$R2_{out}$, AR_{in}	IR_{out} , PC_{in}
T2	Read		$R1_{out}$, ADD	$R0_{out}$, DRE_{in}	
T3	DRE_{in} , Read		Z_{out} , $R0_{in}$	DRE_{out} , Write	
T4	DR_{out} , $R0_{in}$				

$$C_n = \sum_i \left(M_i \cdot T_k \cdot B_j \cdot \sum_m I_m \right)$$

例:

$$\blacksquare \text{ Read} = M_{IF} \cdot (T2+T3) + \text{LOAD} \cdot M_{EX} \cdot (T2+T3)$$

$$\blacksquare \text{ AR}_{in} = M_{IF} \cdot T1 + (\text{LOAD} + \text{STORE}) \cdot M_{EX} \cdot T1。$$



定长指令周期 硬布线控制器设计过程小结:

1. 设计时序产生器: 根据固定机器周期、节拍数固定, 构建状态图 (多时钟周期)
2. 列出所有机器指令的指令周期关系表, 明确每个节拍的控制信号;
3. 找出产生同一微操作控制信号的条件;
4. 写出各微操作控制信号的布尔表达式:
$$C_n = \sum_i \left(M_i \cdot T_k \cdot B_j \cdot \sum_m I_m \right)$$
5. 化简各表达式;
6. 利用组合逻辑电路实现控制器。

硬布线控制器 特点小结

- 结构复杂，无规则；
- 设计和调试困难；
- 要求指令系统和指令功能不变
- 适用于VLSI(超大规模集成电路)
- 速度快。

设计处理器的步骤

1. **选定指令系统**，将指令功能规范地描述出来（例如用RTL寄存器传送语言）
2. **设计部件与通路**，根据指令功能 设计功能部件，并考虑各部件如何互连。
3. **设计控点**，确定所有功能部件、数据通路所需**控制信号**
4. **分析指令信号序列**，汇总所有指令的控制信号序列，生成 指令与控制信号的关系表
5. **设计控制器**，根据关系表**设计微指令、微程序（设计微程序控制器）**，或生成每个控制信号的逻辑表达式，设计硬布线控制器。

MIPS CPU 硬布线控制器设计

- 定长指令周期：这里设定单周期实现，即
 - ◆ 所有指令均在 **1** 个时钟周期内完成， **CPI = 1**
 - ◆ 性能取决于最慢的指令，因此时钟周期过长。
- 变长指令周期：多周期实现
 - 缩短时钟周期，通过复用器件或数据通路
 - 可支持流水操作，提升性能。

分析以下MIPS指令格式，它属于（ ）类型的MIPS指令。



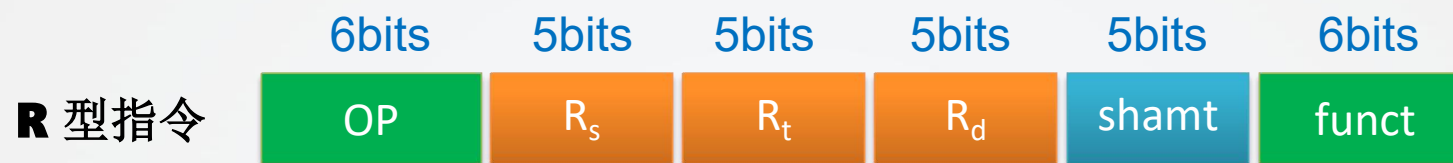
- ☐ A R
- ☐ B I
- ☒ C J
- ☐ D 以上都不是。

提交

|| (回顾) MIPS 32指令格式



R型指令格式



■ `add $s1, $s2, $s3`



■ `sub $s0, $s1, $s2`



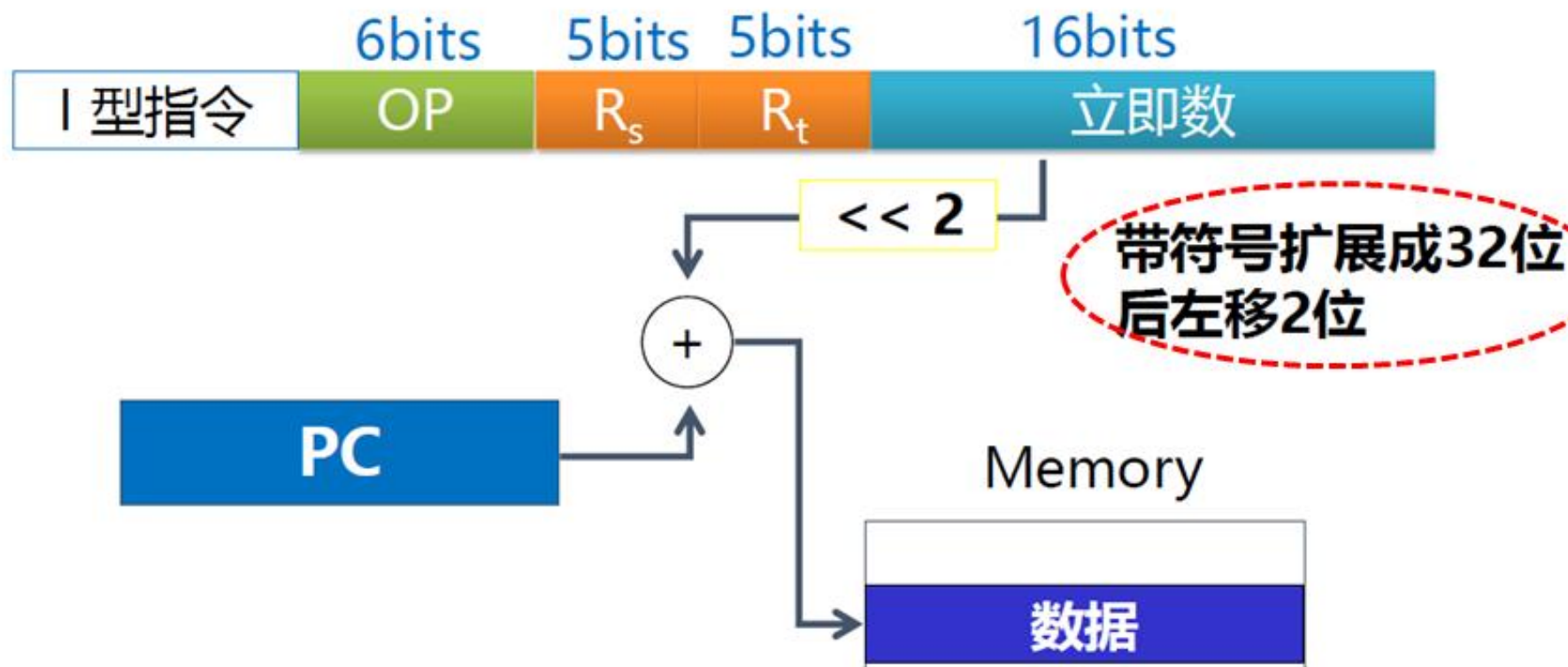
■ `slt $s0, $s1, $s2`



I 型指令格式

I 型指令

相对寻址

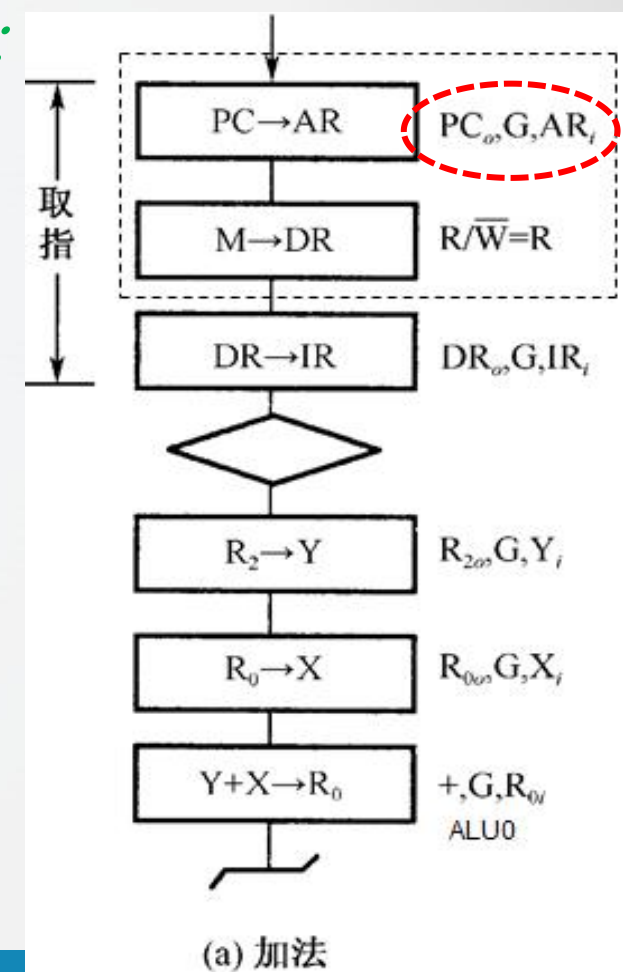


beq \$s1, \$s2, 400 (相对寻址)



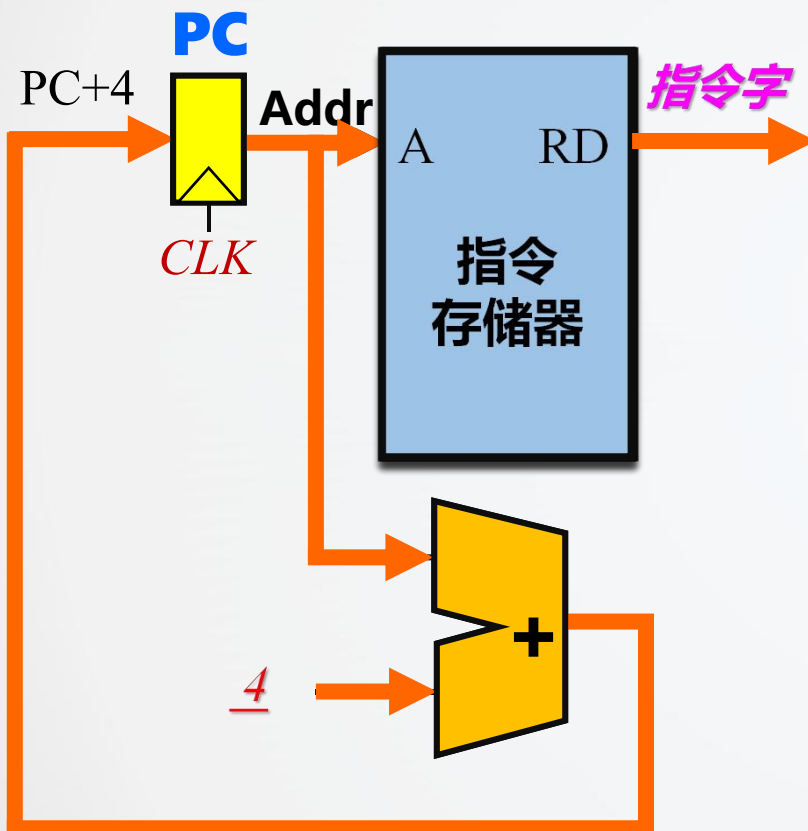
作业 1

- 本子上完成：
- 1、教材P248习题6.3: (2)(3)(4)
- 2、试题书5.2分析题: 3, 4(说明: “控制信号G控制的是一个门电路, 它相当于两条总线间的桥(即旁路器)”)。类似的微操作控制信号序列:





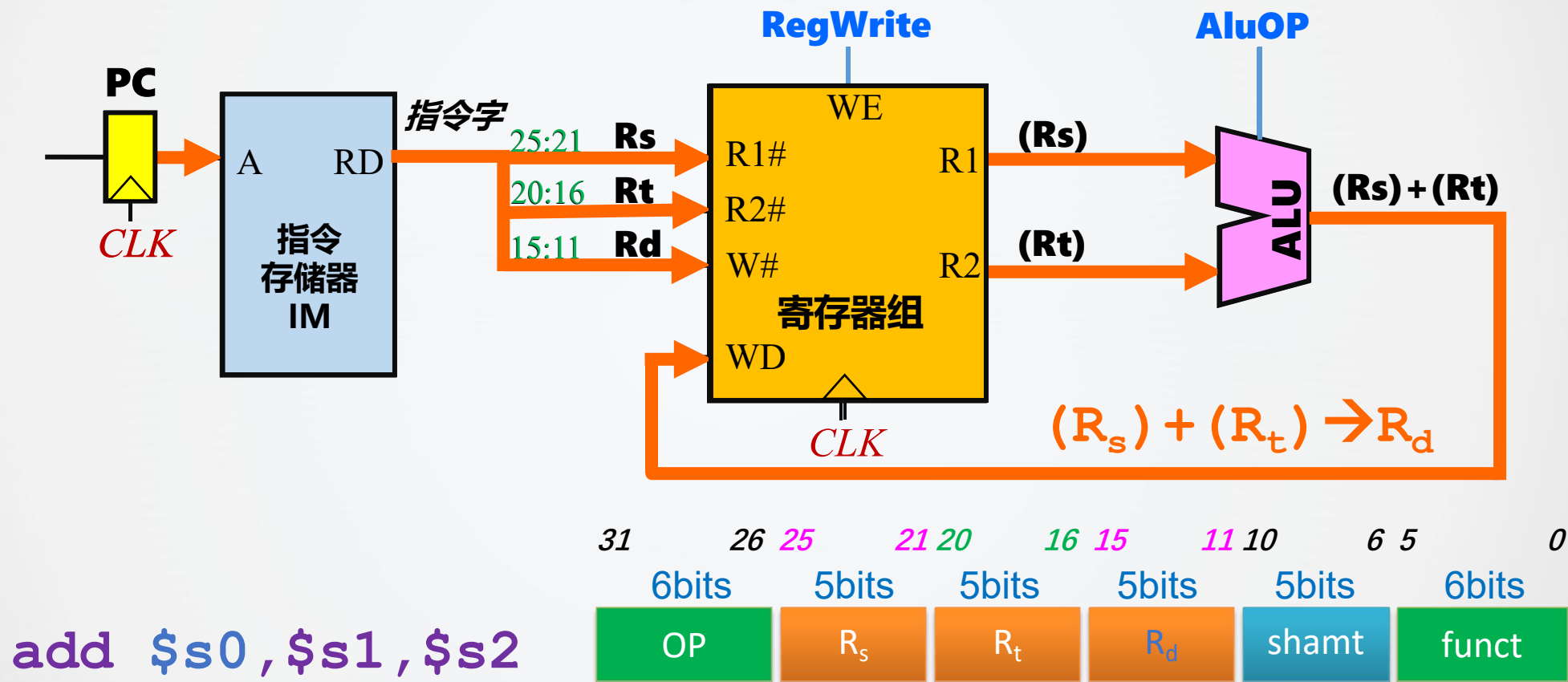
取指令数据通路



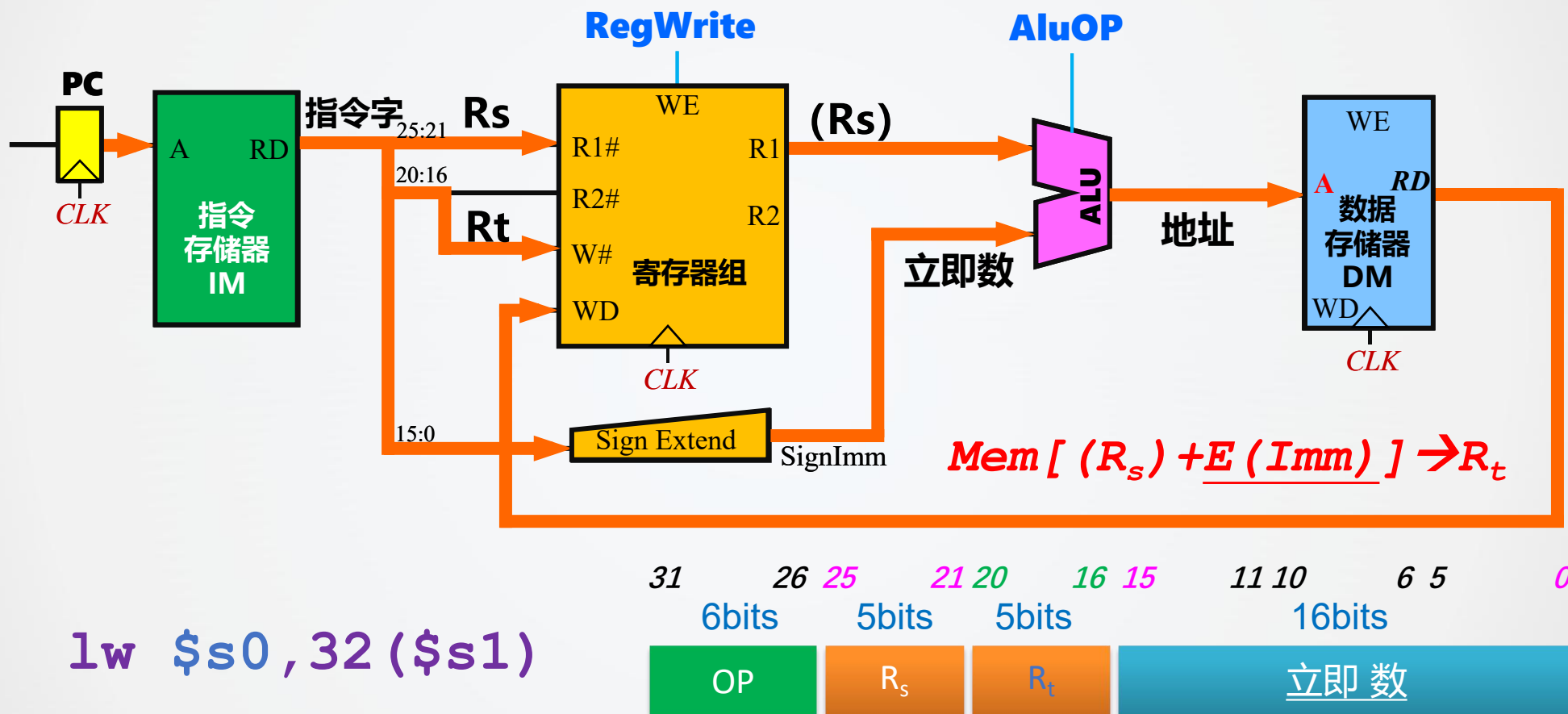
$\text{Mem}[\underline{\text{PC}++}] \rightarrow \text{IR}$

- 单周期 不能设置AR, DR, IR寄存器
- 程序和数据 分开存放——哈佛结构
 - ◆ 指令存储器、数据存储器
 - ◆ 类似: 指令cache, 数据cache (现代计算机)
- **PC累加器** 和**ALU**运算器分离。

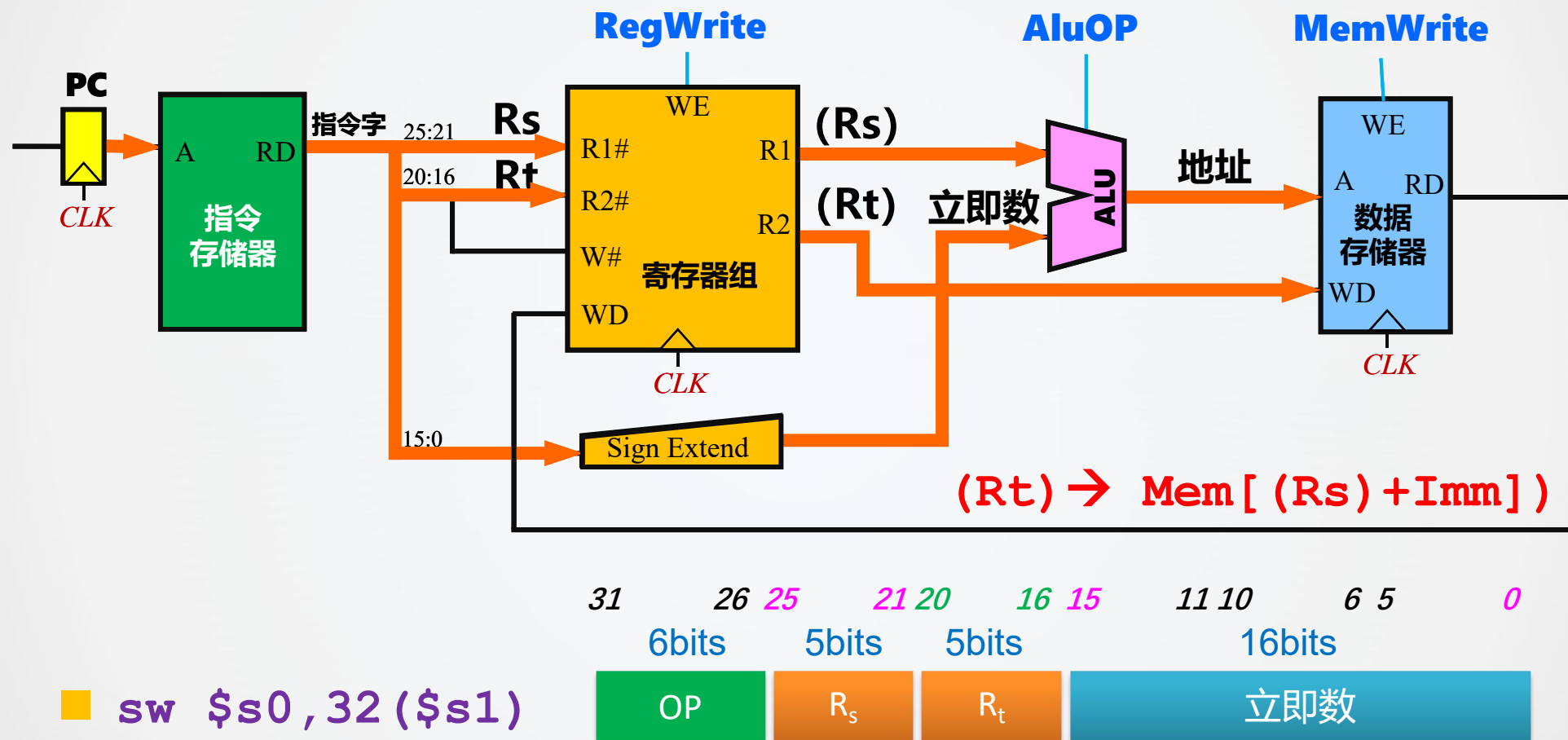
R型指令数据通路

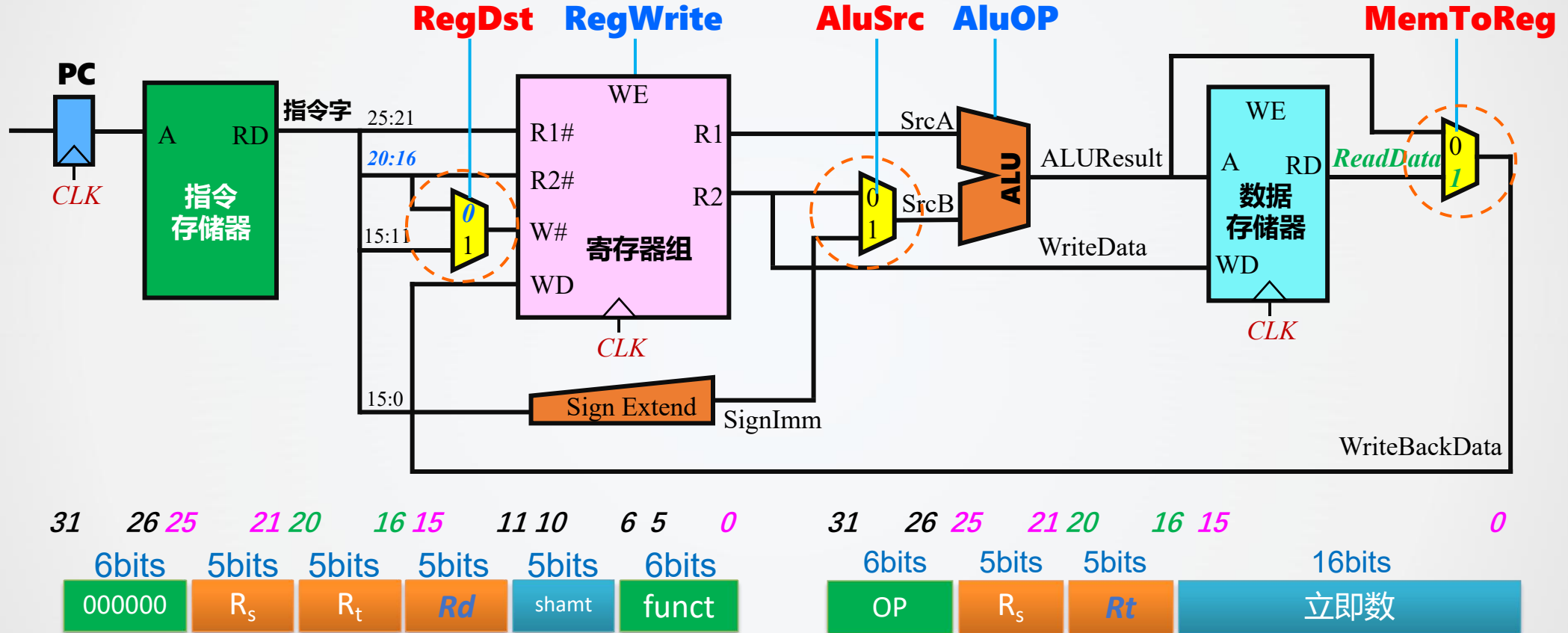


Lw指令数据通路



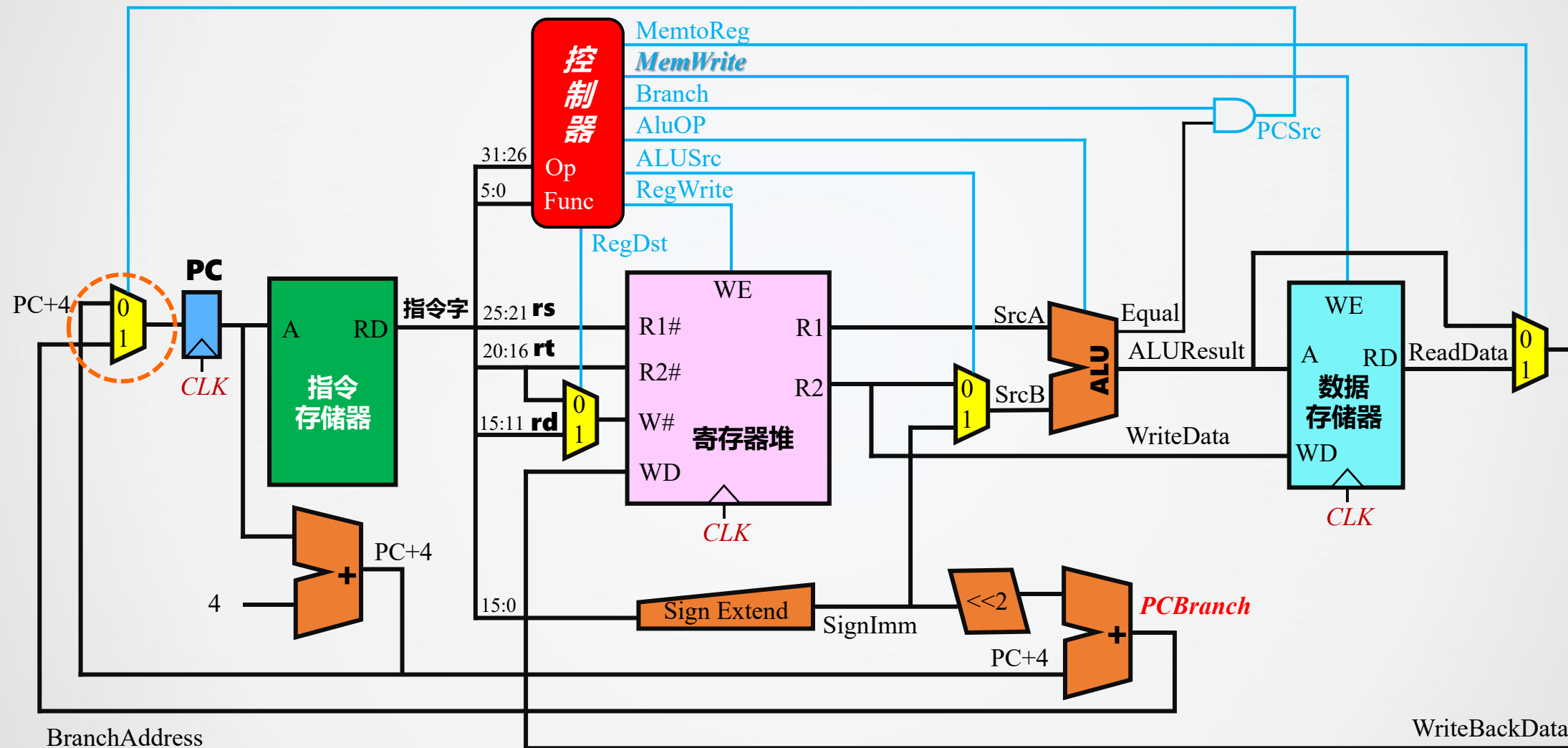
sw指令数据通路





凡是有多个输入来源的, 增加 多路选择器(MUX), 引入**控点**。

单周期MIPS数据通路



R型指令数据通路建立过程

add \$rd,\$rs,\$rt

$R[\$rd] \leftarrow R[\$rs] + R[\$rt]$

31 26 25 21 20 16 15 11 10 6 5 0
6bits 5bits 5bits 5bits 5bits 6bits

R 型指令

OP

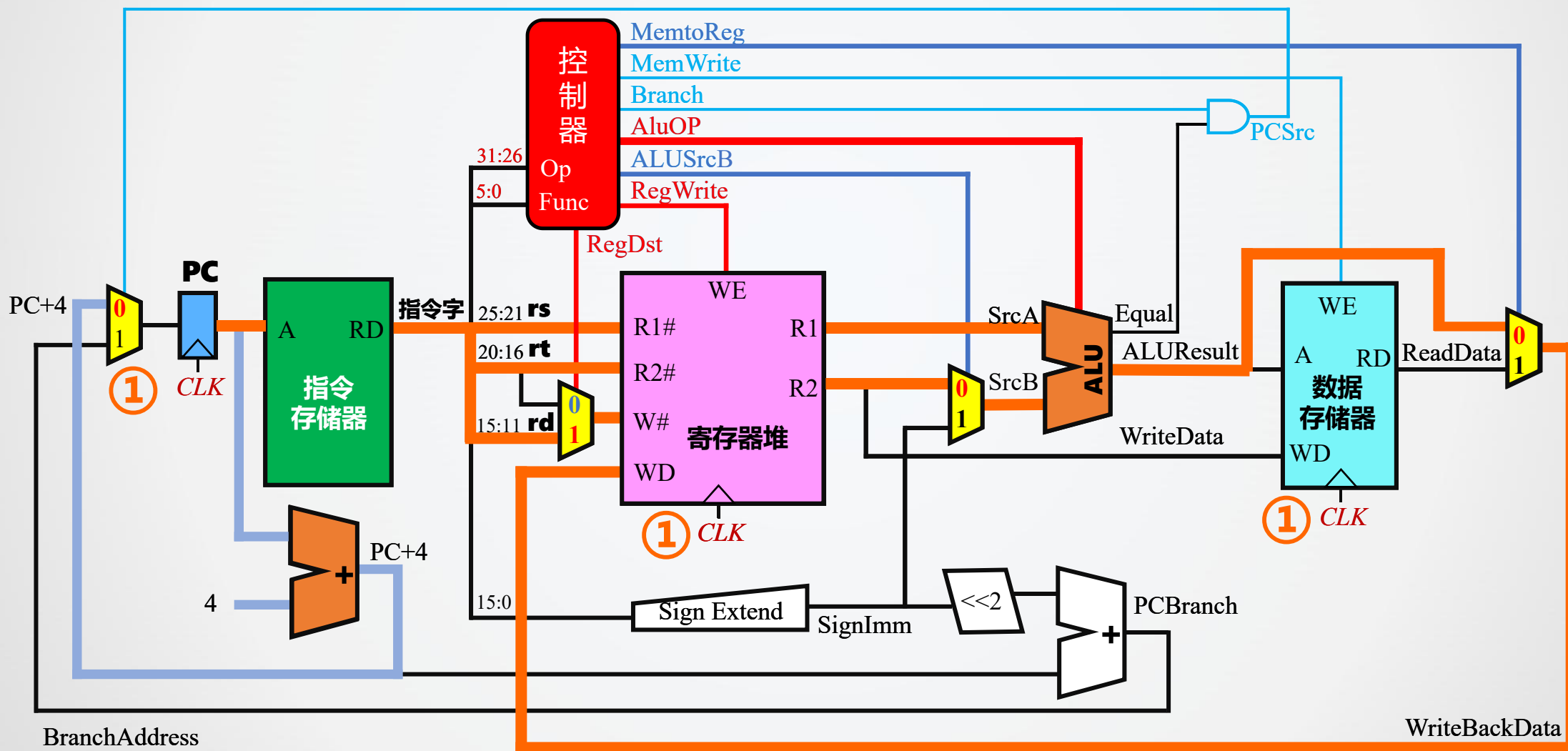
R_s

R_t

R_d

shamt

funct

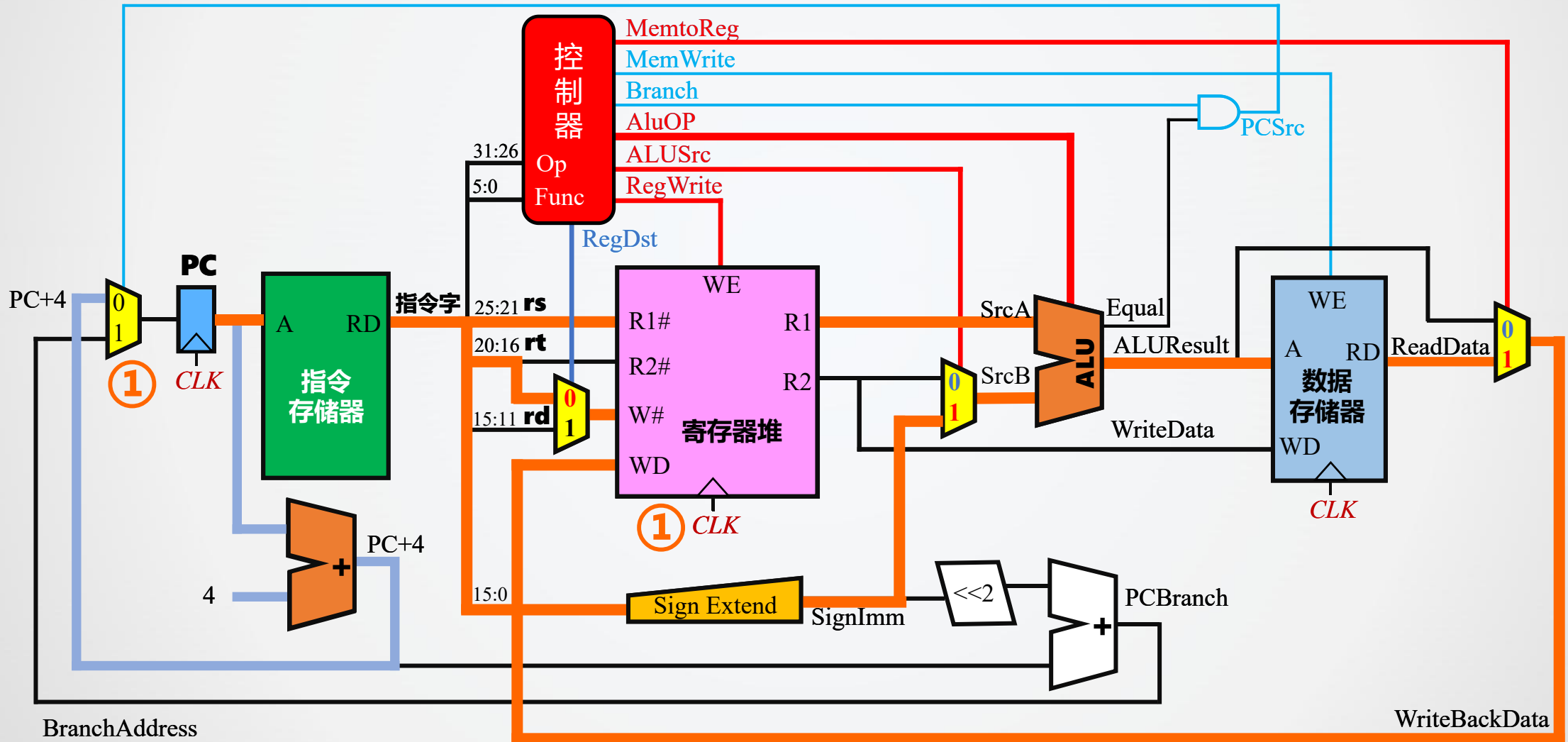


$$R[\$rt] \leftarrow Mem_{4B}(R[\$rs] + SignExt_{16b}(imm))$$

31 26 25 21 20 16 15 0
6bits 5bits 5bits 16bits



LW指令数据通路建立过程



SW指令数据通路建立过程

$$\text{Mem}_{4B}(\text{R}[\$rs] + \text{SignExt}_{16b}(\text{imm})) \leftarrow \text{R}[\$rt]$$

6bits

5bits

5bits

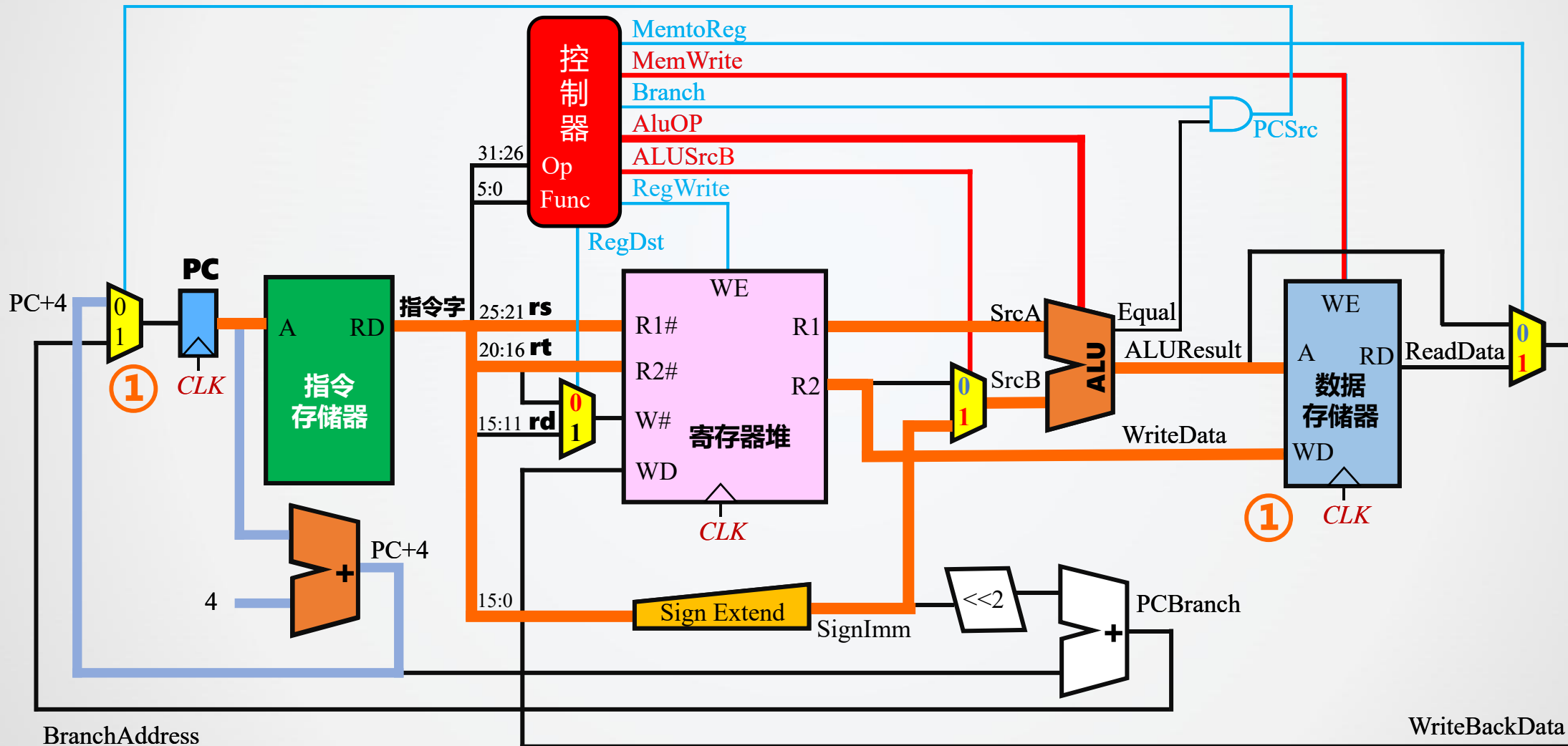
16bits

OP

R_s

R_t

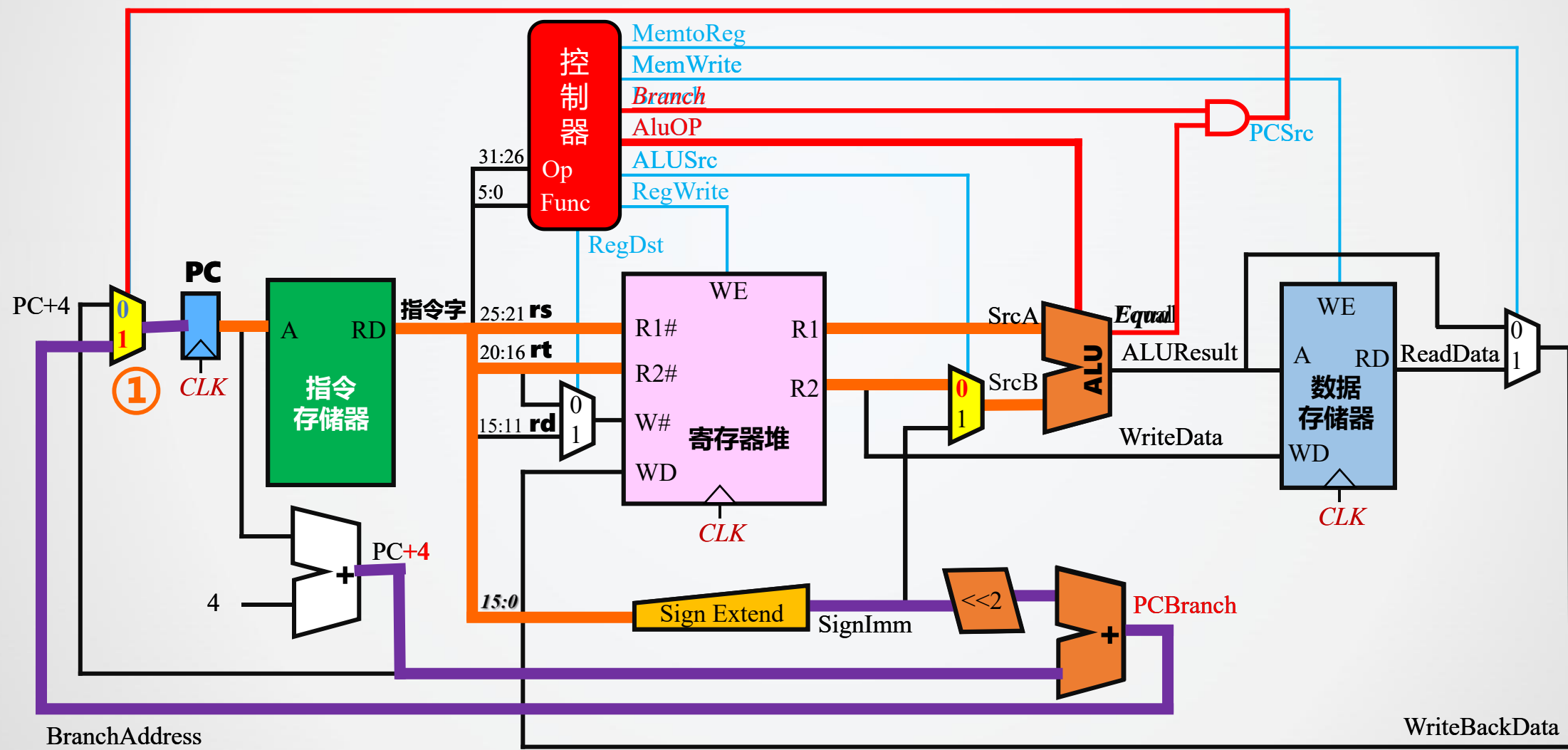
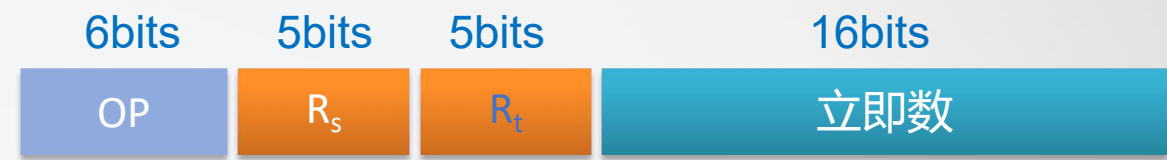
立即数





$$\text{if}(\text{R}[\$rs] = \text{R}[\$rt]) \text{ PC} \leftarrow \text{PC} + \text{SignExt}_{18b}(\{\text{imm}, 00\})$$

Beq指令数据通路建立过程



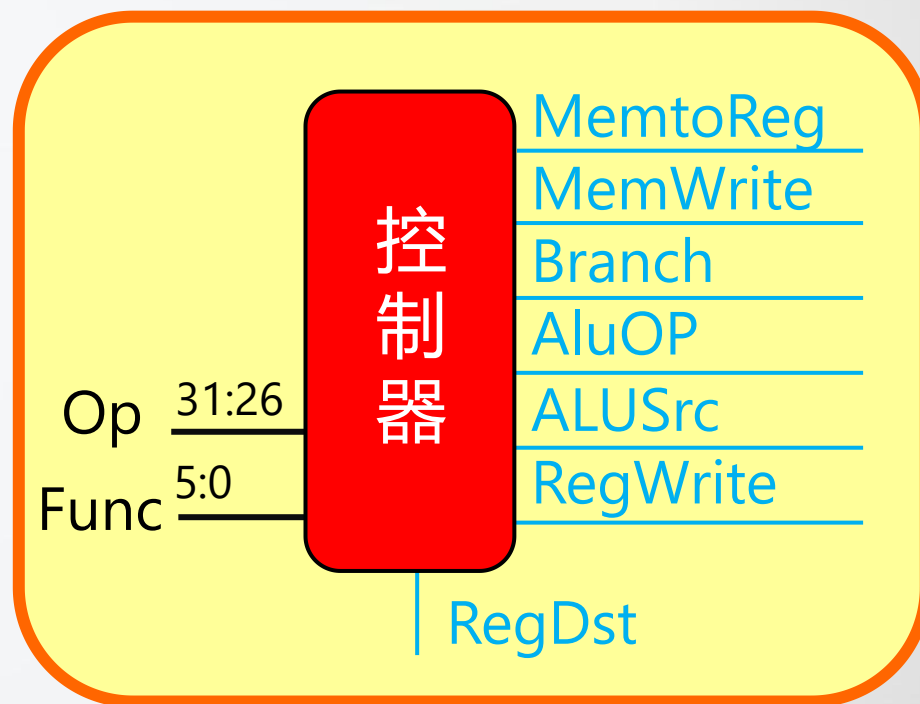
本章主要内容

- CPU的组成和功能
- 数据通路
- 指令周期与时序
- 硬布线控制器设计
- **MIPS单周期CPU设计及实验**
- 微程序控制器设计。



单周期MIPS控制器设计

- 单周期控制器：无时序逻辑，纯组合逻辑电路
- 输入信号：
 - 指令字**Opcode**，**Func**字段（12位）
- 输出信号：
 - 多路选择器选择信号 **MemtoReg**等
 - 内存访问控制信号 **MemWrite**
 - 寄存器写使能信号 **RegWrite**
 - 运算器控制信号 **AluOP**
 - 指令译码信号



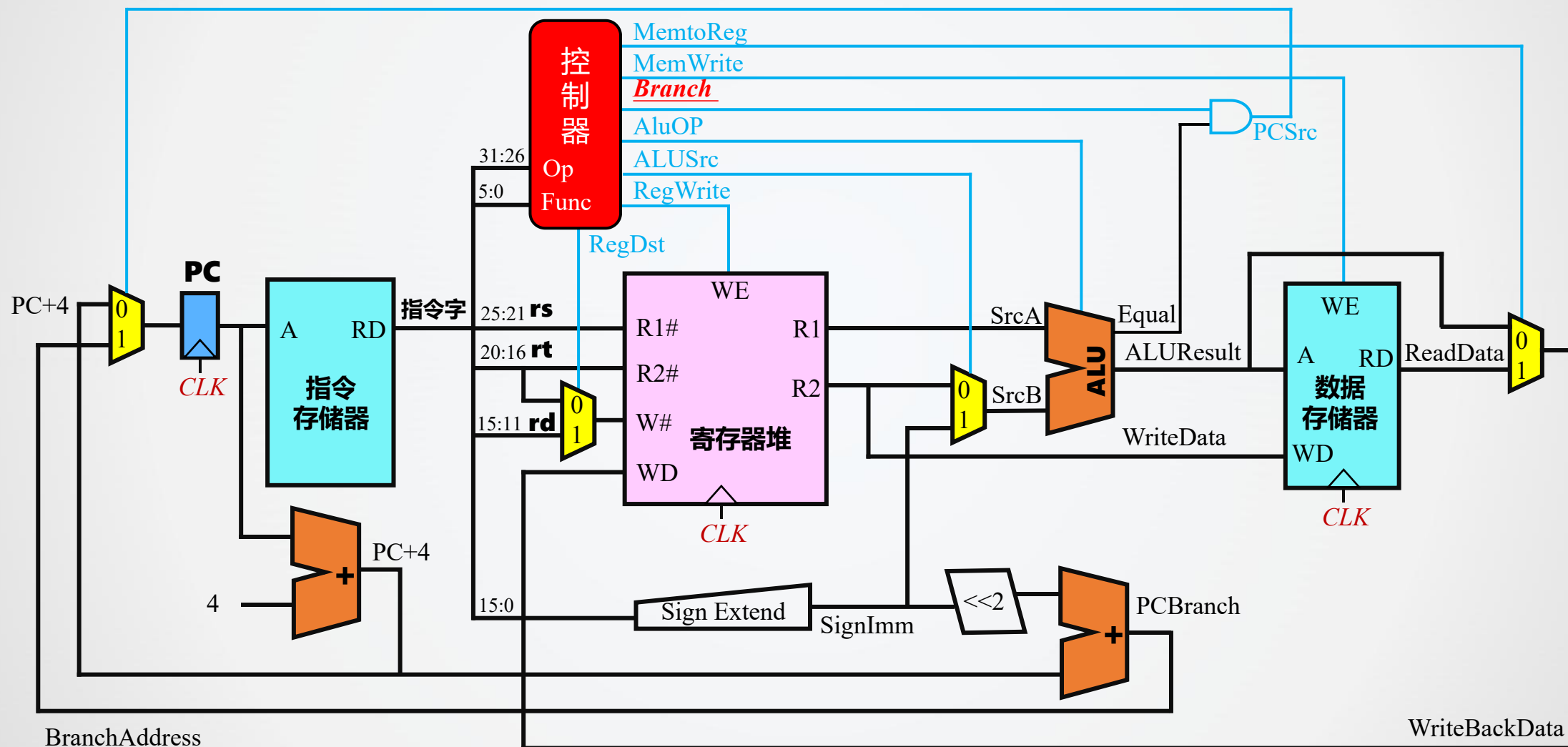
MIPS单周期CPU设计实验目标与任务

- 理解单周期**MIPS CPU**基本原理
 - 能在**Logisim**平台中设计实现单周期**MIPS CPU**
 - **8**条核心指令，能运行 冒泡排序测试程序。

可实现内存区域冒泡排序

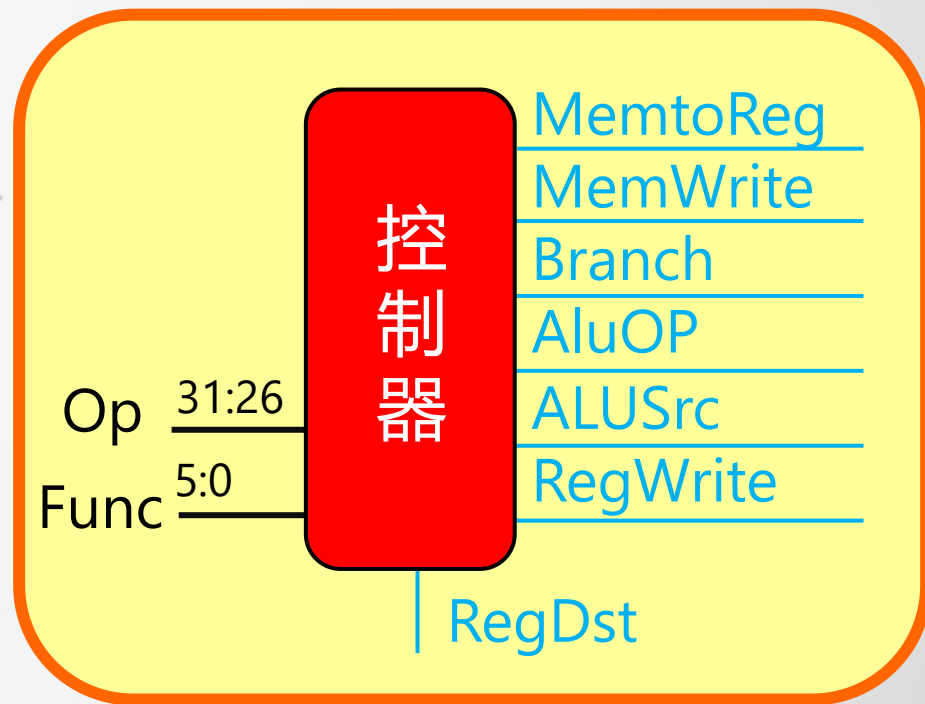
#	MIPS指令	RTL功能描述
1	<code>add \$rd,\$rs,\$rt</code>	$R[\$rd] \leftarrow R[\$rs] + R[\$rt]$ 溢出时产生异常, 且不修改 $R[\$rd]$
2	<code><i>slt \$rd,\$rs,\$rt</i></code>	$R[\$rd] \leftarrow R[\$rs] < R[\$rt]$ 小于则置1。有符号比较
3	<code>addi \$rt,\$rs,imm</code>	$R[\$rt] \leftarrow R[\$rs] + \text{SignExt}_{16b}(\text{imm})$ 溢出产生异常
4	<code>lw \$rt,imm(\$rs)</code>	$R[\$rt] \leftarrow \text{Mem}_{4B}(R[\$rs] + \text{SignExt}_{16b}(\text{imm}))$
5	<code>sw \$rt,imm(\$rs)</code>	$\text{Mem}_{4B}(R[\$rs] + \text{SignExt}_{16b}(\text{imm})) \leftarrow R[\$rt]$
6	<code>beq \$rs,\$rt,imm</code>	if($R[\$rs] = R[\$rt]$) $PC \leftarrow PC + \text{SignExt}_{18b}(\{\text{imm}, 00\})$
7	<code><u>bne</u> \$rs,\$rt,imm</code>	if($R[\$rs] \neq R[\$rt]$) $PC \leftarrow PC + \text{SignExt}_{18b}(\{\text{imm}, 00\})$
8	<code><u>syscall</u></code>	系统调用, 这里用于停机(<code>halt</code>)

构建单周期MIPS数据通路



单周期MIPS控制器 设计

- 单周期控制器: 无时序逻辑, 纯组合逻辑电路
- 输入信号: 指令字**Opcode**, **Func**字段 (**12**位)
- 输出信号: 所有控制信号
- 控制器设计的方法: 真值表→逻辑表达式→逻辑电路
(可利用**Logisim**自动生成电路。)



本章主要内容

- CPU的组成和功能
- 数据通路
- 指令周期与时序
- 硬布线控制器设计
- MIPS单周期CPU设计及实验
- 微程序控制器设计。



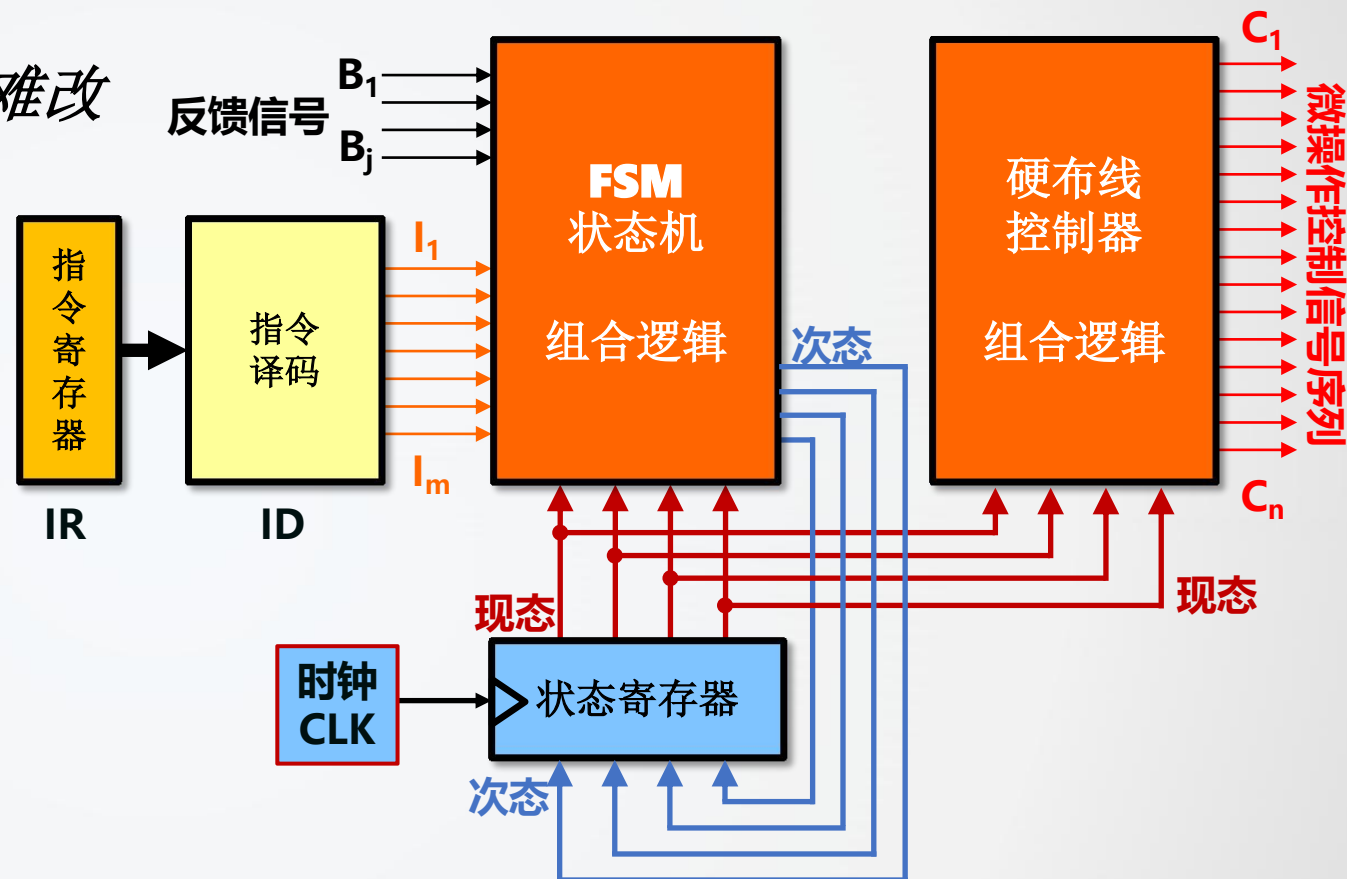
微程序控制器 基本思想

■ 硬布线：同步时序逻辑、繁，快，贵，难改

- ◆ 一般一条指令多个时钟周期
- ◆ 一个时钟周期一个(时序逻辑)状态
- ◆ 一个状态对应一组并发信号。

■ 微程序控制器：

- ◆ 将并发信号事先存储为微指令
- ◆ 一条指令对应多条微指令
- ◆ 状态字 等同于微指令地址。



机器指令字 → 控制器信号序列

微程序控制器工作原理

- 微程序是*利用软件方法*来设计硬件的技术
- 将完成指令所需的*控制信号*按格式编写成*微指令*，存放*到微程序控制存储器*中
 - ◆ 一条*(机器)指令*对应一段*微程序*（含多条*微指令*）
 - ◆ 指令取指执行 → 微程序的执行 → 执行多条微指令 → 依次生成控制信号
- *存储技术*和*程序设计*相结合，回避了复杂的同步时序逻辑设计。

(回顾) 单总线结构CPU

对于**单总线**结构CPU，以下描述正确的是（ ）。

A

CPU中的运算器、控制器、寄存器堆等核心部件均由一条内部的公共总线连接起来。

B

这条内部的公共总线称为CPU的系统总线。

C

某一时刻只能有一个模块向总线发送数据。

D

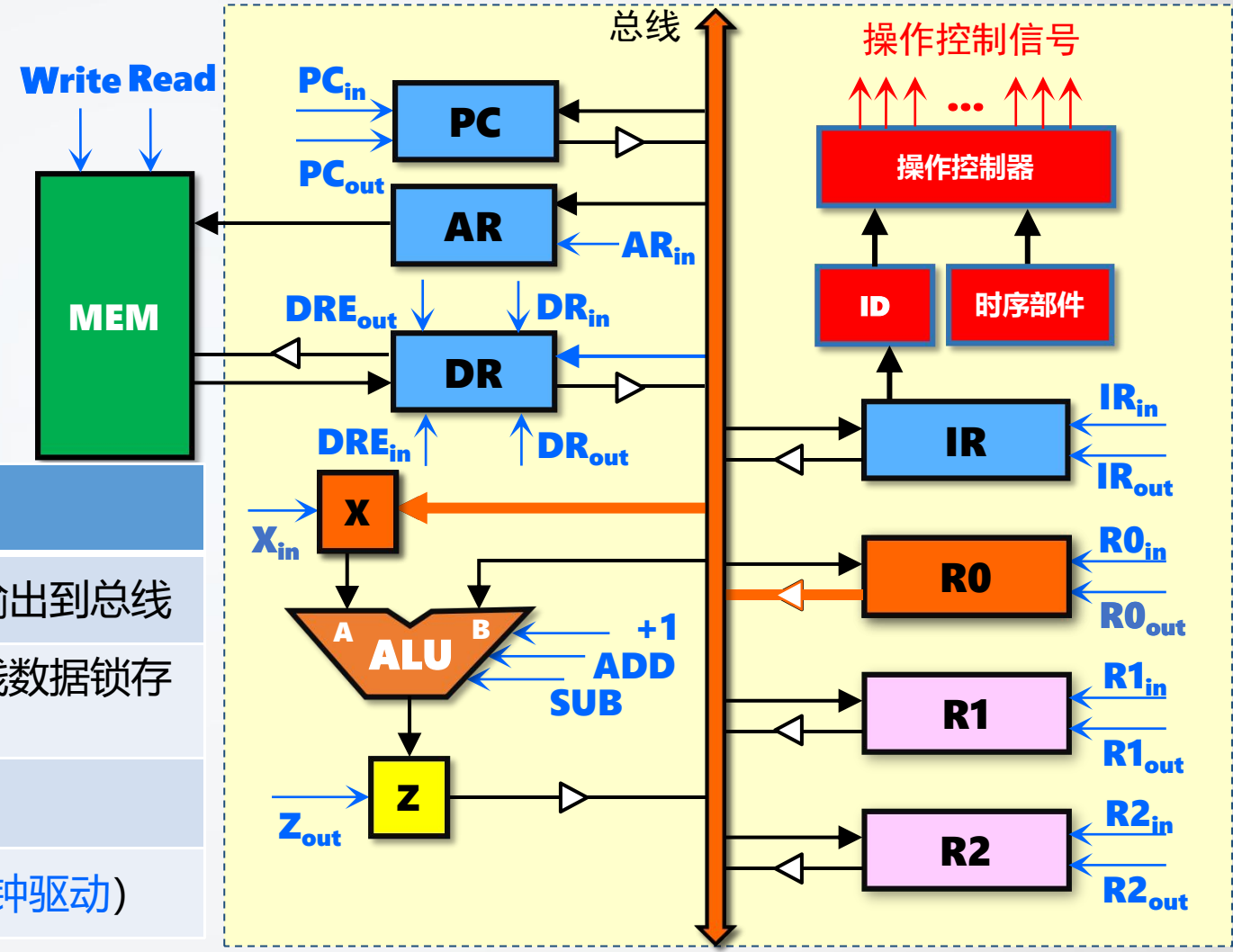
各执行部件间通过总线进行信息的传输。

提交

单总线结构CPU

- 主要部件都连接在总线上
- 各部件间通过总线进行传输

控制信号	作用说明
IR_{out} 、 PC_{out} 、... $R1_{out}$	控制三态门将寄存器值输出到总线
IR_{in} 、 PC_{in} 、... $R1_{in}$	控制寄存器使能端将总线数据锁存 (时钟驱动)
$+1$ 、 ADD 、 SUB	运算控制信号
$Write$ 、 $Read$	内存读写控制信号 (时钟驱动)



—▶: 数据流 —▶: 控制流

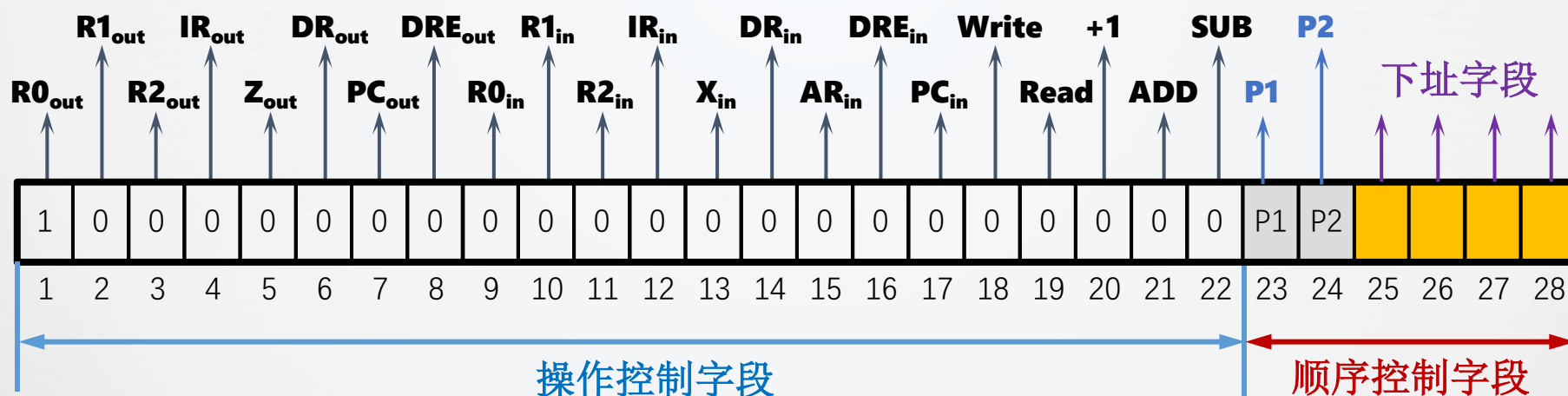
单总线CPU 微指令构造

■ **操作控制字段:** 用于存储 微操作控制信号

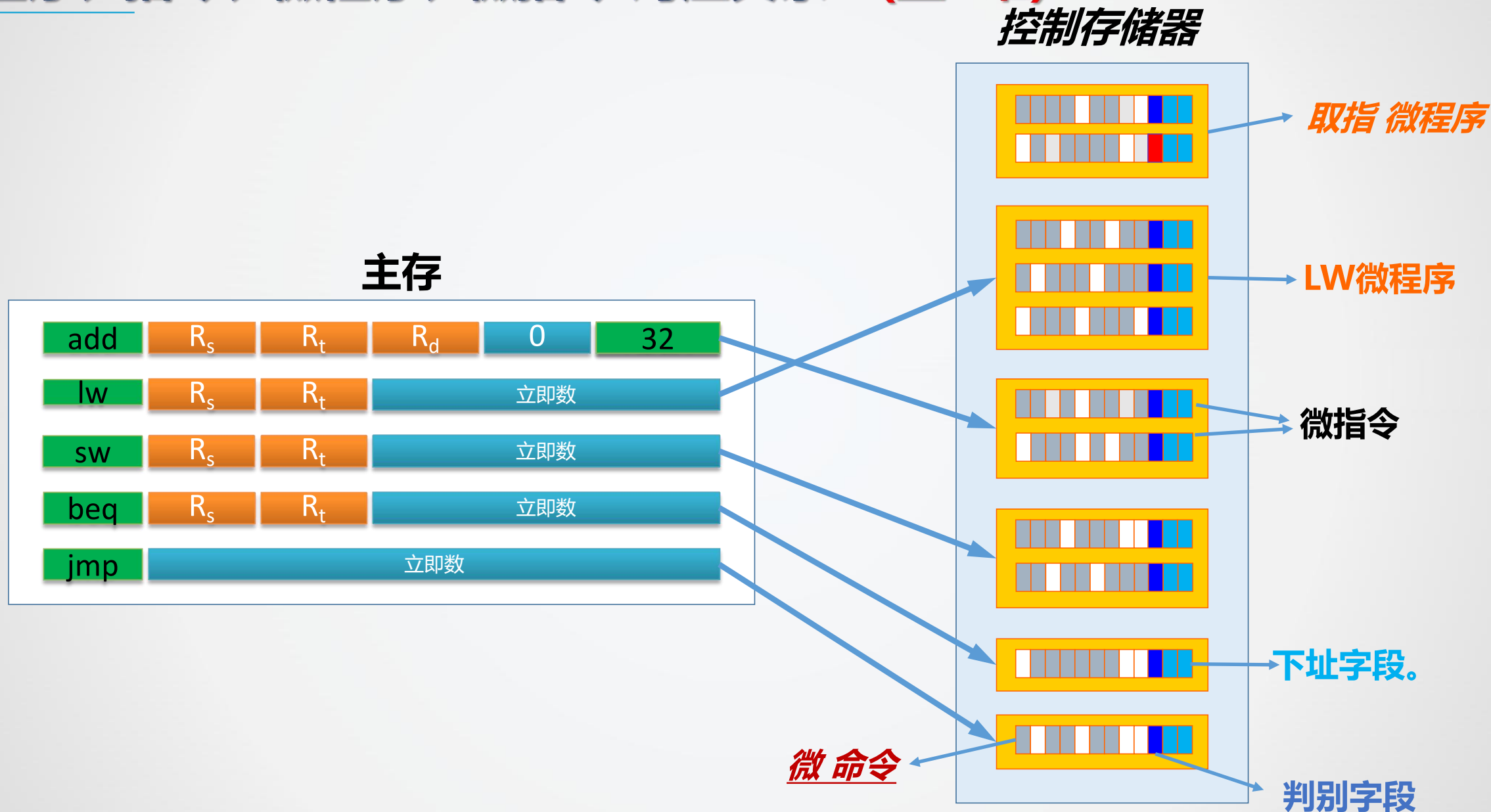
◆ 每**1**位对应**1**个控制信号, 也称 **微命令**。可**同时(并发执行地)** 给出多个微操作信号

■ **顺序控制 字段:** 用于控制微程序的**执行顺序**

◆ 判别字段**P1**、**P2** 值都为零时, 下一条微指令地址从 **下址字段** 获取; 否则按约定规则生成下一条微指令地址。



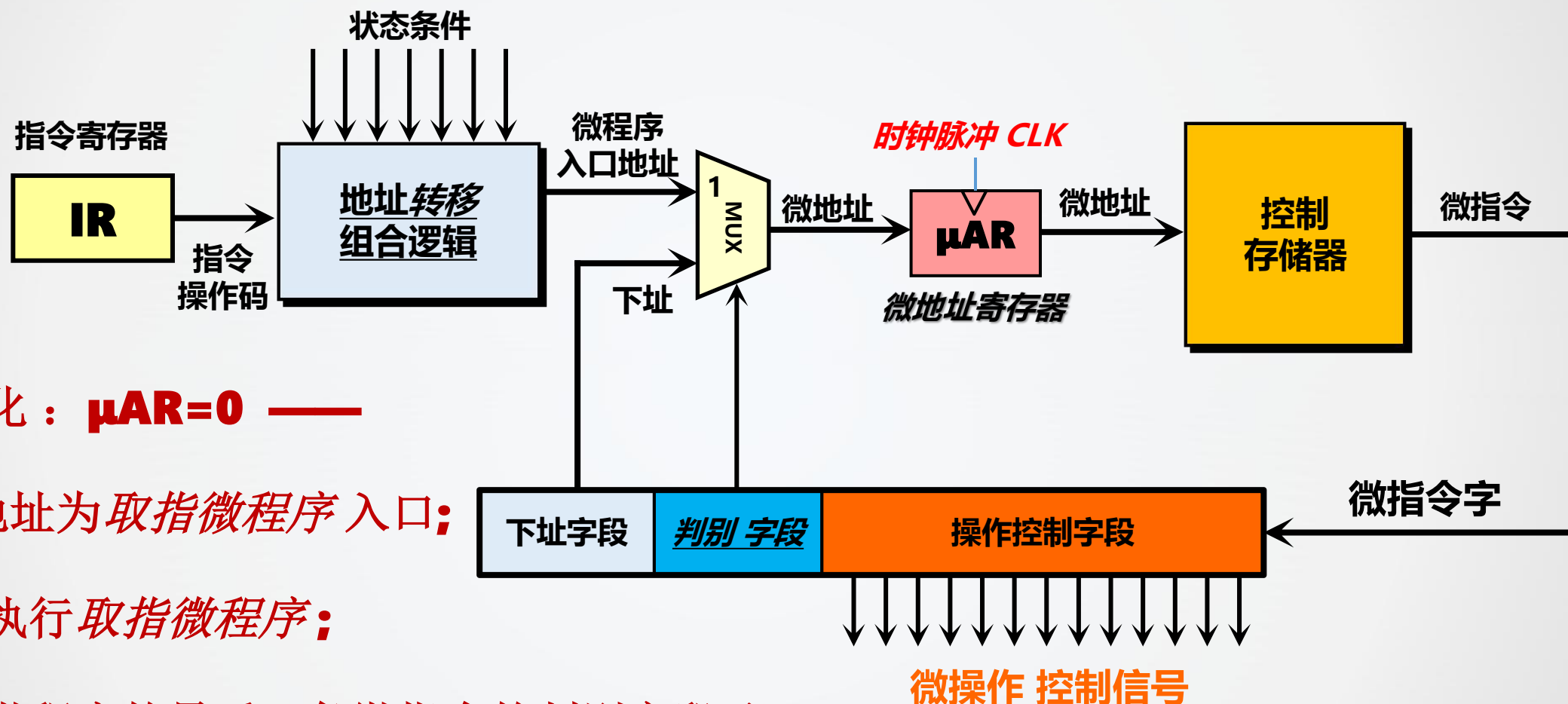
程序、指令、微程序、微指令对应关系: (重+难)



控制存储器

- 用于存放 **指令系统中全部指令 对应的所有微程序**。
- 控存 与 主存 的差异？
 - **存放内容不同：主存 存放当前运行的 程 序与数据**
 - **字长不同：控存字长与微指令长度有关，容量与微程序规模有关**
 - **控存多为非易失的ROM；主存是RAM**
 - **访问速度比主存快。**

微程序控制器 组成原理框图



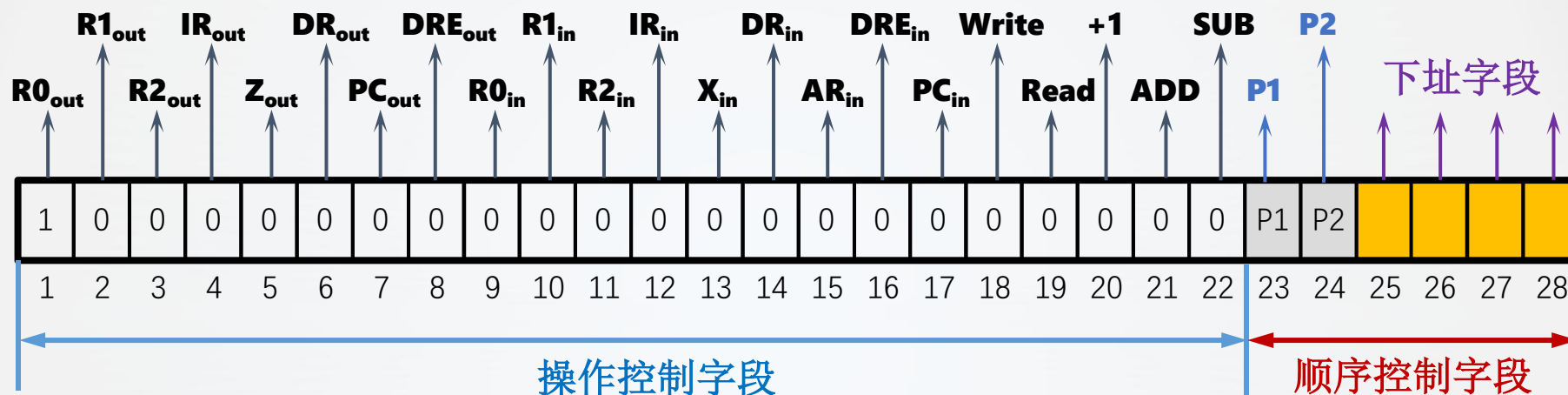
- 初始化： $\mu AR=0$ ——
- 0号地址为取指微程序入口；
- 顺序执行取指微程序；
- 取指微程序的最后一条微指令的判别字段 必为非零，使地址跳转。

微程序控制器(与硬布线控制器相比) 优势与劣势

- 访(控)存频繁、速度慢；成本低廉
- 设计规整，设计简单，易于修改、扩展指令系统的功能
 - ◆ 适合**CISC**等功能较复杂的系列机：**X86、IBM S/360**等
 - ◆ 可写 控存方便，修复出厂故障：**Intel Core 2 、 Intel Xeon**
- **硬布线控制器**：执行速度快，但设计复杂，代价较高，不便于修改
 - 适合**RISC**计算机，如**MIPS， ARM**。



微指令格式



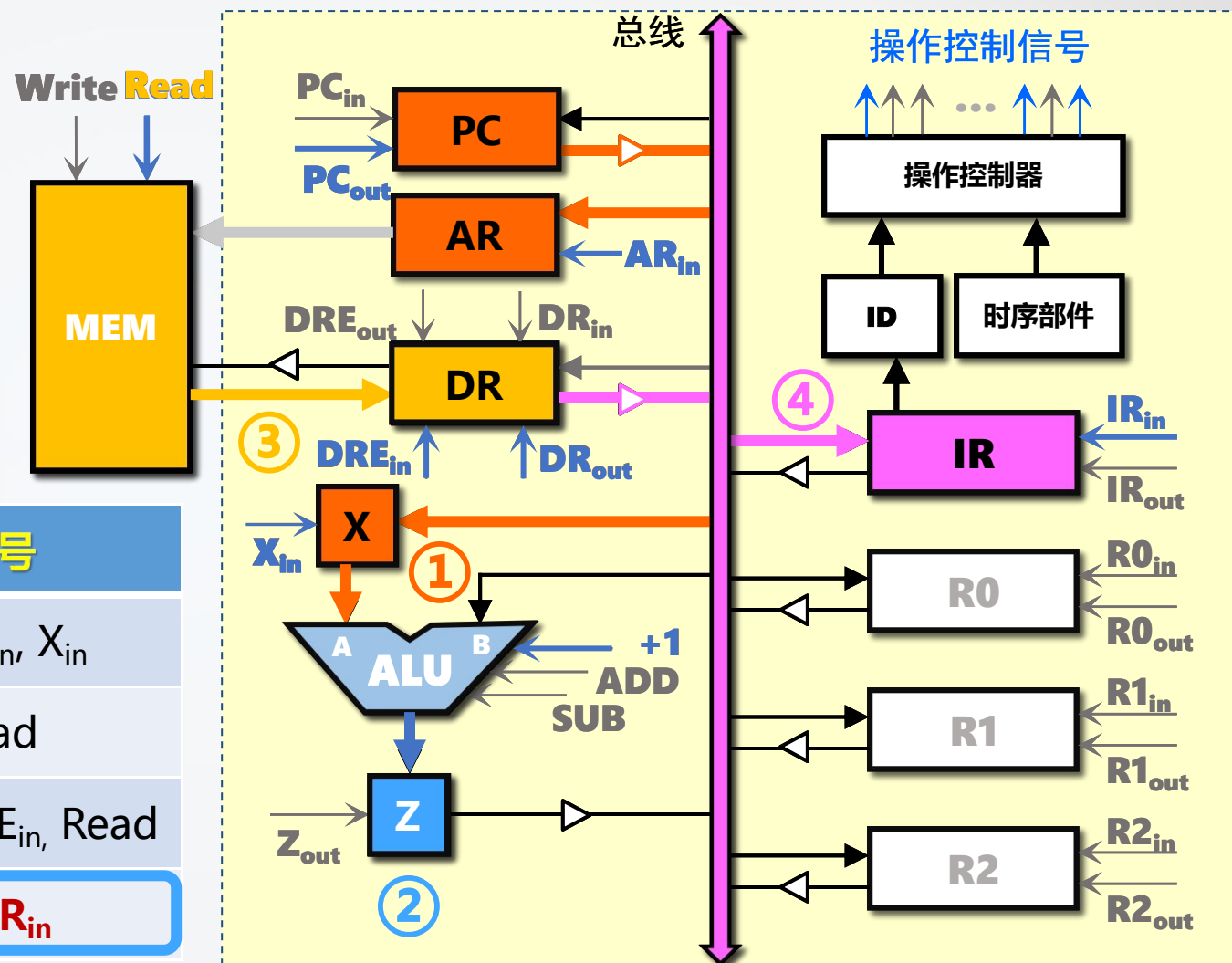
- 一条微指令对应一个时钟周期；微指令操作控制字段的控制信号在该时钟周期内有效。
- 一条(机器)指令的执行需要多少时钟周期就包括多少条微指令。

(回顾) 取指令数据通路

Mem[PC++] → IR

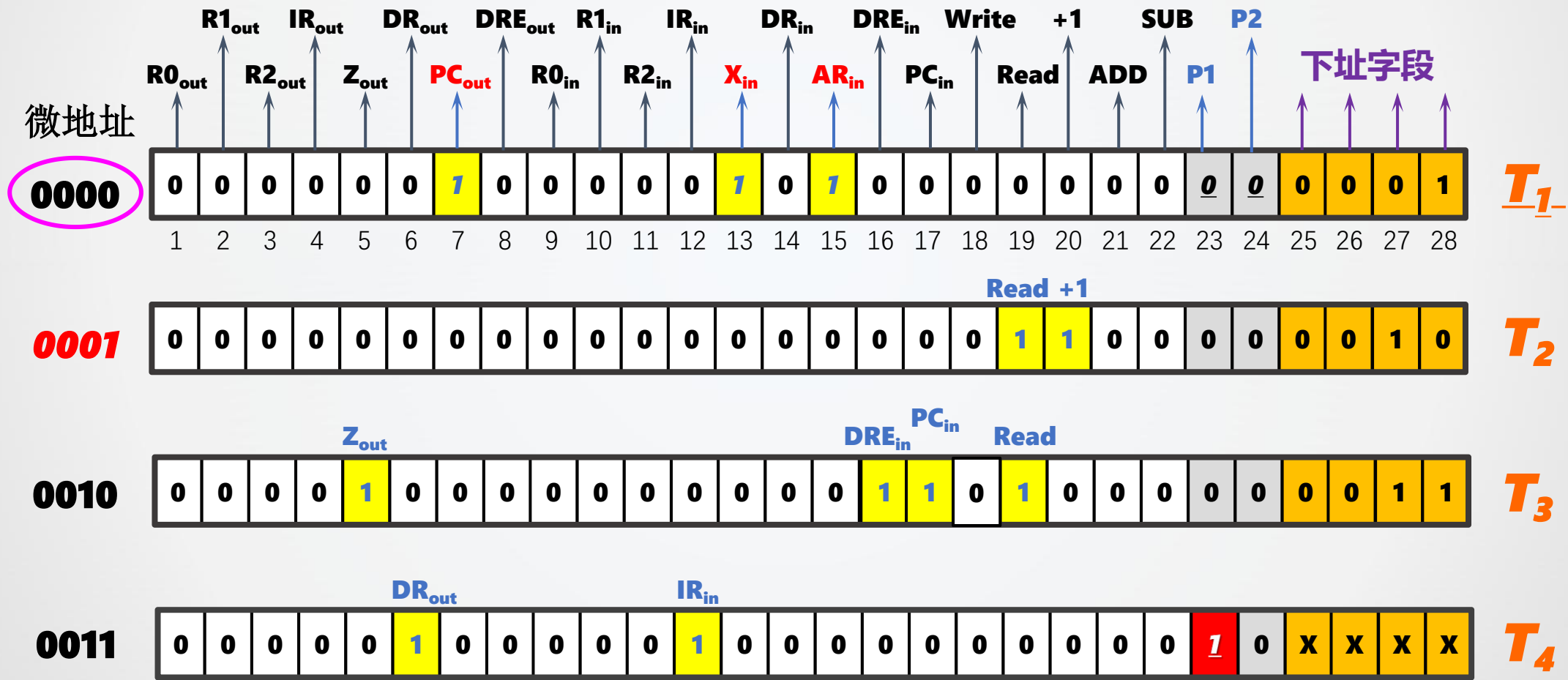
- 4个时钟周期；
- 4条微指令。

节拍	数据通路	控制信号
T1	(PC)→AR, (PC)→X	PC _{out} , AR _{in} , X _{in}
T2	(X)+1→Z	+1, Read
T3	(Z)→PC, Mem[AR]→DR	Z _{out} , PC _{in} , DRE _{in} , Read
T4	(DR)→IR	DR_{out}, IR_{in}



取指令微程序

节拍	取指令数据通路	控制信号
T1	(PC)→AR, (PC)→X	PC _{out} , AR _{in} , X _{in}
T2	(X)+1→Z	+1, Read
T3	(Z)→PC, Mem[AR]→DR	Z _{out} , PC _{in} , DRE _{in} , Read
T4	(DR)→IR	DR _{out} , IR _{in}

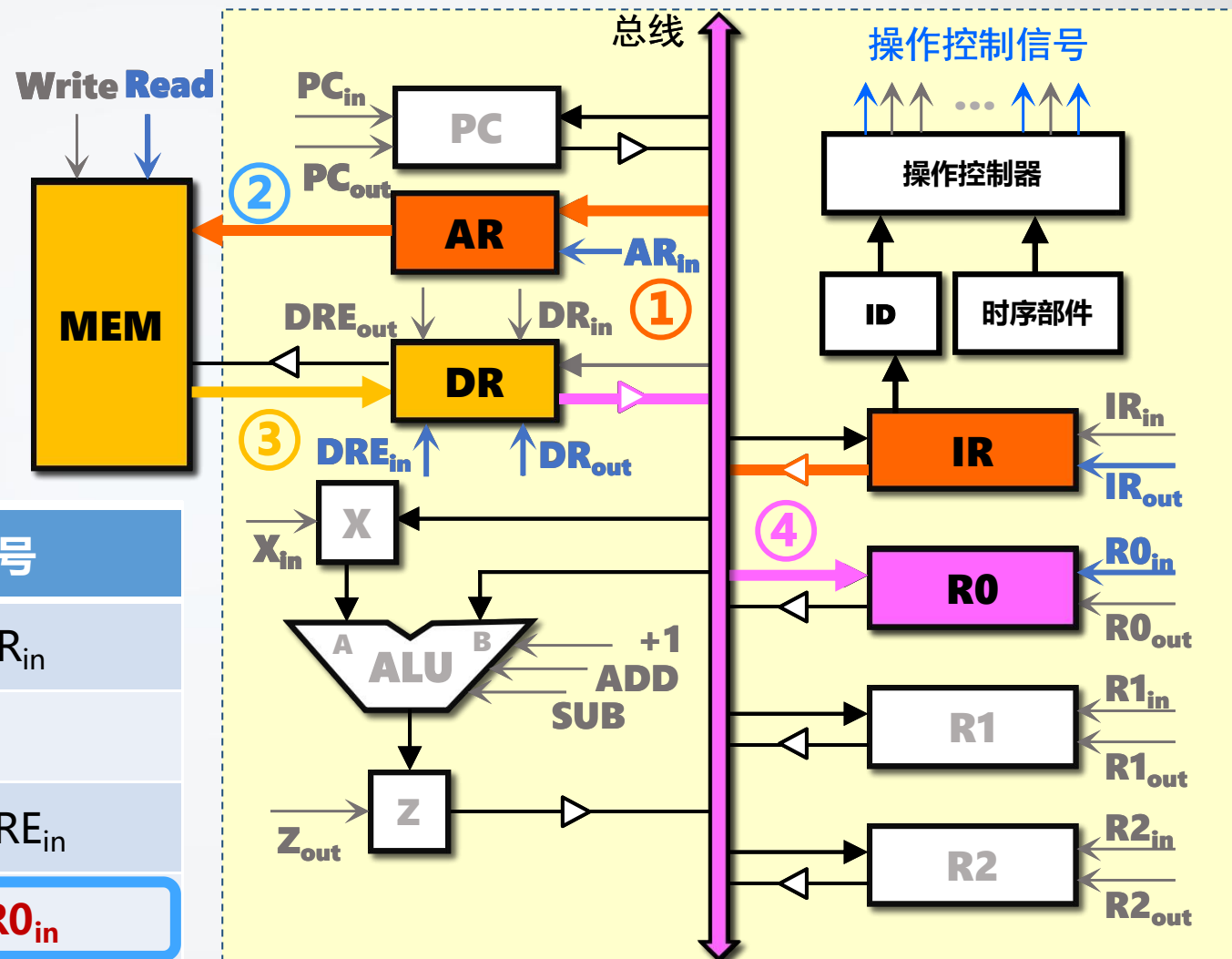


(回顾) LOAD指令执行周期 数据通路

LOAD R0,6#

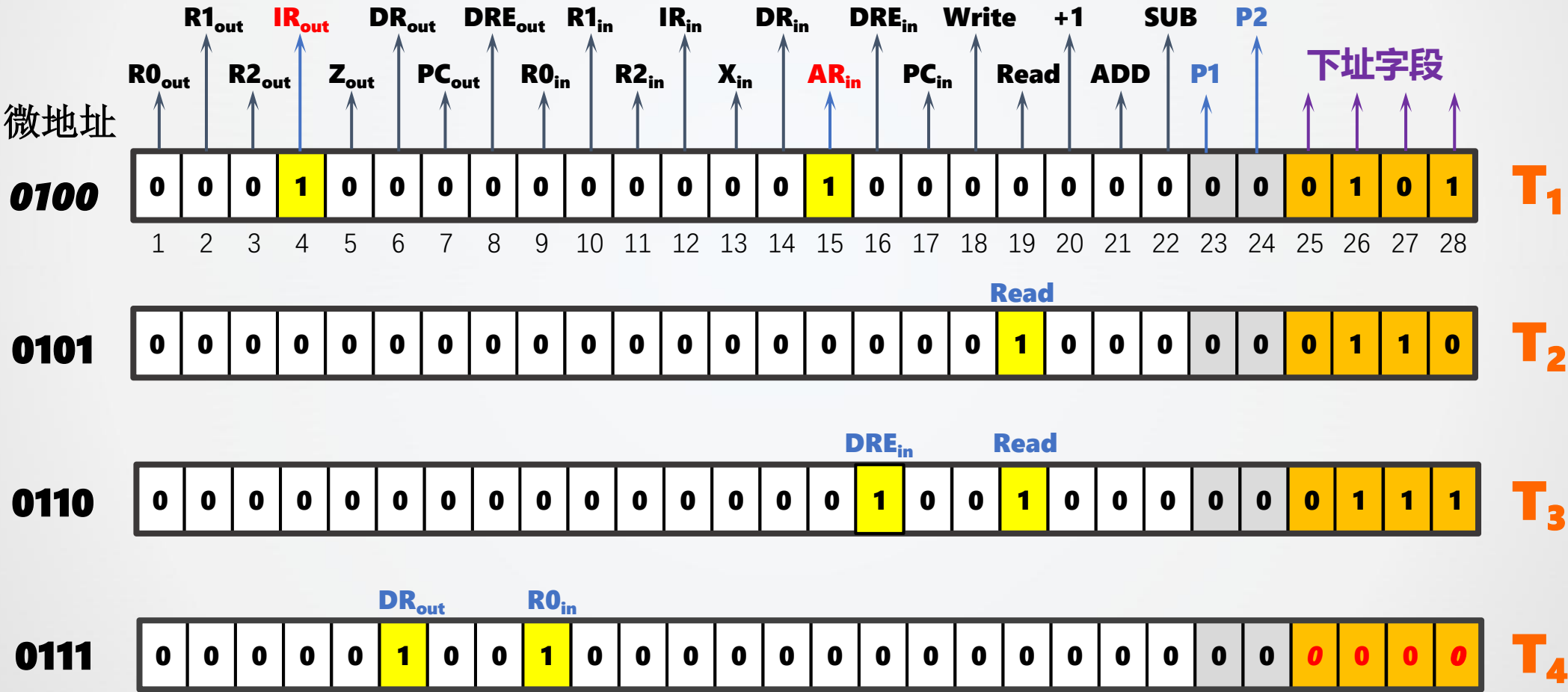
Mem[IR_A] → Reg

节拍	数据通路	控制信号
T1	(IR _A)→AR	IR _{out} , AR _{in}
T2	Read	
T3	Mem[AR]→DR	Read, DRE _{in}
T4	(DR)→R0	DR_{out}, R0_{in}



LOAD指令执行周期 微程序

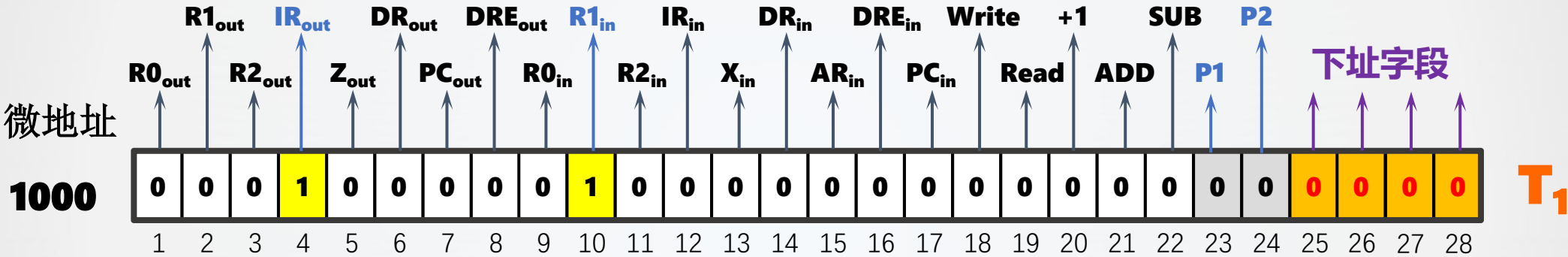
节拍	取指令数据通路	控制信号
T1	(IR _A)→AR	IR _{out} , AR _{in}
T2		Read
T3	Mem[AR]→DR	DRE _{in} , Read
T4	(DR)→R0	DR _{out} , R0 _{in}



MOVE指令 执行周期微程序

MOVE R1,10

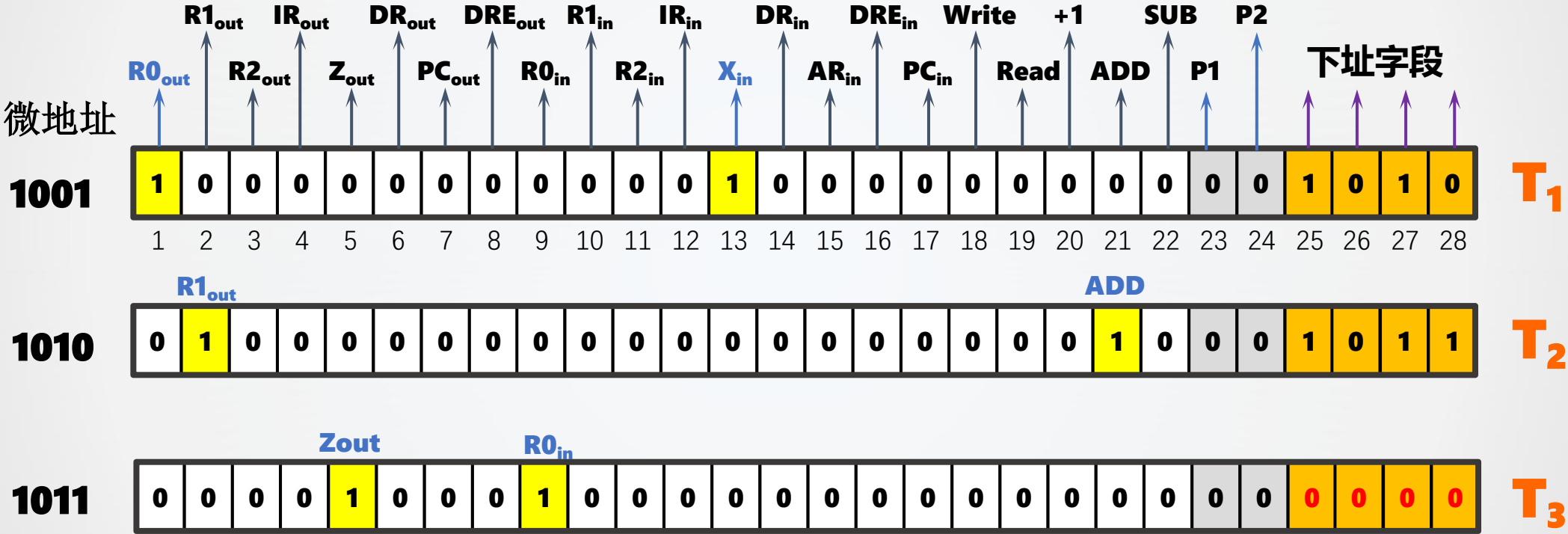
节拍	数据通路	控制信号
T1	(IR _A) → R[1]	IR _{out} , R1 _{in}
T2		
T3		
T4		



ADD指令 执行周期微程序

ADD R0,R1

节拍	数据通路	控制信号
T1	$(R0) \rightarrow X$	$R0_{out}, X_{in}$
T2	$(X) + (R1) \rightarrow Z$	$R1_{out}, ADD$
T3	$(Z) \rightarrow R0$	$Z_{out}, R0_{in}$
T4		



单总线CPU微程序

状态	微地址	操作控制字段																				顺序控制字段									
S0	0000	0	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	1	取指令微程序		
S1	0001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0			
S2	0010	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	1	1			
S3	0011	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	X		X	X
S4	0100	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	LOAD微程序
S5	0101	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	1	0		
S6	0110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	1	1		
S7	0111	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
S8	1000	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	MOVE微程序
S9	1001	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	
S10	1010	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	1	1	ADD 微程序	
S11	1011	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0
S12	1100	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1	STORE微程序
S13	1101	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	
S14	1110	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	
S15	1111	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	JMP 微程序

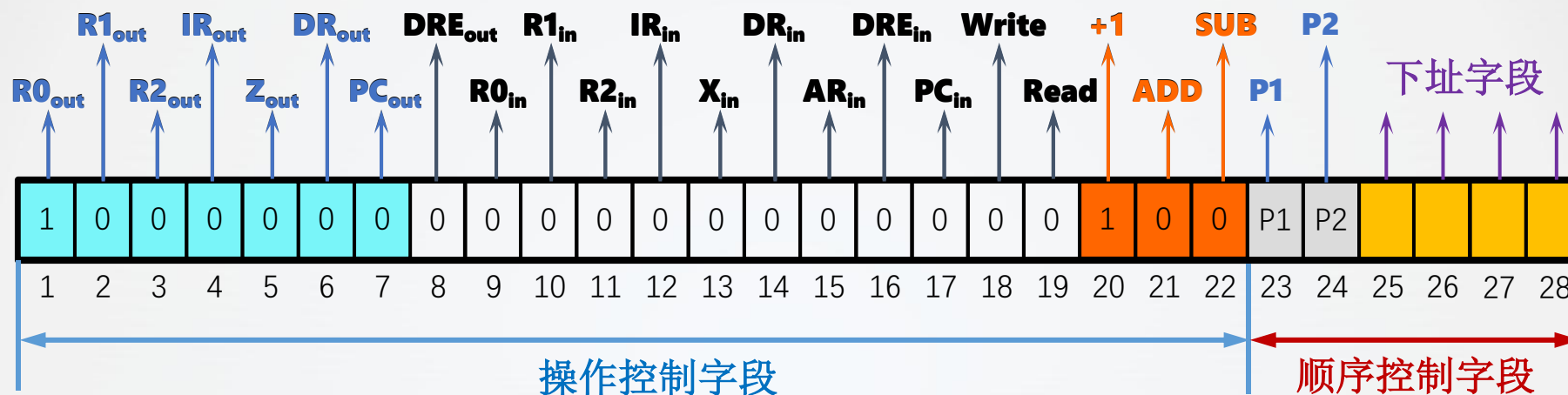
微指令设计原则

- 要有利于缩短微指令字长度
- 有利于减少控制存储器的容量
- 有利于提高微程序执行速度
- 有利于对微指令进行修改
- 有利于提高微程序设计的灵活性。

微命令 编码方式

- 直接表示法
- 编码表示法
- 混合表示法。

直接表示法

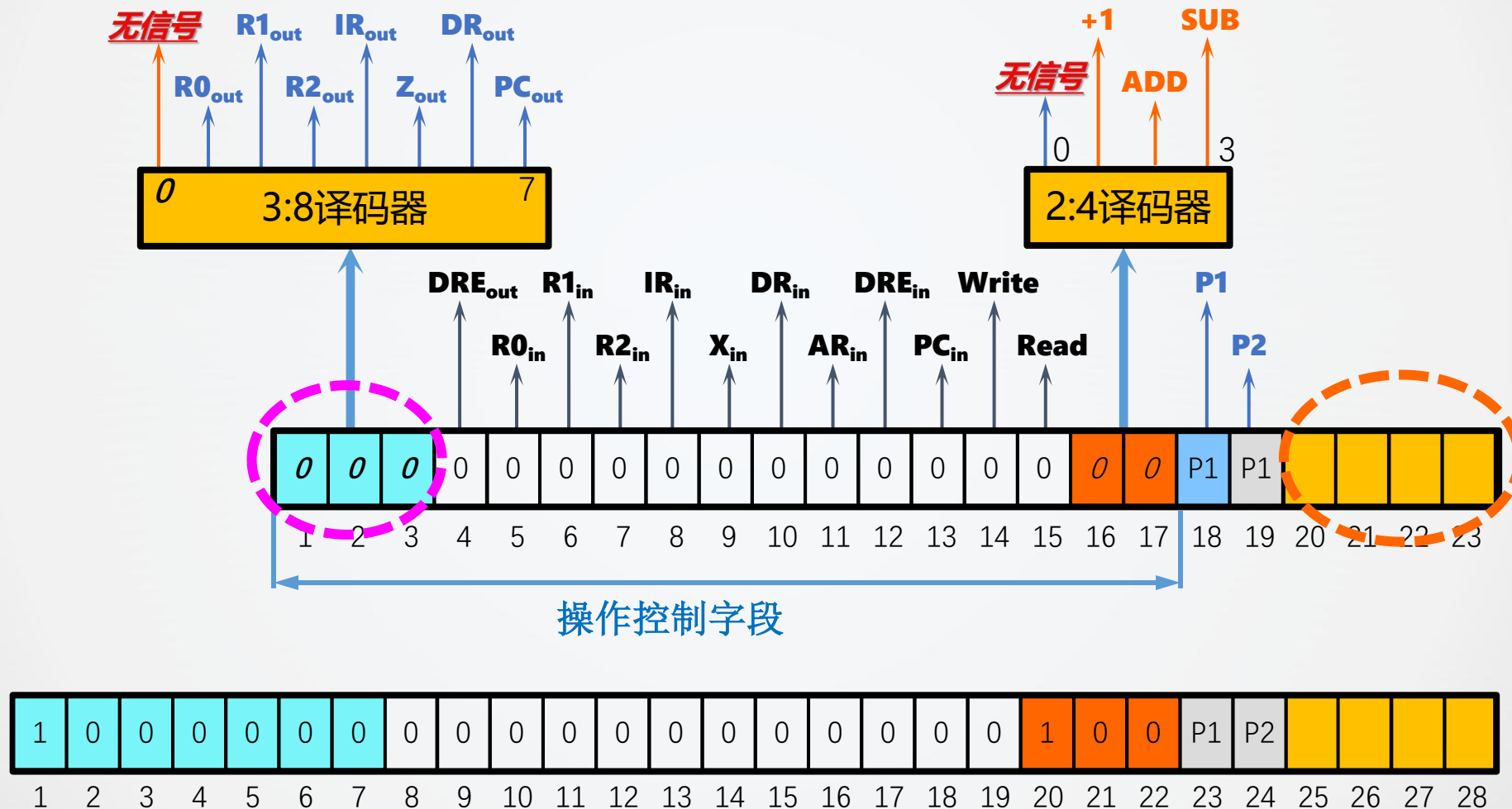


■ 简单直观, 便于输出控制; 但微指令 长度太长, 占控存容量大; 如何压缩微指令长度?

◆ 改 直接表示 为 编码表示 (压缩 互斥性(P229)微操作)

◆ 可去掉 下址字段, 采用 $\mu PC = \mu PC + 1$ 的方式生成微指令地址。

思考：如果互斥的一组输出控制信号有8个，编码后长度多少位？

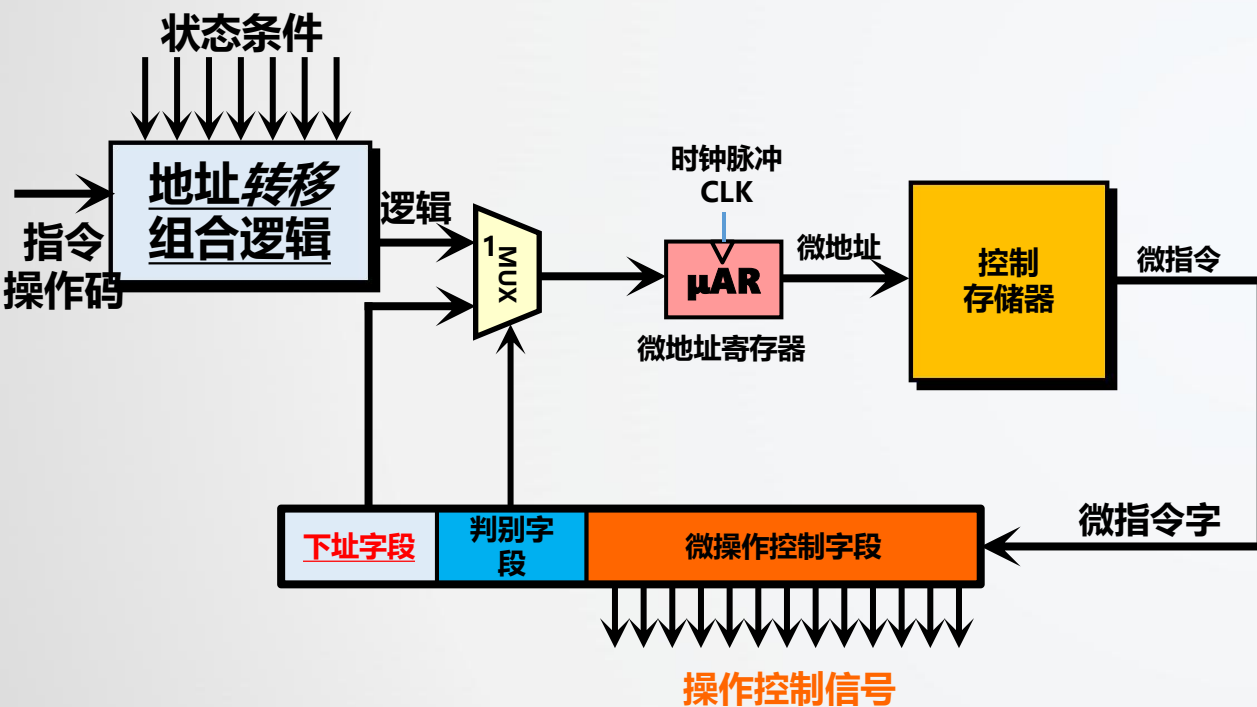




微指令地址形成方法

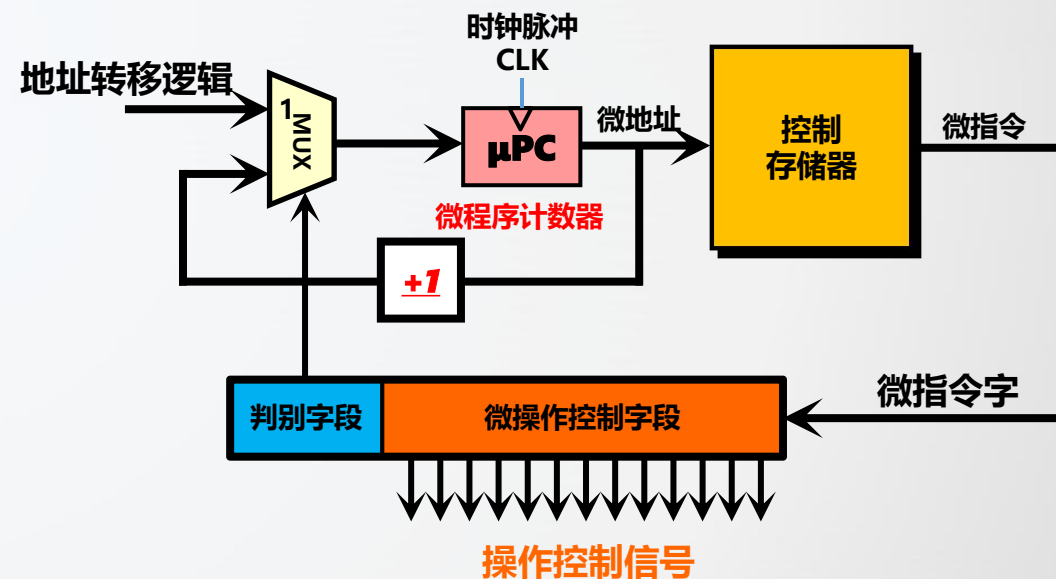
■ 下地址字段法：

■ 微指令长些，控存容量要大些



■ 计数器法 **μPC**:

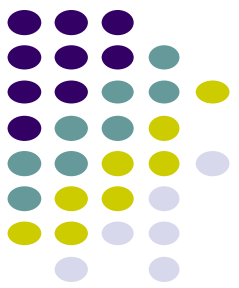
■ 微指令短，需要加法器。





微地址形成方法

- 指令的操作码 转换成微程序入口地址的方式主要有两种：
 - 计数器的方式
 - 多路转移的方式。



微地址形成方法

(1) 计数器方式（增量方式）

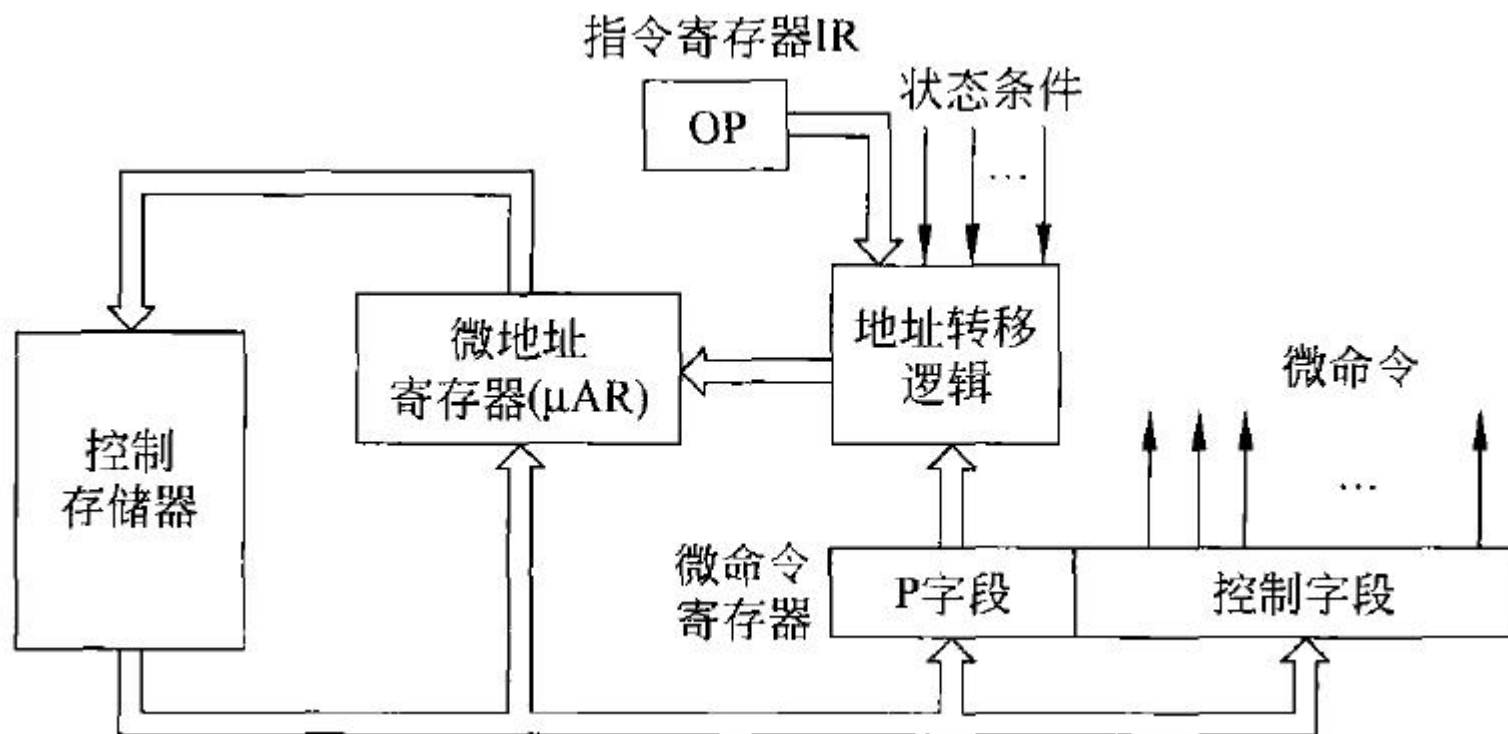
- 方法：

- 微程序*顺序执行*时，其后继微地址就是现行微地址加上一个增量(通常为1)；此时 μAR 被 μPC 代替。
- 当微程序遇到转移或转子程序时，通过微指令的判别测试字*段P*来检测；当测试条件满足时，修改 μPC 相关位 得到转移的微地址。



(2) 多路转移方式（断定方式）

- 当可能出现微程序分支(转移)时，有若干路候选微地址供选择：按顺序控制字段的“判别测试P”标志和“状态条件”信息来选择其中一个微地址，如下图：





微指令格式

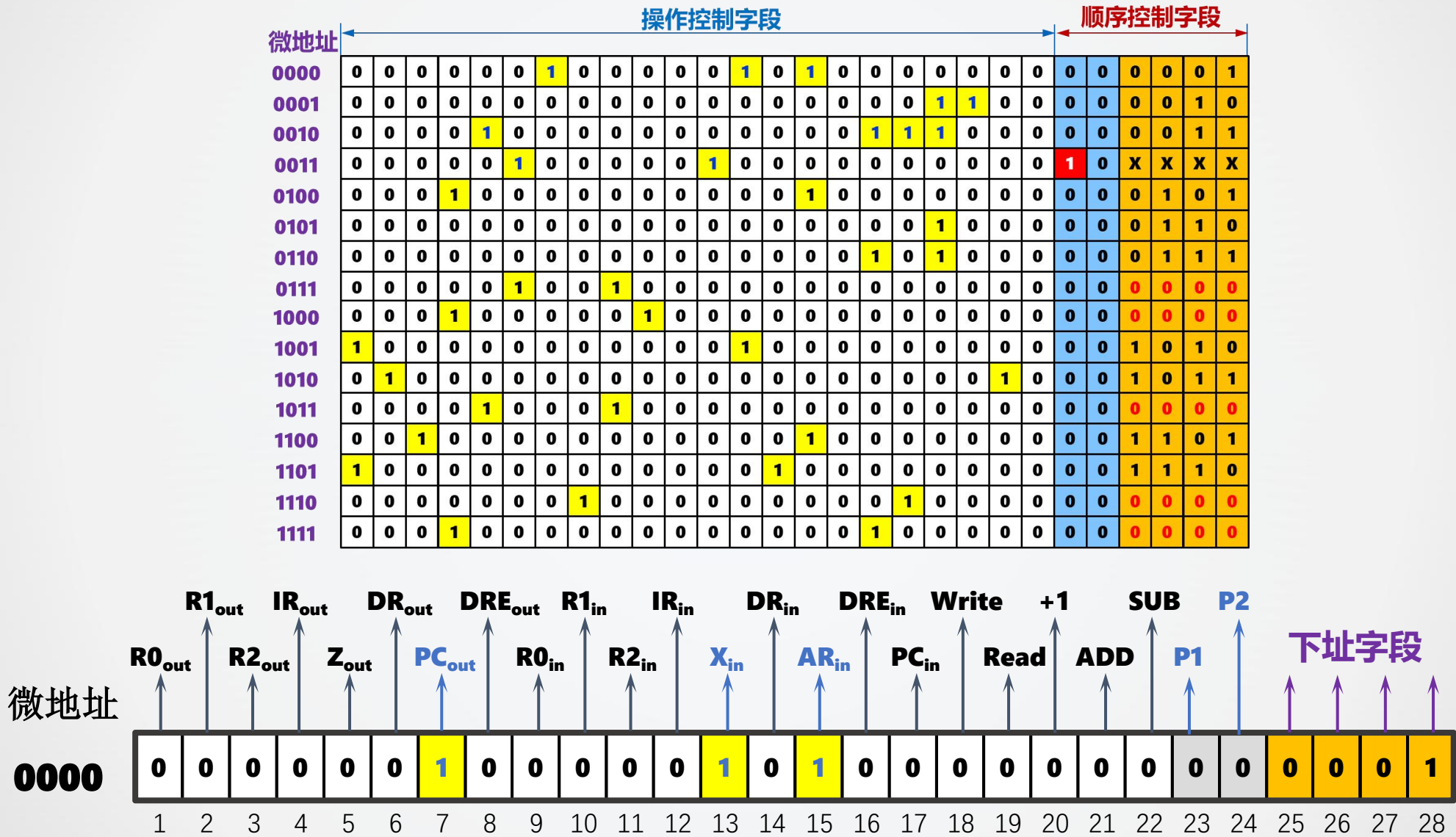
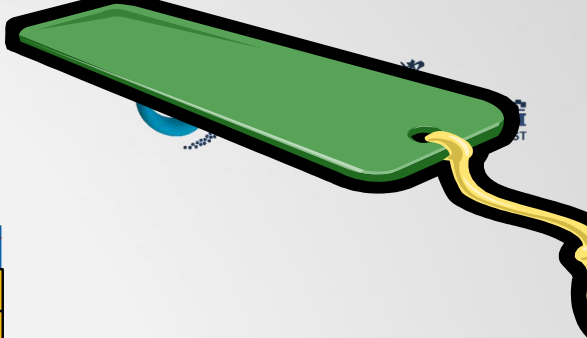
分为两类：水平型微指令 和 垂直型微指令.

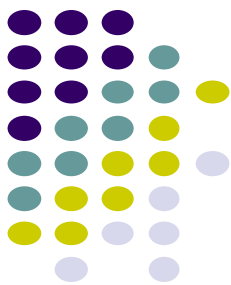
(1) 水平型微指令

- 水平型微指令是指一次能定义且能并行执行多个微命令的微指令.
- 前面介绍的微操作控制字段采用直接表示法、编码表示法的微指令都属于水平型微指令.
- 格式如下:

控制字段	判别测试字段	下地址字段
------	--------	-------

水平型微指令编码效率

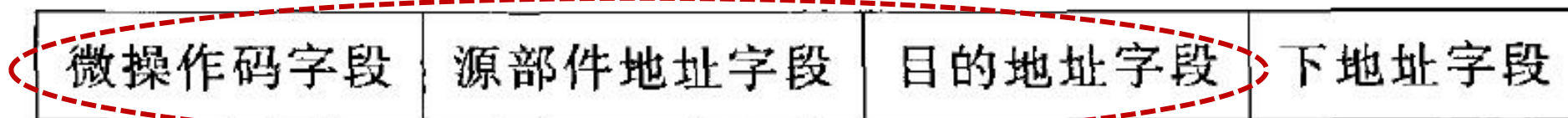




微指令格式

(2) 垂直型微指令：采用微操作码编码方式。

- 微指令中设置微操作码字段时，采用微操作码编译法，由微操作码规定微指令的功能，一次只能执行1~2个微命令的微指令称为垂直型微指令。
- 这种微指令类似于机器指令，其一般格式为：



垂直型微指令

如用扩展操作码:OP只需2位; 共需8位字长

■ 垂直型微指令按 数据通路分类 : (假设单总线CPU)

- ◆ 寄存器传输 MOV reg1, reg2 例 MOV AR, PC
- ◆ 运算类型 ALUOp reg INC (ADD) X
- ◆ 访存类型 Mem LOAD、STORE
- ◆ 执行顺序控制 Branch Branch P1

MOV	目的寄存器(3)	源寄存器(3位)
INC	无操作数	
ADD	源寄存器编号(3位)	
Mem	目的数	源数
Branch	判别 条件	

回顾:

节拍	取指令	LOAD	MOVE	ADD	STORE	JMP
T1	(PC)→AR, (PC)→X	(IR _A)→AR	(IR _A) → R[0]	(R0)→X	(R2)→AR	(IR _A)→PC
T2	(X)+1→Z, ReadMem	ReadMem		(X)+(R1)→Z	(R0)→DR	
T3	(Z)→PC, Mem[AR]→DR	Mem[AR]→DR		(Z)→R0	(DR)→Mem[AR]	
T4	(DR)→IR	(DR)→R0				

垂直型微指令实例

LOAD R0,6#

Mem[IR_A] → Reg

LOAD指令的微程序 容量:

■ 水平型: 直接表示法 + 下址字段

◆ **28位 * 8条 = 224位**

■ 水平型: 编码表示法 + **μPC**

◆ **19位 * 8条 = 152位**

■ **垂直型微指令:**

◆ **8位 * 13条 = 104位**

◆ 微指令容量最小, 但速度慢 (时钟节拍变多), 微指令执行还需译码电路。

节拍	LOAD指令	节拍	垂直型微指令	信号
<u>T1</u>	(PC)→AR, (PC)→X	C1	MOV AR,PC	PC _{out} , AR _{in}
		C2	MOV X,PC	PC _{out} , X _{in}
<u>T2</u>	(X)+1→Z, ReadMem	C3	LOAD	DRE _{in} ,Read
		C4	LOAD	DRE _{in} ,Read
		C5	INC	+1
T3	(Z)→PC, Mem[AR]→DR	C6	MOV PC,Z	Z _{out} , PC _{in}
T4	(DR)→IR	C7	MOV IR,DR	DR _{out} , IR _{in}
		C8	Branch P1	P1
T5	(IR _A)→AR	C9	MOV AR,IR	IR _{out} ,AR _{in}
T6	ReadMem	C10	LOAD	DRE _{in} ,Read
T7	Mem[AR]→DR	C11	LOAD	DRE _{in} ,Read
T8	(DR)→R0	C12	MOV R0, DR	DR _{out} , R0 _{in}
		C13	Branch P1	P1

水平型微指令与垂直型微指令小结

■ 水平型微指令

- ◆ 并行操作能力强，效率高，执行时间短、灵活性强
- ◆ 微指令字较长，微程序短(含微指令数少)，占控存容量大；性能佳。

■ 垂直型微指令

- ◆ 字长短，微程序长(含微指令数多)，占控存容量小；性能差
- ◆ 垂直型与(机器)指令相似，易于掌握
- ◆ 基本被 淘汰。

- CPU由运算器和控制器构成；控制器根据指令的要求控制各执行功能部件按时间先后次序进行相应的操作，从而形成相应的数据通路，最终实现指令的功能。
- 计算机能够执行程序，依赖于控制器、运算器的精妙配合，缺一不可；
- 就像团队项目中的各成员和整个团队的关系，大家注意提高团队合作精神！

作业 2

- 在试题书上完成5.1选择题: 1,2,3,6,8,9,10,11,18,19,21.
- 本子上完成:
- 1、教材P248习题6.3 (7)(8)
- 2. 试题书5.3设计题: 2 (去掉题目中要求的 “寄存器打入信号须注明时钟信号”)。提示: “CPU周期” 即机器周期 .
- 3、试题书P84: 5 (注意图5.18: 每个相斥类中都应加上“ 无信号” ; 判别字段: 要注意, 全0表示“ 不判别测试”)
- 本周内会收QQ作业!

- 结业考试好好复习！预祝成功！
- “理论课件” 文件夹中所有PPT就是考试范围。

The End